



Σavante

Repositorios: estructura y actualización. Generación de código y documentación. Metodologías de desarrollo. Pruebas. Programas para control de versiones. Plataformas de desarrollo colaborativo de software

ÍNDICE

1. El ciclo de vida de los sistemas de información	5
1.1. Concepto de sistema de información y ciclo de vida	5
1.2. Fases del ciclo de vida de los sistemas de información	5
1.3. Relación entre el ciclo de vida del sistema de información y el ciclo de vida del software	6
1.4. Relación entre el ciclo de vida del sistema de información y la gestión de proyectos TI	7
2. Repositorios	7
2.1. Conceptos de tipos de repositorios	10
2.1.1. Repositorios de metadatos	10
2.1.2. Data Mart	11
2.1.2.1. OLAP	11
2.1.2.1.1. Cubo OLAP	13
2.1.2.2. Tipos de Arquitecturas OLAP	16
2.1.2.3. Comparativa de tipos de Arquitecturas OLAP	18
2.1.3. Data Warehouse	20
2.1.4. Data Lakes	25
2.1.4.1. Esquema de Estrella	26
2.1.4.1.1. Tabla de hechos	27
2.1.4.1.2. Tabla de Dimensiones	28
2.1.4.2. Esquema en copo de nieve	28
3. Generación de código y documentación. Herramientas CASE	30
3.1. Clasificación de Herramientas CASE	35
3.1.1. Según la actividad que realizan	36
3.1.2. Según su amplitud	39
3.1.3. Según el ciclo de vida del software que cubren	39
3.1.3.1. U-CASE	40
3.1.3.2. M-CASE	43
3.1.3.3. L-CASE	47
3.1.3.3.1. Pruebas de análisis de código	48
3.1.3.3.2. Pruebas de integración	52

3.1.3.3.3. Pruebas de artefactos. (Diseño de software)	53
3.1.3.3.4. Pruebas de implantación y aceptación del sistema	53
3.1.4. Otras clasificaciones	55
3.2. Generar documentación	56
3.2.1. Acuerdo de Nivel de Servicio (ANS)	56
3.3. Herramientas CASE más utilizadas	57
3.3.1. Herramientas de Análisis de Seguridad	61
4. Metodologías de desarrollo	62
4.1. Gestión de proyectos de Tecnologías de la Información (TI)	65
4.1.1. Definición y características de los proyectos TI	65
4.1.2. Ciclo de vida del proyecto: inicio, planificación, ejecución, control y cierre	65
4.1.3. Metodologías de gestión de proyectos	66
4.1.4. Herramientas de gestión de proyectos	66
4.2. Modelos de desarrollo	67
4.2.1. En Cascada	67
4.2.1.1. Ventajas e inconvenientes de los Modelos en Cascada	68
4.2.2. Modelo Espiral	69
4.2.2.1. Ventajas e inconvenientes del Modelo en Espiral	74
4.2.3. Desarrollo ágil de software	74
4.2.3.1. Marco de trabajo Scrum	77
4.2.3.1.1. Características, metodología y roles	78
4.2.3.1.2. Funcionamiento	80
4.2.3.2. Herramientas y filosofías compatibles	84
4.2.3.2.1. Kanban	84
4.2.3.2.2. Lean Software Development	85
4.2.3.3. Culturas técnicas alineadas	87
4.2.3.3.1. DevOps	87
4.2.3.3.2. DevSecOps	89
4.2.3.4. Beneficios	91
4.3. Capability Maturity Model Integration (CMMI)	92

5. Pruebas. SCM (Gestión de Configuración de <i>Software</i>)	94
5.1. Desarrollo Orientado a Pruebas (TDD)	95
6. Monitorización de infraestructuras informáticas	96
6.1. Métrica	97
6.1.1. Metodología métrica v3	99
6.1.1.1. Conceptos generales	100
6.2. Monitorización	103
7. Programas para Control de Versiones	104
7.1. Características	104
7.2. Clasificación	106
7.2.1. Locales	106
7.2.2. Centralizados	107
7.2.3. Distribuidos	107
7.3. Software para el Control de Versiones	108
8. Plataformas de desarrollo colaborativo de <i>software</i>	115
8.1. Principales plataformas	115
8.2. Archivos de documentación comunes en las plataformas	121
9. Bibliografía	122



1. El ciclo de vida de los sistemas de información

1.1. Concepto de sistema de información y ciclo de vida

Un sistema de información es el conjunto de elementos que permiten gestionar información dentro de una organización. Estos elementos incluyen personas, procedimientos, datos y medios técnicos, y actúan de forma coordinada para recoger, tratar, almacenar y facilitar información necesaria para el funcionamiento diario y la toma de decisiones.

En el ámbito de la Administración General del Estado, los sistemas de información son la base de la gestión administrativa, de la tramitación electrónica y de la prestación de servicios públicos. Su correcto funcionamiento depende tanto de la tecnología utilizada (software, hardware, infraestructura de red) como de la forma en que se organizan los procesos y se utilizan los datos.

El ciclo de vida de un sistema de información describe las distintas etapas por las que pasa un sistema desde que se detecta la necesidad de su creación hasta que deja de utilizarse. No se trata solo del desarrollo del software, sino de todo el proceso asociado al sistema: análisis, implantación, uso, mantenimiento y retirada. Este enfoque permite trabajar de forma ordenada, documentar correctamente el sistema y controlar su evolución a lo largo del tiempo.

Para el Técnico Auxiliar de Informática, el conocimiento del ciclo de vida es esencial, ya que su trabajo se sitúa habitualmente en fases como la implantación, la explotación, el soporte técnico, las pruebas o el mantenimiento de los sistemas de información.

1.2. Fases del ciclo de vida de los sistemas de información

El ciclo de vida de los sistemas de información se divide en una serie de fases que permiten organizar su desarrollo y gestión de forma progresiva. Aunque pueden variar según el modelo empleado, de forma general se reconocen las siguientes fases:

Planificación y estudio de viabilidad

En esta fase se analiza la necesidad del sistema y se valora si su desarrollo es viable. Se estudian los objetivos, los recursos disponibles y las posibles alternativas antes de tomar la decisión de iniciar el proyecto.

Análisis del sistema

Se examinan los procesos existentes y se definen los requisitos que deberá cumplir el sistema. En este punto se concreta qué funciones tendrá el sistema y qué información deberá manejar, sin entrar todavía en detalles técnicos.



Diseño del sistema

Se define cómo va a construirse el sistema desde el punto de vista técnico. Se establecen la arquitectura, los componentes, los modelos de datos y las medidas de seguridad necesarias.

Desarrollo o construcción

Se desarrollan los distintos elementos del sistema, principalmente el software, y se configuran los entornos necesarios. También se genera la documentación técnica asociada.

Pruebas

Se comprueba que el sistema funciona correctamente y que cumple los requisitos definidos. Esta fase permite detectar errores antes de la puesta en producción.

Implantación

El sistema se pone en funcionamiento en el entorno real. Puede incluir la migración de datos desde sistemas anteriores y la formación de los usuarios.

Explotación y mantenimiento

El sistema se utiliza de forma habitual y se realizan tareas de mantenimiento para corregir incidencias, introducir mejoras o adaptarlo a cambios normativos o funcionales.

Retirada, desmantelamiento o sustitución

El sistema deja de utilizarse cuando resulta obsoleto o es reemplazado por otro. Esta fase debe garantizar la conservación, migración o destrucción segura de la información según la normativa de archivo y protección de datos, así como el retiro controlado de los recursos técnicos (hardware, licencias) y el archivo definitivo de la documentación.

1.3. Relación entre el ciclo de vida del sistema de información y el ciclo de vida del software

El ciclo de vida del software forma parte del ciclo de vida del sistema de información, pero no lo abarca completamente. Mientras que el sistema de información incluye aspectos organizativos, procedimentales y humanos, el ciclo de vida del software se centra en el desarrollo, implantación y mantenimiento de las aplicaciones informáticas.



Dentro de un sistema de información pueden existir varias aplicaciones, cada una con su propio ciclo de vida. Estas aplicaciones se desarrollan siguiendo los requisitos y el diseño definidos para el sistema en su conjunto. Por este motivo, los cambios en el software suelen tener impacto en el funcionamiento global del sistema de información.

Desde el punto de vista del Técnico Auxiliar de Informática, esta relación se refleja en tareas como el soporte a aplicaciones, la gestión de versiones, la realización de pruebas o la colaboración en despliegues, siempre dentro del marco general del sistema de información en el que se integran.

1.4. Relación entre el ciclo de vida del sistema de información y la gestión de proyectos TI

La gestión de proyectos TI proporciona el marco necesario para organizar y controlar las actividades que se desarrollan a lo largo del ciclo de vida del sistema de información. Permite planificar los trabajos, asignar recursos, establecer plazos y realizar un seguimiento del avance del sistema.

En la práctica, el ciclo de vida del sistema se materializa a través de proyectos TI que abarcan desde el desarrollo inicial hasta las mejoras y evoluciones posteriores. Cada fase del ciclo de vida (planificación, desarrollo, implantación) se gestiona como un conjunto de tareas, hitos y entregables dentro de un proyecto.

En el entorno de la Administración General del Estado, una correcta gestión de estos proyectos resulta fundamental para garantizar la continuidad del servicio, el cumplimiento normativo y el uso eficiente de los recursos públicos.

El Técnico Auxiliar de Informática participa habitualmente en este contexto realizando tareas técnicas, de apoyo y de explotación, contribuyendo al funcionamiento diario de los sistemas de información a lo largo de todo su ciclo de vida.

2. Repositorios



Recuerda ver las clases emitidas en Temario Audiovisual

Las clases impartidas en directo y disponibles en Campus, en Temario Audiovisual, te ayudarán al entendimiento de la unidad, y además pueden tener información adicional.

[ACCEDE DIRECTAMENTE DESDE AQUÍ](#)



Un repositorio es una gran base de datos o un conjunto de varias, donde se almacenan y gestionan una gran cantidad de datos para obtener informes a partir de su análisis.

Un repositorio es llamado a menudo biblioteca o archivo de datos.

Cada vez, los informes obtenidos a partir de un análisis de datos se utilizan más en la toma de decisiones de los ejecutivos y directivos en cualquier empresa u organización.



+ Info

Algunas instituciones, ofrecen libremente a la comunidad los datos almacenados e informes generados por ellos mismos.

Para entender bien los repositorios, vamos a indicar unas definiciones de conceptos importantes:

- **Vector multidimensional.**

Es un vector que se indexa mediante una lista ordenada de enteros. El número de enteros que se utiliza en esta lista para indexar el vector multidimensional es siempre el mismo y se conoce como la dimensionalidad del vector.

Los límites de cada uno de los enteros que forman parte del índice determinan la dimensión del vector.

- A un vector con dimensionalidad k se le suele llamar k -dimensional.
- Los vectores 1-dimensional se corresponden con los vectores ordinarios en los que los elementos están dispuestos en una única fila (o columna).
- Los vectores 2-dimensional son otra forma de llamar a las clásicas matrices en las que sus elementos están dispuestos en varias filas y columnas (dos dimensiones).

- **Base de datos multidimensional.**

Se utilizan para crear aplicaciones OLAP y pueden verse como bases de datos de una sola tabla.

Por cada dimensión tienen un campo (o columna), y otro campo por cada métrica o hecho, es decir estas tablas almacenan registros cuyos campos son de la forma:

$(d_1, d_2, d_3, \dots, f_1, f_2, f_3, \dots)$



Donde:

- Los campos *d1* hacen referencia a las dimensiones de la tabla.
- Los campos *f1* a las métricas o hechos que se quiere almacenar, estudiar o analizar.

Cada una de estas tablas puede asimilarse a un cubo OLAP, donde las dimensiones del mismo se corresponden con los campos de dimensiones de la tabla (campos *d1...*), y el valor almacenado en cada celda del cubo equivale a la métrica o métricas (campos *f1...*) almacenadas en la tabla.



+ Info

Big Data:

Es un *Gran volumen* de datos con una variedad, complejidad y velocidad de crecimiento enorme y que además tienen la característica de no ser estructurados, es decir, no son relacionales.

Estos datos pueden provenir de diferentes fuentes y formas, como mensajería instantánea, redes sociales, registros de grabaciones, imágenes, mensajes de correo electrónico, etc.

Se pueden analizar en tiempo real.

Para crear repositorios de forma adecuada hay que tener en cuenta su uso, e involucrar en la fase de desarrollo a las personas adecuadas, por ejemplo, si se trata de un repositorio de datos médicos, habrá que contar además de con los expertos en datos y los analistas, con profesionales de la medicina.

Algunas recomendaciones para aprovechar al máximo un repositorio son:

- Seleccionar la herramienta adecuada (ETL: extracción, transformación y carga) que proporcione las características que se adapten más a los fines para los que se usará el repositorio.
- Automatizar al máximo los procesos de carga y mantenimiento del repositorio, lo cual además de trabajo al usuario, también reduce la posibilidad de errores.
- Almacenar de menos a más.

Al inicio es más recomendable, almacenar conjuntos de datos más pequeños y limitar las temáticas, para ir aumentándolas gradualmente cuando los operadores se familiarizan con el sistema.



- Priorizar la escalabilidad.

El repositorio ha de ser flexible para adaptarse a los tipos de datos en evolución y aumentar los volúmenes.



Atención

No hay que olvidar la importancia de la seguridad, por lo que hay que crear reglas de acceso integrales para asegurar que únicamente los operadores autorizados puedan acceder, cambiar o transferir datos.

2.1. Conceptos de tipos de repositorios

Vamos a ver diferentes conceptos relacionados con los repositorios, en función de qué datos almacenan etc.

- Repositorios de Metadatos.
- DATA MART.
- DATA WAREHOUSE (DW).
- DATA LAKE.

2.1.1. Repositorios de metadatos

Hay que diferenciar entre el concepto de metadatos y los repositorios de metadatos:

- Los metadatos incorporan información sobre las estructuras que incluyen los datos reales.
- Repositorios de metadatos:

Contienen información sobre el modelo de datos que almacena y comparte estos datos.

Describen dónde está la fuente de los datos, cómo se recopilaron y su significado, y también definen la disposición de cualquier dato o tema almacenado en cualquier formato.



2.1.2. Data Mart

Un Data Mart (o datamart) es una base de datos departamental, se denomina así por estar especializada en el almacenamiento de los datos de tema específico, normalmente alineado con un área específica como puede ser ventas, marketing, recursos humanos etc.

Puesto que solo incluye los datos de un tema o área específica, su tamaño es más pequeño, por lo que resulta ser una forma económica y rápida de acceder más fácilmente a datos relevantes, adquiriendo así conocimientos procesables rápida y fácilmente.

OLAP siglas del inglés On Line Analytical Processing, (en castellano análisis mul-tidimensional) es el sistema típico de los Data Marts.

2.1.2.1. OLAP

Los sistemas OLAP son bases de datos orientadas al procesamiento analítico, por lo que se realiza la lectura de grandes cantidades de datos, permite profundizar en la información hasta llegar a un alto nivel de detalle, pudiendo extraer información útil, como tendencias de mercado, patrones de comportamiento de usuarios y consumidores, elaboración de informes, etc.

Se analizan datos desde diferentes perspectivas, realizar proyecciones de información para pronosticar lo que puede ocurrir en el futuro, análisis de tendencias, análisis prospectivo, etc.

OLAP tiene las siguientes características:

- Normalmente se accede a los datos en modo de sólo lectura.
- Su uso es para realizan consultas, y ocasionalmente pueden darse inserciones, actualizaciones o borrados.
- Los datos se estructuran dependiendo del área de negocio.
- El historial de datos es a largo plazo, mínimo de dos a cinco años.
- Las bases de datos OLAP suelen obtener la información de los sistemas operacionales existentes, utilizando un proceso ETL.
- Proporciona gran rapidez de respuesta en las consultas, por ello su uso en la toma de decisiones, Data Mining, etc.



+ Info

Data Mining (Minería de Datos, explotación de datos), se ocupa de reunir los datos de manera novedosa, entendible y útil para el propietario o usuario final.



Es un paso que forma parte del KDD e implica el análisis de grandes cantidades de datos observacionales, para encontrar relaciones insospechadas.

KDD, siglas del inglés "Knowledge Discovery in Data-bases", (extracción de conocimiento en BB.DD), es la creación de conocimiento a partir de fuentes estructuradas y no estructuradas. Implica la evaluación e interpretación de patrones y modelos para tomar decisiones con respecto a lo que constituye conocimiento y lo que no lo es.

El objetivo de OLAP, es agilizar la consulta de grandes cantidades de datos, para lograrlo, utiliza estructuras de datos diversas, normalmente multidimensionales (o Cubos OLAP), que contienen datos resumidos de grandes Bases de datos o de OLTP.

En la base de cualquier sistema OLAP se encuentra el concepto de cubo OLAP.

OLTP son sistemas transaccionales diseñados para dar soporte al procesamiento diario de datos de las organizaciones, es decir, gran cantidad de transacciones en cortos períodos de tiempo.



Comparación

Comparativa ente OLAP y OLTP.

Una base de datos relacional almacena entidades en tablas discretas, si han sido normalizadas. Esta estructura es buena en un sistema OLTP pero, para las complejas consultas multitabla, es relativamente lenta.

Un modelo mejor para búsquedas (aunque peor desde el punto de vista operativo) es una base de datos multidimensional.



Un **sistema OLAP** se centra en el análisis histórico de grandes cantidades de datos para **identificar tendencias, patrones** y obtener información útil para la toma de **decisiones estratégicas**, mientras que uno **OLTP** se ocupa del **procesamiento de transacciones en tiempo real**.

Por lo tanto, los sistemas OLAP están diseñados para manejar grandes volúmenes de datos y para realizar operaciones de consulta complejas (SELECT) y análisis de manera eficiente. Por otro lado **OLTP** será generalmente más adecuado para la realización de operaciones de inserción, actualización y borrado (INSERT, UPDATE, DELETE).

2.1.2.1.1. Cubo OLAP

Un Cubo OLAP, también llamado cubo multidimensional o hipercubo, es una lista de datos con múltiples dimensiones (normalmente tres o más) almacenadas como una tabla.

Como puede haber más de tres dimensiones los cubos OLAP también reciben el nombre de hipercubos.

Las diferentes herramientas existentes de software OLAP tienen distintos métodos de creación y vinculación de estos cubos o hipercubos.

Cada dimensión de un cubo de datos significa características específicas de la base de datos, como pueden ser las ventas de un producto diarias, mensuales o anuales.

Se utilizan para evaluar los datos recopilados desde una gran variedad de puntos de vista, por ejemplo, identificar tendencias lo que ayuda en gran manera a la toma de decisiones de una empresa.

Término Cubo OLAP acuñado por Edgar Frank Codd

Un cubo OLAP es una base de datos multidimensional, en la cual el almacenamiento físico de los datos se realiza en un vector multidimensional. Los cubos OLAP se pueden considerar como una ampliación de las dos dimensiones de una hoja de cálculo.

Inicialmente se creía que todas las necesidades de un usuario de bases de datos se podían obtener con un sistema de base de datos relacional.



Codd, propulsor de las bases de datos relacionales, realizó una propuesta que consistía en realizar una disposición de los datos en vectores, llamados cubos, basándose en los siguientes criterios:

- Poder disponer los datos en cubos evitaría la limitación del análisis instantáneo de grandes cantidades de datos de las bases de datos relacionales.
- Las bases de datos relacionales son más adecuadas para registrar datos provenientes de transacciones (conocido como OLTP o procesamiento de transacciones en línea).
- Las herramientas de generación de informes para bases de datos relacionales son lentas de por sí, y lo son mucho más cuando debe explorarse toda la base de datos.

Los parámetros utilizados para analizar los datos se denominan dimensiones. Por ejemplo, en datos financieros serían dimensiones: producto, periodo, ciudad, ingresos, gastos. Y se podrían comparar con datos reales de un presupuesto.

Cada una de las dimensiones de un cubo OLAP puede resumirse mediante una jerarquía, por ejemplo:

- Si se considera una escala (o dimensión) temporal "julio de 2007" se puede incluir en "3º Trimestre de 2007", que a su vez se incluye en "Año 2007".
- Si se refleja una situación geográfica, las ciudades se pueden incluir en regiones, países y Continentes.

El analista puede comenzar en un nivel muy resumido y posteriormente des-cender en el cubo (en sus jerarquías) observando así con mayor detalle los datos, pudiendo obtener gran información, viendo en el cubo los lugares en los que se ha producido esta diferencia, según los productos y períodos.



Atención

Utilizar los cubos OLAP, permiten que las consultas de selección sean muy rápidas (casi instantáneas), pero se presenta el problema de que una vez definida la base de datos, ésta no puede recibir cambios en su estructura, ya que para ello sería necesario rediseñar el cubo.

En un cubo OLAP pueden darse las siguientes situaciones:

- Dispersión: se produce cuando no todas las celdas del cubo se rellenan con datos (escasez de datos o valores nulos).
- Vinculación: se vinculan o enlazan cubos para superar la dispersión.



En vez de tener un cubo disperso, puede resultar más conveniente crear otro cubo distinto y vinculado, en el que un subconjunto de los datos se puede analizar con gran detalle.

Hay que recordar que el tiempo de procesamiento es muy valioso.

La vinculación asegura que los datos de los dos cubos mantengan una coherencia.



+ Info

Cubo OLAP.

Se compone de hechos numéricos o medidas, que se clasifican por dimensiones.

El cubo de metadatos es típicamente creado a partir de un esquema en estrella o copo de nieve, de las tablas en una base de datos relacional. Las medidas se obtienen de los registros de una tabla de hechos y las dimensiones se derivan de la dimensión de los cuadros.

Con OLAP el usuario puede realizar una navegación multidimensional por los datos, esto incluye funcionalidades como:

- **Profundizar/ascender (Drill-down/drill-up):** permite navegar por los datos de forma jerárquica, profundizando en niveles más específicos o ascendiendo a niveles más generales. Los usuarios pueden navegar a través de los niveles de jerarquía para ver datos a diferentes niveles de granularidad. Esa profundización o ascenso hace que la vista cambie, perdiendo de vista los datos de la vista de inicio. El análisis de uno sitio web por ejemplo con el paso de las páginas más visitadas a una página en concreto sería un drill-down.
- **Expandir (Expand):** permite una navegación en profundidad, sin perder la vista inicial. Una función de proyección que te permite navegar jerárquicamente manteniendo el contexto general. Una muñeca rusa sería un buen ejemplo, las vemos todas. Un árbol genealógico sería un buen ejemplo. Con un drill-down se cambia de vista, con un expand se conserva la vista y se le añade detalle.
- **Colapsar (Collapse):** operación inversa a la anterior. Si bien el expand permite añadir detalles y expandir dimensiones, collapse contrae una dimensión desplegada.



- **Pivotar (Pivoting):** permite reorganizar los datos cambiando el orden de las dimensiones o la perspectiva de los datos. Por ejemplo, un usuario podría ver las ventas por producto y luego pivotar para ver las ventas por cliente. Se pueden ver los datos desde diferentes perspectivas y obtener una comprensión más completa de los mismos.
- **Rotar (Swap):** permutar dos dimensiones de análisis, mover los objetos de filas a columnas y de columnas a filas.
- **Enrollar (Roll-up):** esta operación nos permite agrupar datos de niveles inferiores de una jerarquía en un nivel superior obteniendo sumas, promedios, cálculos de máximos y mínimos, etc.
- **Filtrar (Dice):** permite seleccionar subconjuntos específicos de datos a lo largo de una o varias dimensiones para crear vistas personalizadas y realizar análisis detallados según nuestras necesidades y criterios específicos.
- **Cortar (Slice):** operación que nos permite navegar por los datos de forma más precisa y eficiente pues podemos seleccionar subconjuntos de una sola dimensión.

2.1.2.2. Tipos de Arquitecturas OLAP

Los sistemas OLAP se clasifican principalmente según las categorías MOLAP, ROLAP y HOLAP, aunque también hay otros tipos. Los vemos a continuación:

- **MOLAP**

La arquitectura MOLAP usa unas bases de datos multidimensionales para proporcionar el análisis.

Un sistema MOLAP usa una base de datos propietaria multidimensional, en la que la información se almacena multidimensionalmente, para ser visualizada multidimensionalmente.

El sistema MOLAP utiliza una arquitectura de dos niveles:

- La base de datos multidimensional: es la encargada del manejo, acceso y obtención del dato.
- El motor analítico:

El nivel de aplicación es el responsable de la ejecución de los requerimientos OLAP. El nivel de presentación se integra con el de aplicación y proporciona un interfaz a través del cual los usuarios finales visualizan los análisis OLAP. Una arquitectura cliente/servidor permite a varios usuarios acceder a la misma base de datos multidimensional.

- **ROLAP**

Se trata de sistemas y herramientas OLAP contruidos sobre una base de datos relacional.

Es una alternativa a la tecnología MOLAP (Multidimensional OLAP) que se construye sobre bases de datos multidimensionales.



El sistema ROLAP utiliza una arquitectura de tres niveles; de bases de datos relaciona, de aplicación y de presentación, con las siguientes características en cada uno de ellos:

- **El nivel de base de datos relacional:**

Usa bases de datos relacionales para el manejo, acceso y obtención del dato.

- » Consultas.
- » Tablas hash.
- » Indexación paralela.
- » Consolidaciones.
- » Particionado de datos.
- » Joins.
- » Optimizaciones.

- **El nivel de aplicación:**

Es el motor ROLAP que ejecuta las consultas multidimensionales de los usuarios.

- » Ratios.
- » Rangos.
- » Transformaciones.
- » Predicciones.
- » Priorización Querys.
- » Planificación.
- » Manejo de excepciones.

- **El nivel de Presentación:**

El motor ROLAP se integra con niveles de presentación, a través de los cuales los usuarios realizan los análisis OLAP.

- » Gráficos y Alertas.
- » Drill-Down (profundizar): operación que permite apreciar los datos con un mayor nivel de detalle. Se aplica bajando por los niveles de una Jerarquía definida en un Cubo. Esto brinda la posibilidad de introducir un nuevo nivel o criterio de agregación en el análisis. Drill-Down implica ir de lo general a lo específico.



- **HOLAP**

HOLAP (Hybrid Online Analytical Process, procesamiento analítico en línea híbrido), es una combinación de ROLAP y MOLAP. Almacena algunos datos en un motor relacional y otros en una base de datos multidimensional.

El grado de control que el operador de la aplicación tiene sobre este particionamiento es diferente dependiendo del producto:

- Particionamiento vertical.

En este modo, HOLAP almacena agregaciones como un MOLAP para mejorar la velocidad de las consultas, y los datos se detallan en ROLAP para optimizar el tiempo en que se procesa el cubo.

- Particionamiento horizontal.

En este modo HOLAP almacena una sección de los datos, normalmente los más recientes (por ejemplo, particionando por la dimensión tiempo) en modo MOLAP para mejorar la velocidad de las consultas, y los datos más antiguos en ROLAP. Además, se pueden almacenar algunos cubos en MOLAP y otros en ROLAP.

Algunos productos comerciales que soportan el modo HOLAP de almacenamiento son: Microsoft Analysis Services, MicroStrategy, SAP AG BI Accelerator.

- **Otros tipos:**

Los siguientes acrónimos a veces también se utilizan, aunque no son sistemas tan generalizados como los anteriores:

- WOLAP o Web OLAP (OLAP orientado a la web).
- DOLAP o Desktop OLAP (OLAP de escritorio).
- RTOLAP o Real Time OLAP (OLAP en tiempo real).
- SOLAP o Spatial OLAP (OLAP espacial).

2.1.2.3. Comparativa de tipos de Arquitecturas OLAP

ROLAP y MOLAP, están diseñadas para realizar análisis de datos a través del uso de modelos de datos multidimensionales.

- En el caso de ROLAP estos modelos se implementan sobre un sistema relacional clásico, por tanto:
 - Los datos son detallados, evitando las agregaciones y las tablas se encuentran desnormalizadas.
 - Los esquemas más comunes sobre los que se trabaja son estrella o copo de nieve, aunque es posible trabajar sobre cualquier base de datos relacional.



- La arquitectura está compuesta por un servidor de banco de datos relacional y el motor OLAP se encuentra en un servidor dedicado.
- La principal ventaja de esta arquitectura es que permite el análisis de una enorme cantidad de datos.
- En el caso de MOLAP estos modelos se implementan sobre un sistema multidimensional.

Ventajas de MOLAP

- Consultas rápidas debido a la optimización del rendimiento de almacenamiento, la indexación multidimensional y la memoria caché.
- Ocupa menos tamaño en disco en comparación con los datos almacenados en base de datos relacional debido a técnicas de compresión.
- Automatización del procesamiento de los datos agregados de mayor nivel.
- Muy compacto para conjuntos de datos de pocas dimensiones.
- El modelo de almacenamiento en vectores/matrices proporciona una indexación natural.
- Eficaz extracción de datos lograda gracias a la preestructuración de los datos agregados.

Desventajas de MOLAP

- La etapa de procesamiento (carga de datos) puede ser bastante larga, sobre todo para grandes volúmenes de datos. Normalmente, esto se puede evitar con un procesamiento incremental, es decir, sólo el procesamiento de los datos que han cambiado (por lo general, los nuevos datos) en lugar de volver a procesar de todo el conjunto de datos.
- Las herramientas MOLAP tradicionalmente tienen dificultades para consultar con modelos con dimensiones muy altas (del orden de millones de miembros).
- Algunas herramientas MOLAP (como Essbase) tienen dificultades para actualizar y consultar los modelos con más de diez dimensiones. Este límite varía en función de la complejidad y la cardinalidad de las dimensiones de que se trate, y de la cantidad de hechos o medidas almacenados. Otras herramientas MOLAP (por ejemplo, Microsoft Analysis Services o Applix TM1) puede manejar cientos de dimensiones.
- El enfoque MOLAP introduce redundancia en los datos.



+ Info

La mayoría de las herramientas comerciales OLAP utilizan ahora un "Híbrido OLAP" (HOLAP), que permite al diseñador del modelo decidir qué parte de los datos se almacenarán en MOLAP y qué parte en el ROLAP.

Ejemplos de productos comerciales que utilizan MOLAP son: Oracle OLAP, Microsoft Analysis Services, Essbase, icCube Server, Infor OLAP y IBM TM1.

Existe un servidor MOLAP con una versión en código abierto llamado PALO.

MOLAP también se utiliza en Microsoft SQL server en la mayoría de sus versiones.

2.1.3. Data Warehouse

Un Data Warehouse (nombrado como DW) es un repositorio unificado donde una empresa u organización mantiene una gran cantidad de datos que recogen diversos temas de esa empresa u organización. Estos datos deben almacenarse de forma segura, fiable, fácil de administrar y de acceder.

Se almacenan datos estructurados, de diversas fuentes o sistemas de la empresa, para tenerlos juntos y así poder dividirlos en función de las necesidades de cada momento, realizando un análisis preciso y de alta calidad para ayudar a la toma de decisiones.

Según la tendencia marcada por Bill Inmon sobre los Data Warehouse, un Data Mart dependiente es un subconjunto lógico (vista) o un subconjunto físico (extracto) de un almacén de datos más grande (Data Warehouse), que se ha aislado por alguna de las siguientes razones:

- Necesidad de un esquema: Se necesita para un esquema o modelo de datos espacial (por ejemplo, para reestructurar los datos para alguna herramienta OLAP).
- Prestaciones: Para descargar el Data Mart a un ordenador independiente para mejorar la eficiencia o para obviar las necesidades de gestionar todo el volumen del Data Warehouse centralizado.
- Seguridad: Para separar un subconjunto de datos de forma selectiva a los que queremos permitir o restringir el acceso.
- Conveniencia: la de poder pasar por alto las autorizaciones y requerimientos necesarios para poder incorporar una nueva aplicación en el Data Warehouse principal de la Empresa.



- Demostración sobre el terreno: para demostrar la viabilidad y el potencial de una aplicación antes de migrarla al Data Warehouse de la Empresa.
- Política: Razones internas de la organización para hacer esta división o separación de los datos del almacén de datos, por ejemplo:
 - Cuando se decide una estrategia para las TI (Tecnologías de la información) en situaciones en las que un grupo de usuarios tiene más influencia, para determinar si se financia dicha estrategia o descubrir si ésta no sería buena para el almacén de datos centralizado.
 - Estrategia para los consumidores de los datos en situaciones en las que un equipo de almacén de datos no está en condiciones de crear un almacén de datos utilizable.

Según la escuela Inmon de Data Warehouse, entre las pérdidas inherentes al uso de Data Marts están la escalabilidad limitada, la duplicación de datos, la inconsistencia de los datos con respecto a otros almacenes de información y la incapacidad para aprovechar las fuentes de datos de la empresa. Así y todo, estas herramientas son de gran importancia.



Comparativa

La diferencia de Data Warehouse y Data Marts es solamente en cuanto al alcance:

- Data Warehouse es un sistema centralizado con datos globales de la empresa y de todos sus procesos operacionales.
- Data Mart es un subconjunto temático de datos, orientado a un proceso o un área de negocio específica.

Historia

El concepto de almacén de datos se originó en 1988 con el trabajo de los investigadores de IBM, Barry Devlin y Paul Murphy.

El término Data Warehouse fue acuñado por William H. Inmon, (conocido como el padre de Data Warehousing), y lo describió como:

"Una colección de datos orientada a un tema específico, integrado, variante en el tiempo y no volátil, que soporta el proceso de toma de decisiones".



Los datos de diferentes aplicaciones de procesamiento de transacciones Online (OLTP) y otras fuentes se extraen selectivamente para su uso por aplicaciones analíticas y de consultas por usuarios, ayudando a los ejecutivos de negocios organizar, comprender y utilizar sus datos para tomar decisiones estratégicas.

Los sistemas de soporte a la decisión usando tecnologías de Data Warehouse, se llaman sistemas OLAP (siglas de On Line Analytical Processing (OLAP)).



+ Info

Data Warehouse Institute, (TDWI).

Es una asociación mundial de profesionales de BI (Business Intelligence), que ofrece educación, capacitación, investigación y certificación, capacitando a empresas y a profesionales de TI (tecnología de la información), sobre las estrategias, herramientas y técnicas para diseñar, construir y mantener soluciones de BI y Data Warehouse con éxito.

Estructuras de un Data Warehouse

La arquitectura de un Data Warehouse puede ser dividida en tres estructuras simplificadas:

- Básica.

Proporciona datos brutos junto con metadatos, para facilitar que los usuarios finales puedan acceder a ellos para su análisis, generación de informes y minería.

- Básica con un área de ensayo.

El área de ensayo se coloca entre las fuentes de datos y el almacén, y proporciona un lugar donde los datos se pueden "limpiar" antes de entrar en el almacén. Ofrece la posibilidad de limpiar la información en función de la utilidad que se le vaya a dar.

- Básica con área de ensayo y Data Marts.

Los Data Marts, son sistemas diseñados para una línea de negocio concreta, así se pueden tener Data Marts separados para ventas, inventario, compras, publicidad etc.

Los usuarios finales pueden acceder a datos de uno o de todos los Data Marts obteniendo la información que consideren oportuna.



Tratamiento de los datos en el Data Warehouse

En un principio, los Data Warehouses se formaban utilizando datos repetitivos estructurados que eran filtrados antes de entrar en el Data Warehouse. Sin embargo, en los últimos años, el Data Warehouse ha evolucionado pudiendo almacenar también información contextual que se puede adjuntar a los datos no estructurados.

Los datos relacionales estructurados no podían ser mezclados y emparejados para temas analíticos con datos textuales no estructurados. Pero ahora, con la contextualización, estos tipos de análisis pueden hacerse de forma natural y fácil.

Hay que diferenciar entre datos repetitivos y no repetitivos.

Los datos no repetitivos, como los comentarios en una encuesta, correos electrónicos y conversaciones se tratan de forma diferente a las ocurrencias repetitivas de datos, como el flujo de clics, mediciones o el procesamiento máquina o analógico.

Los datos no repetitivos son datos basados en textos que fueron generados por la palabra escrita o hablada, leída y reformateada y, lo que es más importante, ahora puede ser contextualizada. Para poder extraer sentido y conocimiento de estos datos no repetitivos deben tener el contexto de los datos establecidos. Este contexto incluso puede ser más importante que los datos en sí. Por ello, si no se ha establecido el contexto, estos datos no repetitivos no se pueden utilizar para la toma de decisiones.

El Data Warehouse continúa evolucionando diariamente.



+ Info

Herramientas ETL.

Acrónimo de Extract, Transform y Load (extraer, transformar y cargar).

Es el software que se utiliza para la construcción de Data Warehouse o almacenamiento de datos.

Las ETL conforman el proceso de alimentación del repositorio de datos o estructura de datos multidimensional, asegurando que los datos sean extraídos, transformados y cargados de manera eficiente y precisa.



Data Warehouse en la nube

Un Data Warehouse se aloja normalmente en un servidor corporativo, y en la actualidad es cada vez más común que dicho alojamiento sea en la "nube".

La nube permite a los Data Warehouses aumentar la agilidad general (la demanda de información a datos históricos y actuales es cada vez mayor), y mejorar el control de costes garantizando que todos los datos sensibles y estratégicos estén completamente asegurados, a lo largo de todo el ciclo de vida, de una forma rentable.

El Data Warehouse en la nube está directamente vinculado a tres factores clave:

- **Mayor agilidad:**

La demanda de información a datos históricos y actuales es cada vez mayor.

Las empresas quieren aprovechar al máximo los flujos de datos y nuevos tipos de análisis para apoyar e impulsar nuevas áreas, y ello requieren nuevos entornos de hardware y software.

(Por ejemplo, estos nuevos tipos de análisis son: analítica de clientes de 360°, análisis predictivo, detección de fraude, análisis de IoT, establecimiento de los datos como centro de beneficio).

Ventaja: Más fácil consolidación y racionalización.

- **Mejor control de costes:**

La BI busca consolidar los Data Marts existentes en un único entorno integrado, ejecutándose cada uno en hardware dedicado o incluso en hardware propietario.

Hay que recordar que los activos de datos deben estar protegidos a lo largo de todo el ciclo de vida, y esto debe ser garantizado por los servicios en la nube, siendo más rentables ya que todas las características de seguridad se pueden habilitar de forma predeterminada y actualizada de forma transparente.

Ventaja: Monetización (dar curso legal) más rápida de los datos en la nube.

- **Colocalización para una carga más rápida:**

Diferentes aplicaciones como entrada de pedidos, ventas, materiales... generan datos directamente a los Data Warehouse, es lógico querer ubicar el Data Warehouse junto con los sistemas fuente que se estén ejecutando ya en la nube. De esta forma, se obtiene una carga de datos más rápida, proporcionando un mejor servicio a los usuarios.

Ventaja: La nube ofrece mejor protección.

Vamos a ver diferentes tipos de esquemas y cual elegir:

- Esquema en Estrella.
- Esquema en copo de nieve.



2.1.4. Data Lakes

Los Data Lakes han surgido en los últimos años, son repositorios de almacenamiento centralizado que contienen Big Data de varias fuentes en un formato granular y sin procesar.

Puede guardar datos estructurados, semiestructurados o no estructurados, por lo que los datos se almacenan en un formato más flexible para usarlos en el futuro para, por ejemplo, informes, análisis avanzados y aprendizaje automático.

Algunas diferencias importantes entre Data Lake y Data Warehouse:

- **En cuanto a Datos:**
 - Un Data Warehouse sólo almacena datos que han sido modelados o estructurados.
 - Un Data Lake no hace acepción de datos, lo almacena todo, estructurado, semiestructurado y no estructurado.
- **En cuanto a Procesamiento:**
 - En un Data Warehouse, antes de que una empresa pueda cargar datos, primero debe darles forma y estructura, es decir, los datos deben ser modelados. Esto se llama schema-on-write.
 - Con un Data Lake, sólo se cargan los datos sin procesar, tal y como están, y cuando se van a usar los datos, es cuando se da forma y estructura. Esto se llama schema-on-read.

- **En cuanto a Almacenamiento:**

El coste de almacenamiento de datos de las tecnologías de Big Data normalmente es más bajo que el de un Data Warehouse.

- **En cuanto a Agilidad:**

- En un Data Warehouse, (que es por definición un almacén de datos, un repositorio altamente estructurado), no es técnicamente difícil cambiar la estructura, aunque puede tomar mucho tiempo puesto que hay muchos procesos vinculados.
- En un Data Lake, (que carece de la estructura de un Data Warehouse) los desarrolladores y científicos de datos tienen la capacidad de configurar y reconfigurar fácilmente y en tiempo real sus modelos, consultas y aplicaciones.

Vamos a ver diferentes tipos de esquemas y cual elegir:

- Esquema en Estrella.
- Esquema en copo de nieve.



2.1.4.1. Esquema de Estrella

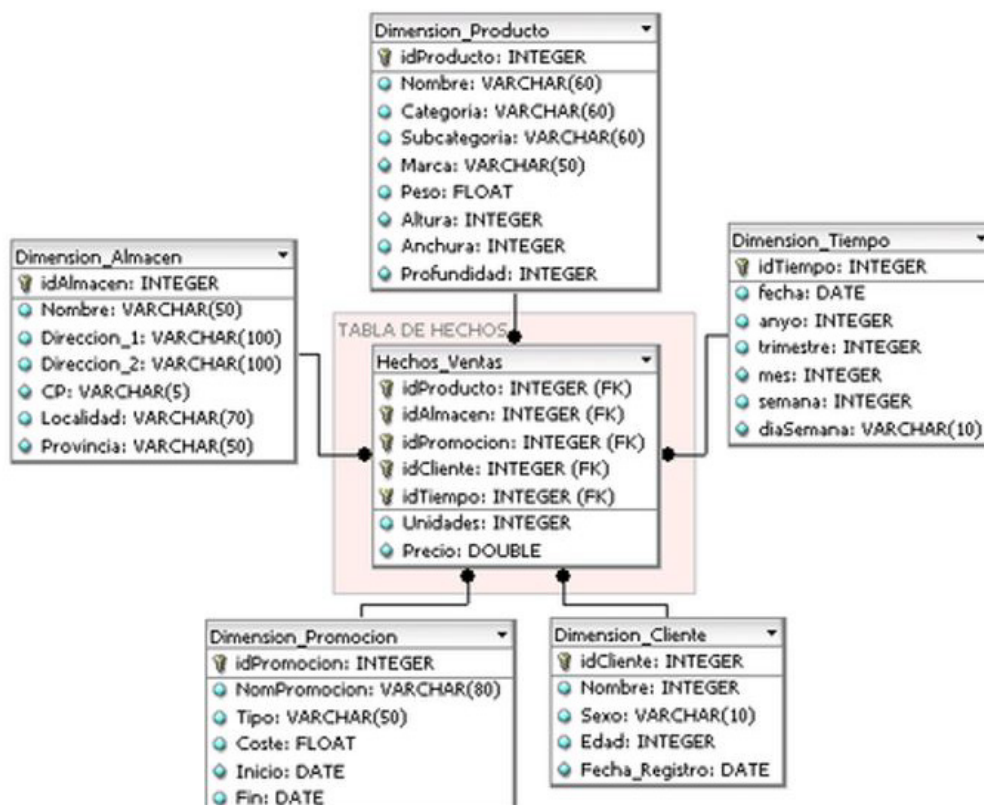
Un esquema en estrella es un modelo de datos que tiene una tabla de hechos (o tabla fact) que contiene los datos para el análisis, rodeada de las tablas de dimensiones.

Las ventajas de usar el esquema en estrella son:

- Su simplicidad y velocidad resulta ideal para ser usado en análisis mul-tidimensionales (OLAP, Datamarts, EIS, ...).
- Permite acceder tanto a datos agregados como de detalle.
- Permite implementar la funcionalidad de una base de datos multidimensional utilizando una clásica base de datos relacional (más extendidas que las multidimensionales).
- Simplicidad desde el punto de vista del usuario final.
- Las consultas no son complicadas, ya que las condiciones y las uniones (JOIN) necesarias solo involucran a la tabla de hechos y a las de dimensiones, no haciendo falta que se encadenen uniones y condiciones a dos o más niveles como ocurriría en un esquema en copo de nieve.

Este aspecto, de tabla de hechos (o central) más grande rodeada de radios o tablas más pequeñas es lo que asemeja a una estrella, dándole nombre a este tipo de construcciones.

Las tablas de dimensiones tendrán siempre una clave primaria simple, mientras que, en la tabla de hechos, la clave principal estará compuesta por las claves principales de las tablas dimensionales.





El esquema estrella separa los datos del proceso de negocios en:

- **Hechos:** Los hechos contienen datos medibles, cuantitativos, relacionados con la transacción del negocio.
- **Dimensiones:** Las dimensiones son atributos que describen los datos indicados en los hechos (una especie de metadatos, o sea datos que describen otros datos).

Vemos los dos conceptos a continuación.

2.1.4.1.1. Tabla de hechos

La tabla de hechos registra medidas o métricas de un Evento específico.

Un evento hace referencia a un suceso que se da en un punto determinado del tiempo, por ejemplo: un cliente compra: 4 pares de zapatillas marca: Adidas, número: 43, color: gris y negro, el día: 7 de octubre de 2020 a la hora: 12:48 h en la Sucursal: Casablanca 1200, Rue Chok, Marruecos, el vendedor fue: Adil Roel. (Esto es un evento, en este caso una venta concreta que se da en un punto determinado del tiempo).

Las tablas de hechos generalmente consisten en valores numéricos (datos asociados específicamente con el evento), y claves foráneas que referencian a tablas de datos dimensionales que guardan información descriptiva. (En el ejemplo de la compra de zapatillas, hay datos que, por poder ser repetitivos en otros sucesos, como es la sucursal, se hace otra tabla aparte que contenga todos los datos de las sucursales..., en caso de que la sucursal se cambie de dirección etc, solo será necesario hacer un solo cambio en la base de datos, en esa tabla de sucursales y no hay que hacerlo en la tabla de eventos, en cada transacción que se efectuó en esa sucursal).

Las tablas de hechos se diseñan para contener detalles uniformes a bajo nivel (referidos como "granularidad" o "grano"), o sea que los hechos pueden registrar eventos a un gran nivel de atomicidad. Esto puede resultar en la acumulación de un gran número de registros en la tabla de hechos, a lo largo del tiempo.

Las tablas de hechos se definen como uno de los siguientes tres tipos:

- **Tablas de hechos transaccionales:** registran hechos relativos a eventos específicos (por ejemplo, el evento de una venta).
- **Tablas de hechos Snapshot:** registran hechos en un punto dado en el tiempo, con un intervalo específico para realizar la medición sobre la entidad objeto de observación. (Ejemplo: detalles de una cuenta al final del mes).
- **Tablas Snapshot Acumulativas:** registran hechos agregados, acumulados a un punto dado en el tiempo. (Ejemplo, ventas totales mensuales para un producto dado).

Las tablas de hechos generalmente tienen asignada una surrogate key (clave sustituta, clave sintética, pseudoclave), que puede ser generada automáticamente) para asegurar que cada fila puede ser identificada de forma unívoca.



2.1.4.1.2. Tabla de Dimensiones

Las tablas de Dimensiones generalmente tienen un bajo número de registros, en comparación a las tablas de hechos, pero cada registro puede tener un gran número de atributos para describir los datos del hecho.

Las Dimensiones pueden definir una amplia variedad de características, algunos de los atributos más comunes definidos en las tablas de dimensiones incluyen:

- **Tablas de tiempo:** describen el tiempo al más pequeño nivel de granularidad de tiempo para el cual los eventos se registran en el esquema estrella.
- **Tablas de dimensión Geográficas:** describen datos de localización, tales como país, region, provincia, estado, o ciudad.
- **Tablas de dimensión de Productos:** describen los productos.
- **Tablas de dimensiones de Empleados:** describen los empleados, por ejemplo, los vendedores.
- **Tablas de dimensión de Rangos:** describen rangos de tiempo, valores de dólar, u otras cantidades medibles para simplificar los reportes.

Las tablas de Dimensiones generalmente tienen asignada una surrogate primary key, usualmente una columna simple de tipo de dato entero, que mapea a la combinación de atributos de dimensiones que forman la clave natural.

2.1.4.2. Esquema en copo de nieve

La estructura es algo más compleja que el esquema en estrella.

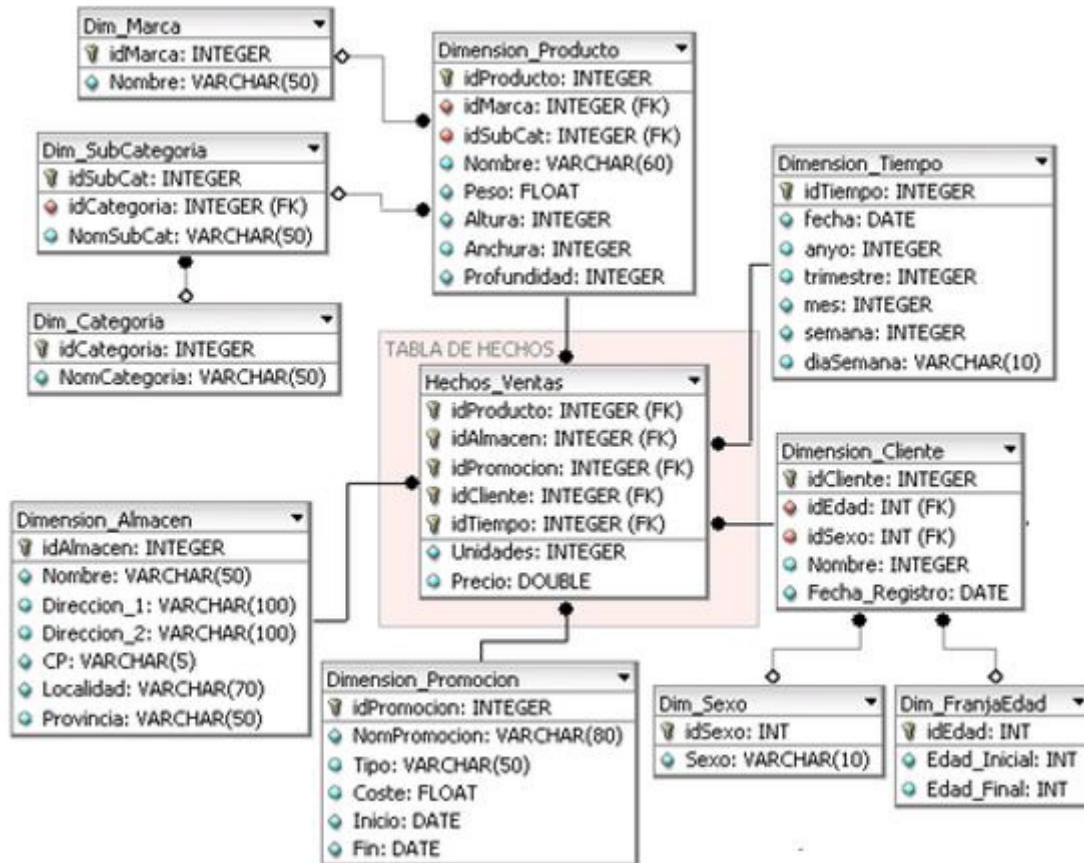
Se da cuando alguna de las dimensiones se implementa con más de una tabla de datos.

La finalidad es normalizar las tablas y así reducir el espacio de almacenamiento al eliminar la redundancia de datos; pero tiene la contrapartida de generar peores rendimientos al tener que crear más tablas de dimensiones y más relaciones entre las tablas (JOINS) lo que tiene un impacto directo sobre el rendimiento.

En las aplicaciones OLAP implementadas sobre bases de datos relacionales (ROLAP), un elemento clave es el Cubo OLAP, que almacenan grandes volúmenes de datos que posteriormente deben ser analizados en función de unos determinados parámetros.

Al diseñar las tablas en las que se han de almacenar estos datos y parámetros, si se aplican las técnicas de Normalización de bases de datos para optimizar el espacio requerido para guardar estos datos eliminando las redundancias, es habitual que se termine obteniendo un esquema en copo de nieve.

En este tipo de esquemas en copo de nieve, se tiene una tabla central de hechos en la que se guardan las medidas del negocio que se quiere analizar, y en las tablas adyacentes se tendrán las dimensiones (parámetros) de que dependen los datos del negocio. Si por alguna dimensión se requiere más de una tabla se dice que el esquema resultante es un esquema en copo de nieve.



Ejemplo de modelo de datos en copo de nieve. Fuente: Wikipedia

En el ejemplo, pese a no estar totalmente normalizada (algunas tablas como 'Dimension_Almacen' tiene redundancias), se observa como para algunas dimensiones de la tabla de hechos como Producto y Cliente se ha empleado más de una tabla, dando lugar a una jerarquía de dimensiones. Por ejemplo, los productos se pueden clasificar por marcas, además, estos mismos productos se pueden agrupar por categorías y subcategorías.

Las ventajas de usar el esquema en copo de nieve son:

- Al estar normalizadas las tablas de dimensiones, se evita la redundancia de datos y con ello se ahorra espacio.
- Como actualmente el espacio en disco no es un problema, y lo que se debe tener es un buen rendimiento, se usa copo de nieve cuando una tabla de dimensiones es relativamente grande. (y el de estrella cuando tiene menos filas en la tabla de dimensiones)



Conclusión

Se puede usar un esquema de copo de nieve en un Data Warehouse, aunque estos sean realmente grandes y complejos, pero nunca en sistemas donde el tiempo de respuesta sea un factor crítico para los usuarios.

En la mayoría de los casos son preferibles los de estrellas por su simplicidad respecto a los de copo de nieve por ser más fáciles de manejar. Es la opción con mejor rendimiento y velocidad pues permite indexar las dimensiones de forma individualizada sin que repercuta en el rendimiento de la base de datos en su conjunto.

3. Generación de código y documentación. Herramientas CASE

CASE es un acrónimo para *Computer-Aided Software Engineering*.



+ Info

Existen variaciones sobre este acrónimo:

- "C" siempre es *Computer*.
- "A" puede ser *Aided*, *Assisted* o *Automated*.
- "S" puede ser *Software* o *Systems*.
- "E" siempre es *Engineering*.

Las herramientas CASE son aplicaciones informáticas destinadas a aumentar la productividad en el desarrollo de *software* reduciendo el coste de estas en términos de tiempo y de dinero.



B. Terry y D. Logee definen CASE como "herramientas individuales para ayudar al desarrollador de *software* o administrador de proyecto durante una o más fases del desarrollo de *software* (o mantenimiento)".

Estas herramientas nos pueden ayudar en todos los aspectos del ciclo de vida de desarrollo del *software*.

Algunas de las tareas en las que nos puede ayudar son:

- Planificación temporal.
- Diseño de proyectos.
- Cálculo de costes.
- Implementación automática de código.
- Compilación automática.
- Documentación.
- Detección de errores (bugs).

Un error de software, error o simplemente fallo es un problema en un programa de computador o sistema de software que desencadena un resultado indeseado. Los programas que ayudan a la detección y eliminación de errores de software son denominados depuradores (debuggers).

- Etcétera.

Objetivos

La tecnología CASE Automatiza parte del desarrollo del *software*, y Contribuye a mejorar la calidad y la productividad en el desarrollo de sistemas de información.

Algunos de los objetivos de las herramientas CASE son:

- Mejorar la productividad del software.
- Aumentar la calidad del software y su portabilidad.
- Reducir el tiempo y costo de desarrollo y mantenimiento de los sistemas informáticos.
- Mejorar la planificación de un proyecto.
- Aumentar la biblioteca de conocimiento informático de una empresa ayudando a la búsqueda de soluciones para los requisitos.



- Automatizar el desarrollo del software, la generación de código, las pruebas de errores y la gestión del proyecto.
- Gestión global en todas las fases de desarrollo de software con una misma herramienta.
- Facilitar el uso de las distintas metodologías propias de la ingeniería del *software*.
- Permitir la aplicación práctica de metodologías estructuradas, agilizando el trabajo.
- Permitir el desarrollo y refinamiento visual de las aplicaciones mediante la utilización de gráficos.
- Facilitar la realización de prototipos y el desarrollo de aplicaciones.
- Simplificar el mantenimiento de los programas.
- Mejorar y estandarizar la documentación.
- Facilitar la reutilización de componentes *software*.

Componentes

Los componentes más comunes de las herramientas CASE son:

- **Depósito central:**

Es el componente más importante. Nos sirve como fuente de común, consistente e integrada información. El depósito central es un lugar de almacenamiento donde se guardan:

- Los requisitos del producto.
- Los documentos requeridos.
- Los informes.
- Diagramas.
- Cualquier otra información útil sobre la gestión del proyecto.

El depósito central también sirve como diccionario de datos.

- **Metamodelo:** constituye el marco para la definición de las técnicas y metodologías soportadas por la herramienta.
- **Gestión de datos:** proporciona un medio de comunicación con otras herramientas. Son componentes que permiten:
 - Cargar datos provenientes de otros sistemas en el depósito central.
 - Generar a partir de la propia herramienta esquemas de base de datos, programas, etcétera. Estos, a su vez, podrán alimentar a otros sistemas.



- **Comprobación de errores:** componentes que permiten llevar a cabo un análisis de la exactitud, integridad y consistencia de los esquemas generados por la herramienta.
- **Herramientas para realizar diagramas:** es una interfaz para realizar diagramas mediante herramientas de diseño gráfico y editores de texto.

El **repositorio** es la base de datos central de una herramienta CASE. Amplía el concepto de diccionario de datos para incluir toda la información que se va generando a lo largo del ciclo de vida del sistema, como, por ejemplo: componentes de análisis y diseño (diagramas de flujo de datos, diagramas entidad-relación, esquemas de bases de datos, diseños de pantallas), estructuras de programas, algoritmos, etcétera.

La mayoría de las herramientas CASE poseen un repositorio propio, o bien trabajan sobre un repositorio suministrado por otro fabricante o vendedor.

En el repositorio se almacena toda la información de uno o varios sistemas de información. Por ejemplo, datos acerca de:

- El dominio (problema) de los sistemas desarrollados o en desarrollo.
- Modelos de solución e implementación.
- Información de la metodología que está siendo usada.
- Historia de los proyectos, recursos, presupuestos, etcétera.
- Componentes de análisis y diseño:
 - Diagramas de flujo de datos.
 - Diagramas entidad-relación.
 - Esquemas de bases de datos.
 - Diseños de interface con el usuario.
- Estructuras de programas.
- Algoritmos.
- Contexto organizacional:
 - Organigramas.
 - Planes estratégicos.
 - Factores críticos de éxito.
 - Etcétera.



Cada ítem en el repositorio es descrito en detalle.

Algunos atributos típicos podrían ser:

- Identificación.
- Definición.
- Tipo.
- Alias.
- Ítems componentes.
- Ítems padres.
- Reglas de uso.
- Quién y cuándo lo creó.
- Quién y cuándo lo actualizó por última vez.
- Quiénes pueden actualizarlo y/o consultarlo.
- Cuál es su estado:
 - Completo.
 - Incompleto.
 - Etcétera.
- Número de versión.
- Dónde está almacenado físicamente.

Estructura y actualización de Repositorio

Los distintos elementos que puede contener un repositorio que utilice una herramienta CASE son:

- **Tipo de información:** metodología, datos, gráficos, procesos, informes, modelos o reglas.
- **Tipo de controles:**
 - Módulo de gestión de cambios.
 - Módulo de mantenimiento de versiones.



- Módulo de acceso por clave.
- Módulo de redundancia de la información.
- Gestión de cambios.
- Control de versiones.
- Comprobación de consistencia e integridad:
 - » Comprobación datos no definidos.
 - » Comprobación de datos autodefinidos.
 - » Comprobación de alias.
- **Tipo de actualización:** la actualización de un repositorio es todo cambio que se realice sobre alguno de sus elementos (código, documentación, diagramas, etcétera).

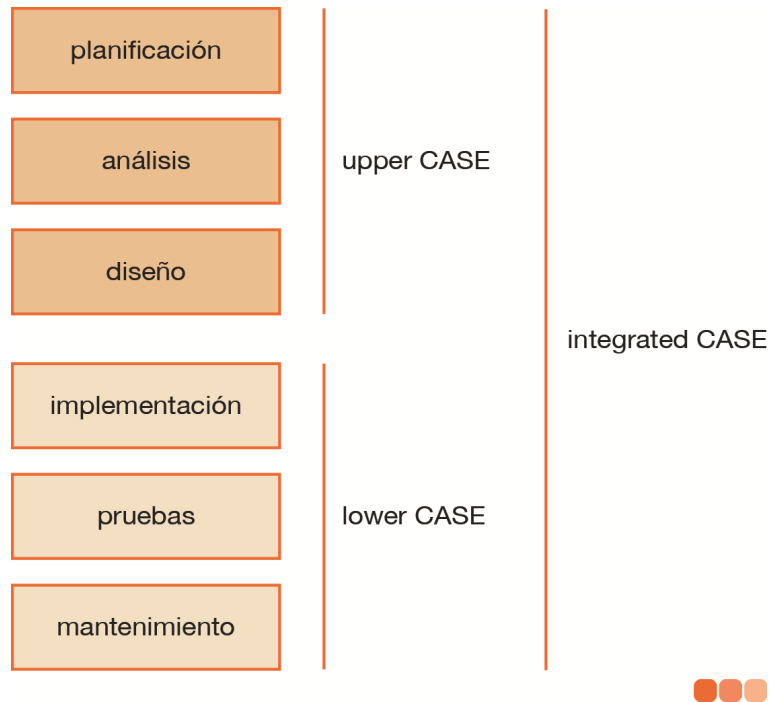
Hay dos tipos de actualización:

- **Tiempo real:** los cambios en los elementos se ven reflejados en el repositorio en tiempo real por todos los usuarios.
- **Batch:** se realiza mediante un proceso por lotes. El resto de los usuarios no lo verán hasta que sincronicen con el repositorio.
- **Módulos reutilizables para otros proyectos:** el repositorio es la clave para identificar, localizar y extraer código para su reutilización.
- **Interfaz para importación y exportación:** extrae información del repositorio para tratarla con otra herramienta o incorporar al repositorio información generada por otros medios.
- **Interfaces automáticas:** pueden conectarse con otros repositorios o bases de datos externas.

3.1. Clasificación de Herramientas CASE

Las Herramientas Case, se pueden clasificar en:

- Según su amplitud.
- Según las fases del ciclo de vida de software que cubren.
- Según la actividad que realizan.
- Otras clasificaciones.



3.1.1. Según la actividad que realizan

Existen muchos tipos de herramientas según el tipo de actividad. Vamos a resumir algunas de ellas:

- **Diagramas.** Se usan para representar de forma gráfica:
 - Componentes del sistema.
 - Datos.
 - Control de flujo de los procesos o los datos.
 - Estructura del software.
- **Administración de procesos.** Estas herramientas se usan para:
 - Planificación del proyecto.
 - Estimación de costes.
 - Estimación de esfuerzo.
 - Planificación temporal.
 - Organización de recursos.



- **Documentación.** La documentación cubre todo el ciclo de vida del software. Algunos tipos de documentos son:
 - Manual de usuario.
 - Manuales de sistemas.
 - Manuales de referencia.
 - Manuales de formación.
 - Manuales de instalación.
- **Análisis.** Estas herramientas ayudan a cumplir con los requisitos. Examinan de forma automática:
 - Inconsistencia.
 - Informaciones no acordes con los diagramas.
 - Posibles redundancias.
 - Omisiones.
 - Errores.
- **Diseño.** Ayuda a los diseñadores de software a crear la estructura de los programas. Aportan los detalles de cada módulo y la interconexión entre estos.
- **Gestión de la configuración.** Se ocupan de:
 - Control de versiones.
 - Líneas base.
 - Gestión del control de cambios.
- **Control de cambios.** Es parte de la gestión de la configuración. Se ocupan de:
 - Los cambios hechos en el software después de que se haya fijado una línea base.
 - Resaltar los cambios.
 - La gestión de archivos.
 - La gestión del código.
 - Etcétera.



- **Programación.** Incluyen características para simulación y prueba.

Son entornos de programación como:

- IDE (Integrated Development Environment).
 - Bibliotecas de módulos integrados.
 - Herramientas de simulación.
- **Desarrollo *software*.** Las herramientas de prototipado CASEP permiten crear interfaces de usuario y producen un prototipo *software*. Un prototipo es una versión simulada del producto *software* que se intenta desarrollar.

El prototipo da una idea inicial del producto y simula algunos aspectos del producto real.

- **Desarrollo web.** Aportan una vista preliminar.

Estas herramientas ayudan en el diseño de páginas web:

- Impresos.
 - Textos.
 - Secuencias de comandos.
 - Gráficos.
- **Aseguramiento de la calidad.** Es la supervisión del proceso de Ingeniería y de los métodos adoptados para desarrollar el producto *software*. Se trata de asegurar la conformidad con los estándares de calidad adoptados por la organización.

Las herramientas de aseguramiento de la calidad suelen estar formadas por:

- Herramientas de control de cambios.
- Herramientas de gestión de la configuración.
- Herramientas para pruebas de *software*.

- **Mantenimiento.**

El mantenimiento del *software* incluye modificaciones en el producto *software* una vez puesto en producción. Algunas de las técnicas utilizadas son:

- Inicio automático.
- Reporte de error.
- Producción automática de etiqueta de error.
- Análisis de causa raíz.



3.1.2. Según su amplitud

Según su amplitud (etapas de la vida del *software* a las que dan soporte) existen dos tipos:

- **Toolkit:**

Herramientas sueltas, no necesariamente integradas, enfocadas en tareas concretas (ej. editores de diagramas, generadores de código).

- **Toolset:**

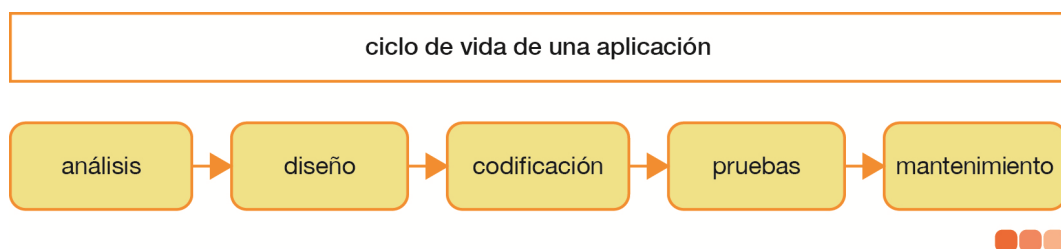
Un Toolset es un conjunto de herramientas CASE con cierta relación entre sí, pero sin integración completa. Su amplitud es mayor que la de un Toolkit.

- **WorkBench:**

Son conjuntos integrados de herramientas que dan soporte a la **automatización del proceso completo de desarrollo del sistema informático**. Permiten cubrir el ciclo de vida completo. El producto final aportado por ellas es un sistema en código ejecutable y su documentación.

3.1.3. Según el ciclo de vida del software que cubren

Es la clasificación de herramientas CASE más habitual, se realiza en función de las fases (etapas) del ciclo de vida del software que cubre la herramienta.



El ciclo de vida define los pasos que sigue el proceso de creación de una aplicación desde que se propone hasta que finaliza su construcción. Los pasos son:

- **Upper CASE (U-CASE).**

Herramientas que ayudan en las fases de planificación, análisis de requisitos y estrategia del desarrollo, usando, entre otros diagramas UML. (En alguna de las fases).

- **Middle CASE (M-CASE).**

herramientas para automatizar tareas en el análisis y diseño de la aplicación.



- **Lower CASE (L-CASE).**

Estas herramientas, se basan en una metodología, y aportan técnicas estructuradas para todas las fases del ciclo de vida.

Ahora vamos a indicar las características deseables en cada una de las herramientas U-CASE, M-CASE y L-CASE.

3.1.3.1. U-CASE

Cubren la gestión del proyecto y la planificación de recursos.

Las características deseables son:

- **Gestión de múltiples proyectos:**

Poder gestionar múltiples proyectos desde una sola interfaz.

Cada proyecto puede tener una configuración totalmente diferente y el usuario tener un rol distinto en cada uno.

Dentro de cada proyecto pueden definirse subproyectos.

- **Personalización de proyectos:**

Los módulos deben poder activarse o desactivarse en cada proyecto. Algunos de estos módulos son:

- Peticiones.
- Control del tiempo.
- Documentos.
- Archivos.

- **Sistema flexible de seguimiento de tareas:**

Una de las mecánicas más útiles para el desarrollo de un proyecto son las peticiones y su visualización.

Estas peticiones pueden ser errores, tareas o soporte y deben poder asignarse a un miembro del proyecto.



Se debe poder, para cada petición:

- Indicar una fecha de inicio y fin.
 - Llevar un control del tiempo.
 - Saber porcentaje realizado.
 - Asignar una prioridad.
 - Asignar antecesores y sucesores.
- **Uso de calendario y diagrama de Gantt:**

Debe incluir un calendario para visualizar todas las peticiones a lo largo de un periodo elegido, marcando claramente el día de inicio y de fin de cada petición.

Igualmente ocurre con la vista en diagrama de Gantt, que va marcando el porcentaje completado conforme avanzan los días.

- **Notificaciones:**

Se configura como un servidor de correo electrónico automatizado.

Debe permitir enviar notificaciones por correo en todos los proyectos, definiendo los eventos que activan estos avisos.

- **Exportación a distintos formatos:**

Los informes de peticiones deben poder exportarse a formatos estándar como PDF y CSV.

Veamos algunas de las herramientas más importantes de U-CASE.

Microsoft Project



Fuente:
https://commons.wikimedia.org/wiki/File:MS_Project_Logo.png



Repositorios: estructura y actualización. Generación de código y documentación. Metodologías de desarrollo. Pruebas. Programas para control de versiones. Plataformas de desarrollo colaborativo de software

Microsoft Project es un *software* de administración y gestión de proyectos diseñado, desarrollado y comercializado por Microsoft. Asiste a los administradores de proyectos en:

- El desarrollo de planes.
- Asignación de recursos a tareas.
- Seguimiento del progreso.
- Administración de presupuesto.
- Análisis de cargas de trabajo.

El *software* Microsoft Office Project aplica procedimientos descritos en el PMBoK del Project Management Institute.



+ Info

PMBoK (*A Guide to the Project Management Body of Knowledge*) es un libro en el que se presentan estándares, pautas y normas para la gestión de proyectos.

Microsoft Project permite conectar con Microsoft Project Online y Project Server, vinculando y adquiriendo bondades de administración centralizada del portafolio de proyectos y otras grandes funcionalidades para administrar, proyectos, programas y portafolios.

Redmine



Fuente: https://commons.wikimedia.org/wiki/File:Redmine_logo.svg

Redmine es una herramienta para la gestión de proyectos multiproyecto y multiusuario. Es posible optimizar su funcionamiento agregando funcionalidades.



Incluye:

- Sistema de seguimiento de incidentes con seguimiento de errores.
- Calendario de actividades.
- Diagramas de Gantt.
- Wiki.
- Foro.
- Visor del repositorio de control de versiones.
- RSS.
- Control de flujo de trabajo basado en roles.
- Integración con correo electrónico.
- Etcétera.

Está escrito usando el *framework* Ruby on Rails. Es *software* libre y de código abierto, disponible bajo la licencia pública general de GNU v2.

3.1.3.2. M-CASE

En este nivel se ubican las actividades referentes a la etapa de diseño y modelado. Algunas incluso atraviesan el nivel, generando código a partir de los diseños.

Algunas de las características deseables son:

- **Diseño orientado a objetos UML:**

Es una metodología basada en el lenguaje de modelado UML, que permite, construir y documentar el conjunto de elementos que forman un desarrollo de *software* orientado a objetos. Esta notación ha sido ampliamente aceptada.

- **Adaptar el proceso:**

Cada proceso deberá adaptarse a las necesidades de cliente, considerando como lo más importante la interacción con éste.

- **Elevar el nivel de abstracción:**

Al elevar el nivel de abstracción, se reduce la complejidad, así como la cantidad de documentación requerida por el proyecto.

Beneficios:

- Productividad.
- Reducción de la complejidad.



Patrones:

- Reusar activos ya existentes.
- Usar herramientas y lenguajes de alto nivel para reducir la cantidad de documentación.
- Enfocarse primero en la arquitectura.
- Realizar la arquitectura pensando en resiliencia, calidad, simplicidad y control de la complejidad.

Antipatrones:

- Ir directamente de requerimientos vagos de alto nivel a código hecho a mano.
- Dado que se utilizan pocas abstracciones, existen conflictos entre el nivel de código y el nivel conceptual, por lo que se pierden oportunidades de reutilización.
- La captura informal de requerimientos y de otra información hace necesario el revisar las especificaciones continuamente.
- Si se hace poco énfasis en la arquitectura se incurrirá en *retrabajos* en el futuro.

- **Diseño de la base de datos:**

Para el desarrollo de un *software* es importante que se pueda emplear una herramienta que permita generar las bases de datos que se utilizarán dentro del proceso.

También es importante que este programa permita manipularla, consultarla y crear diagramas que puedan ser consultados y modificados conforme se avanza en la etapa de desarrollo.

Veamos algunas de las herramientas más importantes de M-CASE.

Visio



Fuente:
https://fr.m.wikipedia.org/wiki/Fichier:Visio_2016.png



Microsoft Visio es un *software* de pago que funciona con el sistema Windows y que fue ideado específicamente para crear todo tipo de gráficos y diagramas.

No es una herramienta destinada a la gestión de proyectos como tal. Se trata de un programa de dibujo vectorial versátil y muy fácil de utilizar, pues ofrece numerosas posibilidades de edición.

Permite, entre otras cosas, diseñar:

- Diagramas de flujo.
- De procesos.
- Mapas conceptuales.
- Líneas de tiempo.
- Organigramas con gran facilidad.
- Diagramas UML a partir de bases de datos.
- Cronogramas de Gantt.

La presentación es visual, atractiva y moderna y los diseños son muy fáciles de hacer.

Otras funciones son:

- Puede agrupar varias tareas subordinadas.
- Incluye gran variedad de herramientas de diseño, modernas plantillas y diferentes formatos.
- Facilita el trabajo en equipo.
- Sirve para comunicar los proyectos y su desarrollo a los diferentes equipos y personas.

Inconvenientes:

- Es una herramienta de pago.
- Aunque es compatible con los productos de Microsoft Office (que son muy usados), no ofrece mucha compatibilidad con otras herramientas.
- La gran cantidad de herramientas que posee puede abrumar.



Lucidchart



Lucidchart es una herramienta de diagramación basada en la web, construida con estándares como HTML5 y JavaScript. Permite a los usuarios colaborar y trabajar juntos en tiempo real.

Cuenta con una gran colección de librerías de formas estándar. Estas librerías incluyen formas y plantillas para:

- Diagramas de flujo.
- Diagramas de red.
- Modelos y notación de procesos de negocio.
- Diagramas de circuitos.
- Planos.
- UML. Lenguaje unificado de modelado.
- Bocetos.
- Esquemas.
- Mapas mentales.
- Mapas conceptuales.
- Organigramas.
- Modelos de entidad-relación.
- Diagramas de Venn.

EASY CASE

Permite automatizar las fases de análisis y diseño, creando aplicaciones eficazmente, desde procesamiento de transacciones a la aplicación de bases de datos de cliente/servidor, así como sistemas de tiempo real.



Características:

- Permite generar esquemas de base de datos, capturar los detalles de diseño de un sistema y comunicar las ideas gráficamente, para que sean fáciles de ver y entender, y permite crear y mantener diagramas de flujo de datos, diagramas de entidad-relación, mapas de estructura etc. Se puede reusar diagramas o partes de diagramas para economizar el diseño de un proyecto.
- Posee herramientas de corrección avanzadas que permiten revisiones generales.
- Soporta una gama amplia de metodologías estructuradas, permitiendo escoger los métodos más apropiados para realizar las tareas.
- Determina los tipos de esquemas según la metodología del proyecto seleccionada y notifica de errores a medida que el modelo vaya construyéndose.
- Soporte comprensivo al modelado de datos, procesos y eventos.
- Posee desde el editor de diagramas flexible y un diccionario de los datos, así como una extensa cantidad de reportes y análisis.
- Es una herramienta multiusuario.
- Permite compartir datos y trabajar en un proyecto con otros departamentos. El equipo completo puede acceder a proyectos localizados en el servidor de la red concurrentemente.
- Para asegurar la seguridad de los datos, existe el diagrama y diccionario de los datos que bloquean por niveles al registro, al archivo y al proyecto, y niveles de control de acceso.

Cada vez que se crea un diagrama EasyCase automáticamente graba información en el diccionario de datos (DD).



Atención

EasyCASE Profesional, es un producto para la generación de esquemas de base de datos e ingeniería reversa, trabaja para proveer una solución comprensible para el diseño, consistencia y documentación del sistema en conjunto.

3.1.3.3. L-CASE

Son herramientas que semiautomatizan la generación de código, crean programas de detección de errores, soportan la depuración de programas y pruebas.

Además, automatizan la documentación completa de la aplicación (en todas las fases).



Aquí pueden incluirse las herramientas de desarrollo rápido de aplicaciones.

Son herramientas que automatizan o semiautomatizan:

- La generación de código.
- La creación de programas de detección de errores.
- El soporte a la depuración de programas y pruebas.
- La documentación completa de la aplicación.

Ya has aprendido que las Herramientas de tipo L-CASE, son herramientas que semiautomatizan la generación de código, crean programas de detección de errores, soportan la depuración de programas y pruebas.

Ahora vamos a ver las Pruebas realizables con Herramientas L-CASE.

3.1.3.3.1. Pruebas de análisis de código

Debemos asegurarnos de que el código funciona correctamente, por lo que debemos realizar pruebas de Análisis de código, para evaluarlo, esta evaluación supone buscar problemas de funcionamiento del código y mejorarlo.

Algunas de las características deseables son:

- **Generación de código:**

Algunos *frameworks* permiten generar código y formularios a partir de los procesos que se encuentran bien definidos a partir de determinados modelos.

También existen IDE que, mediante herramientas de arrastre *drag & drop*, permiten modelar formularios de *software* de manera automática.

- **Manejo de versiones y repositorios:**

En ocasiones, la codificación se realiza por varios integrantes de un equipo que comparten módulos.

Para ello se debe tener un repositorio central sobre el cual se irán haciendo modificaciones. Estas modificaciones se deben hacer de forma ordenada y sincronizada.

En esta tarea cobran importancia los conceptos de cohesión y acoplamiento.

- **Pruebas:**

Cuando el *software* se encuentra en su fase final es necesario realizar pruebas.



Generador de código

Un generador de código es una herramienta que genera parte del código en base a los diagramas y modelos del repositorio.

Las características más importantes de los generadores de código son:

- **Lenguaje generado:**
 - Lenguaje estándar.
 - Lenguaje propietario.
- **Portabilidad del código generado:** capacidad para poder ejecutarlo en distintas plataformas.
- **Generación del esqueleto del programa o del programa completo:** si únicamente genera el esqueleto será necesario codificar el resto.
- **Posibilidad de modificación del código generado:** suele ser aconsejable modificar el código generado para optimizarlo.
- **Generación del código del interfaz de usuario e informes de la aplicación.**

Pruebas a realizar

Se pueden realizar diferentes tipos de pruebas para asegurar el correcto funcionamiento de una aplicación.

Cuando el *software* se encuentra en su fase final es necesario realizar pruebas:

- De diseño.
- Depuración.
- Eficiencia.
- Complejidad.

Pruebas de código

Se pueden realizar las pruebas de código de un *software* de diversos modos.

Cuando el análisis es realizado por un humano, es llamado comprensión de programas (o entendimiento de programas) y también se le conoce como revisión de código.

Si se realiza de forma automática mediante aplicaciones dedicadas a ello, puede clasificarse en análisis de código estático y dinámico.



- **Análisis Estático de código:**

No se necesita ejecución del código, detecta errores en una fase muy temprana de la escritura, lo que ahorra tiempo en posteriores fases.

Tiene un problema importante al realizarse de este modo, y es que puede arrojar positivos que no lo son y que solo sabremos su falsedad cuando se realice la ejecución del código.

Es necesario, especialmente en este tipo, tener una buena gestión de la documentación, y su mantenimiento.

Es un tipo de análisis de *software* que se realiza sin ejecutar el programa.

En la mayoría de los casos, el análisis se realiza en alguna versión del código fuente y en otros casos se realiza en el código objeto.

El término se aplica generalmente a los análisis realizados por una herramienta automática, el análisis realizado por un humano es llamado comprensión de programas (o entendimiento de programas) como también revisión de código.

La sofisticación de los análisis realizados por las herramientas varía de aquellos que sólo tienen en cuenta el comportamiento de instrucciones y declaraciones individuales, a los que se incluye el código fuente completo de un programa en su análisis.

Los usos de la información obtenida de un análisis varían desde indicar posibles errores de codificación hasta demostrar matemáticamente con métodos formales ciertas propiedades acerca de un programa dado (por ejemplo, que su comportamiento coincide con el de su especificación) dependiendo de qué programa se utilice para el análisis.

Las métricas del *software* y la ingeniería inversa pueden ser descritas como forma de análisis estático de *software*. El análisis estático y las métricas del *software* se han comenzado a desarrollar a la par, sobre todo en sistemas embebidos donde se definen lo que se llama objetivos del *software* de calidad.

- **Análisis Dinámico de código:**

Se realiza mientras el código se está ejecutando, permite ver muchos errores que quedan ocultos en un análisis estático, pero es más lento ya que necesita un proceso completo de testeo.

Estos análisis, independientemente del tipo de prueba a realizar, necesitan un equipo de control de calidad que los lleve a cabo. Debe haber independencia entre los desarrolladores y el equipo de pruebas, para obtener un buen resultado de análisis.

El Análisis dinámico de *software* es un tipo de análisis de *software* que supone la ejecución del programa y observar su comportamiento.

Para que el análisis dinámico resulte efectivo el programa a ser analizado se debe ejecutar con los suficientes casos de prueba como para producir un comportamiento interesante, se pueden usar varias estrategias de pruebas de *software* para lograr esto tales como cobertura de código o simplemente programas conocidos como *fuzzers* que ayudan a asegurar que una porción adecuada del conjunto de posibles comportamientos del programa ha sido observada.



Otras herramientas en vez de probar casos de pruebas buscan a otros tipos de deficiencias en el *software*.

Tipos de estos análisis dinámicos son:

- **De caja negra.**

El objetivo de estas pruebas es comprobar que las salidas son correctas. No se presta atención al modo en que dichas salidas se realizan. Se atiende a una independencia modular para una implementación más fácil de cada módulo. Así resulta más sencillo abordar el fallo.

- **De caja blanca.**

Se centran en los fallos de procedimiento relativos a las entradas. El método suele consistir en realizar todas las entradas posibles para obtener una salida determinada. Este tipo de pruebas debe modificarse cada vez que varía la implementación en el proyecto.

SonarQube

Solución diseñada para realizar análisis estático de código fuente de manera automatizada.

Conocido anteriormente como Sonar, es una plataforma para evaluar código fuente. Es software libre y usa diversas herramientas de análisis estático de código fuente como Checkstyle, PMD o FindBugs para obtener métricas que pueden ayudar a mejorar la calidad del código de un programa. Es una herramienta de análisis de seguridad y calidad del código. Ejecuta análisis automáticos de código fuente para lenguajes como Java, Javascript, C#, Python, Ruby, etc. Su cometido es el localizar y reportar automáticamente vulnerabilidades de seguridad, bugs y problemas de codificación. Evalúa la mantenibilidad y simplicidad del código. Asimismo proporciona paneles de informes que permiten a desarrolladores y equipos de desarrollo visualizar la calidad del código fuente y los problemas a tratar.

Es un tipo de análisis estático de software, multiplataforma, programado en Java con licencia LGPL.



+ Info

La Licencia Pública General Reducida de GNU, (en inglés GNU Lesser General Public License), conocida como LGPL, es una licencia de software creada por la Free Software Foundation, que pretende garantizar la libertad de compartir y modificar el software cubierto por ella, asegurando que el software es libre para todos sus usuarios.



3.1.3.3.2. Pruebas de integración

La Integración continua de software: entrega continua desde el código hasta el despliegue, (en inglés, *Continuous Integration*.)

La finalidad de las pruebas de integración es verificar que los distintos componentes del sistema interactúan correctamente a través de sus interfaces.

Es una práctica de ingeniería de *software* que consiste en hacer integraciones (compilación y ejecución de pruebas de todo un proyecto), automáticas de un proyecto lo más a menudo posible para así poder detectar fallos cuanto antes.

El proceso suele realizarse del siguiente modo:

- Cada cierto tiempo (horas), descargarse las fuentes desde el control de versiones.
(Por ejemplo, CVS, Git, Subversion, Mercurial o Microsoft Visual SourceSafe).
- Compilarlo.
- Ejecutar pruebas.
- Generar informes.

Para realizar este proceso de integración continua, suelen utilizarse aplicaciones, que se encargan de controlar las ejecuciones, como: Solano CI, Bamboo, Pipeline, Apache Continuum, Hudson, Jenkins, GoCD, CruiseControl o Anthill (para proyectos Java) o CruiseControl.Net, Team Foundation Build para .Net.

Estas aplicaciones a su vez suelen apoyarse en otras herramientas que se encargan de realizar las compilaciones, ejecutar las pruebas y realizar los informes, como por ejemplo Ant o Maven (también para proyectos Java), o Nant o MSBUILD (para .Net).



Ejemplo

Bamboo, es una herramienta de integración continua y despliegue, que reúne compilaciones, pruebas y versiones automatizadas en un solo flujo de trabajo.



3.1.3.3.3. Pruebas de artefactos. (Diseño de software)

El término Artefacto, en conexión con el desarrollo de *software*, está mayormente asociado a métodos o procesos de desarrollo específicos, como el Proceso Unificado.

Un artefacto es un producto tangible resultante del proceso de desarrollo de *software*.

Algunos artefactos como los casos de uso, diagrama de clases u otros modelos UML ayudan a la descripción de la función, la arquitectura o el diseño del *software*.

Otros se enfocan en el proceso de desarrollo en sí mismo, como planes de proyecto, casos de negocios o enfoque de riesgos. El código fuente compilado para el testeo se suele considerar un artefacto, ya que el ejecutable es necesario para el plan de testeo.

En ocasiones un artefacto puede referirse a un producto terminado, como el código o el ejecutable, pero más habitualmente se refiere a la documentación generada a lo largo del desarrollo del producto en lugar del producto en sí.

Los artefactos pueden variar en su necesidad de mantenimiento y actualización:

- Los artefactos que detallan el diseño pretendido para el producto suelen realizarse al principio del proyecto y no necesitan mantenerse.
- Hay otros que se mantienen a lo largo del ciclo de vida con información que se actualiza durante el desarrollo.

3.1.3.3.4. Pruebas de implantación y aceptación del sistema

Con las pruebas de implantación debemos comprobar el funcionamiento correcto del sistema integrado de *hardware* y *software* en el entorno de operación.

Hay que conseguir, la aceptación del usuario, del sistema una vez instalado en su entorno real, satisfaciendo los requisitos de rendimiento, operación y coexistencia con el resto de los sistemas y seguridad.

Las pruebas de aceptación (*User Acceptance Testing, UAT*) pertenecen a las últimas etapas previas a la liberación en firme de versiones nuevas a fin de determinar si cumplen con las necesidades y/o requerimientos de las empresas y sus usuarios.

Al finalizar las pruebas automatizadas, que garantizan los requisitos tecnológicos del diseño inicial, se pasa a las pruebas manuales hechas por usuarios internos. Luego viene el acceso beta a los clientes que así lo soliciten repitiendo de nuevo el ciclo, pero estas son en entornos realistas.

Las pruebas de aceptación van mucho más allá de cuando finalmente se libera el *software* al público en general. Se presta atención en recabar los detalles y comentarios por medio de encuestas o envíos de datos estadísticos no sin antes presentar un cuadro de diálogo al usuario primerizo o con una versión nueva.



Pruebas Beta

Las pruebas beta, del inglés *beta testing*, son las pruebas de *software* que se realizan cuando el sistema está teóricamente correcto y pasa a ejecutarse en un entorno real.

Es la fase siguiente a las pruebas Alpha. No importa lo bueno que sea nuestro proceso de desarrollo, siempre habrá fallos que no han sido descubiertos por los desarrolladores ni por el equipo de pruebas. Las pruebas beta son pruebas para localizar esos problemas que no han sido detectados y poder corregirlos antes de liberar una versión. Debería ser realizada por usuarios finales. Dependiendo de la naturaleza del *software* podría, por ejemplo, ser realizada por compañeros de trabajo, por algunos clientes reales, o por una combinación de ambos.

Las herramientas de *betateesting* generan circunstancias extremas para distintos tipos de pruebas.

A los que realizan las pruebas beta se les suele llamar *betatester*.

Pruebas de regresión

Su propósito es asegurar que los casos de prueba que ya habían sido probados y fueron exitosos permanezcan así, que no se modifiquen al realizar otro tipo de pruebas automatizadas.

Las pruebas de regresión se pueden considerarse, como el subconjunto de pruebas planificadas, que se seleccionan para ser ejecutadas, generalmente de forma automática y periódicamente en cada nueva liberación del producto/*software*, teniendo como objetivo la verificación de que el producto no haya sufrido regresiones.

Este tipo de cambio puede ser debido a prácticas no adecuadas de control de versiones, falta de consideración acerca del ámbito o contexto de producción final y extensibilidad del error que fue corregido (fragilidad de la corrección), o simplemente una consecuencia del rediseño de la aplicación.

Cuando se localiza y corrige un bug, es recomendable, que se grabe una prueba que exponga el bug y se vuelvan a probar regularmente después de los cambios subsiguientes que experimente el programa.

Características:

- Ante cambios sobre un componente *software*, ayudan a garantizar que el resto de componentes no se ve afectado.
- Son compatibles con las metodologías ágiles de desarrollo.
- Suelen implicar la repetición de las pruebas que ya se han realizado previamente.
- Existen herramientas de *software* que permiten detectar este tipo de errores de manera parcial o totalmente automatizada.
- Lo normal es que, este tipo de pruebas se ejecuten en cada uno de los pasos del ciclo de vida del desarrollo del *software*.



Tipos de regresión:

- **Clasificación de ámbito:**
 - Local: los cambios introducen nuevos errores.
 - Desenmascarada: los cambios revelan errores previos.
 - Remota: los cambios vinculan algunas partes del programa (módulo) e introducen errores en ella.
- **Clasificación temporal:**
 - Nueva característica:

Los cambios realizados con respecto a nuevas funcionalidades en la versión introducen errores en otras novedades en la misma versión del software.
 - Característica preexistente:

Los cambios realizados con respecto a nuevas funcionalidades introducen errores en funcionalidad existente de previas versiones.

Como mitigar los riesgos:

- Repetición completa y habitual de la batería de pruebas, manual o mediante automatización.
- Repetición parcial basada en trazabilidad y análisis de riesgos.
- Pruebas de cliente o usuario:
 - Beta: distribución a clientes potenciales y actuales de versiones beta.
 - Pilot: distribución a un subconjunto bien definido y localizado.
 - Paralela: simultaneando uso de ambos sistemas.
- Parches de emergencia:

Estos parches se publican inmediatamente, y serán incluidos en releases (lanzamientos) de mantenimiento futuras.

3.1.4. Otras clasificaciones

Existen otros nombres que se le dan a este tipo de herramientas, y que no es una clasificación excluyente entre sí, ni con las fases del ciclo de vida del desarrollo:

- **Integrated CASE (I-CASE).**

Herramientas que engloban todo el proceso de desarrollo software, desde el análisis hasta la implementación.

(Se corresponde con la categoría *WorkBench* de la clasificación según su amplitud).



- **MetaCASE.**

Herramientas que permiten la definición de nuestra propia técnica de modelado. Los elementos permitidos del metamodelo generado se guardan en un repositorio y pueden ser usados por otros analistas, es decir, es como si definiéramos nuestro propio UML, con nuestros elementos, restricciones y relaciones posibles.

- **CAST (Computer-Aided Software Testing).**

Herramientas de soporte a la prueba de software.

- **IPSE (Integrated Programming Support Environment).**

Herramientas que soportan todo el ciclo de vida, incluyen componentes para la gestión de proyectos y gestión de la configuración activa.

3.2. Generar documentación

El módulo generador de la documentación se alimenta del repositorio para transcribir las especificaciones que este contiene.

Algunas características de los generadores de documentación son:

- **Generación automática:** genera la documentación de forma automática a partir de los datos del repositorio.
- **Combinación de información textual y gráfica:** esto facilita la comprensión de la documentación generada.
- **Generación de referencias cruzadas:** con ello se podrá localizar fácilmente en qué partes de la aplicación se encuentra un determinado objeto o elemento.

De esta forma, podemos analizar el impacto de un cambio o identificar los módulos afectados por un determinado error.

- **Ayuda de tratamiento de textos:** facilita la modificación de los textos o la introducción de textos complementarios a la documentación generada automáticamente.
- **Interfaz con otras herramientas:** debe poder conectarse con procesadores de textos, editores gráficos, etcétera.

3.2.1. Acuerdo de Nivel de Servicio (ANS)

También conocido por las siglas SLA, del inglés *Service Level Agreement*.

Es un documento que sirve para detallar las funciones y responsabilidades, así como los objetivos a los que se compromete un proveedor al proporcionar un servicio informático.



Es un acuerdo escrito, entre un proveedor de servicio y su cliente, con objeto de fijar el nivel acordado para la calidad de dicho servicio.

El ANS es una herramienta que ayuda a ambas partes a llegar a un consenso en términos del nivel de calidad del servicio, en aspectos tales como tiempo de respuesta, disponibilidad horaria, documentación disponible, personal asignado al servicio, etc.

Establece la relación entre ambas partes: proveedor y cliente. Un ANS identifica y define las necesidades del cliente a la vez que controla sus expectativas de servicio en relación a la capacidad del proveedor, proporciona un marco de entendimiento, simplifica asuntos complicados, reduce las áreas de conflicto y favorece el diálogo ante la disputa.

También constituye un punto de referencia para el proceso de mejora continua, ya que el poder medir adecuadamente los niveles de servicio es el primer paso para mejorarlos y de esa forma aumentar los índices de calidad, KPI etc.



+ Info

Un KPI, conocido también como indicador clave o medidor de desempeño o indicador clave de rendimiento. Es una medida del nivel del rendimiento de un proceso. El valor del indicador está directamente relacionado con un objetivo fijado previamente y normalmente se expresa en valores porcentuales.

3.3. Herramientas CASE más utilizadas

Existen múltiples herramientas Cases en el mercado, que pueden clasificarse según los criterios vistos anteriormente.

En este apartado vamos a nombrar algunas de las más utilizadas.

Ansible



ANSIBLE

Fuente:
Ansible_logo.svg de
Wikipedia



"Control de todo el ciclo de vida"

Ansible es una plataforma de software libre para configurar y administrar ordenadores.

Cuando se define la aplicación con Ansible y se maneja su despliegue con Ansible Tower es posible llevar un "control de todo el ciclo de vida de una aplicación", desde desarrollo hasta producción.

Combina:

- Instalación multinodo, es decir: permite desplegar configuraciones de servidores y servicios por lotes.
- Ejecuciones de tareas ad hoc.
- Administración de configuraciones.



Anécdota

El nombre lo puso DeHaan por el sistema de comunicación instantáneo del hiperespacio imaginado por Orson Scott Card en la novela El juego de Ender,⁷ y originalmente inventado por Ursula K. Le Guin en su novela de 1966 El mundo de Rocannon.

Generalmente Ansible es agrupado con otras herramientas de Gestión de la Configuración, no puede encasillarse solo a Gestión de la Configuración ya que puede ser usada en otros tipos de escenarios.

Características:

- **Aprovisionamiento:**

Con Ansible se pueden aprovisionar las últimas plataformas en la nube, host virtualizados e hipervisores, dispositivos de red y servidores físicos.

- **Gestión de la configuración:**

Establece y mantiene el rendimiento del producto, al registrar y actualizar la información que describe el software y hardware de una empresa. Esta información generalmente incluye las versiones y actualizaciones que se han aplicado a los paquetes de software instalados y las ubicaciones y direcciones de red de los dispositivos de hardware.



- **Seguridad y Cumplimiento:**

Ansible permite definir la seguridad en los sistemas de forma sencilla, es posible definir reglas de firewall, gestión de usuarios y grupos y políticas de seguridad personalizadas en los sistemas que se estén gestionando y además posee un gran número de módulos que ayudan en la labor.

- **Orquestación:**

Ansible se usa para orquestar los despliegues por ejemplo de OpenStack (proyecto de computación en la nube para proporcionar una infraestructura como servicio).

Es software libre y de código abierto distribuido bajo los términos de la licencia Apache).

Muchas Compañías confían en Ansible para mantener sus nubes disponibles de manera simple y segura.

La orquestación de servicios en la nube se refiere a la composición de los elementos del sistema para respaldar las actividades del proveedor de la nube en la disposición, coordinación y administración de los recursos informáticos con el propósito de proporcionar los servicios para los consumidores de la nube.



+ Info

La plataforma está incluida como parte de la distribución de Linux Fedora, heredada de Red Hat Inc.

Y también está disponible para Red Hat Enterprise Linux, CentOS y Scientific Linux a través de los Paquetes Extras para Enterprise Linux (EPEL) como también para otros sistemas operativos.



Es una herramienta de diseño de base de datos, que brinda productividad en diseño, generación, y mantenimiento de aplicaciones. Desde un modelo lógico de los requerimientos de información, hasta el modelo físico perfeccionado para las características específicas de la base de datos diseñada.



Permite visualizar la estructura, los elementos importantes, y optimizar el diseño de la base de datos.

Genera automáticamente las tablas y miles de líneas de stored procedure y triggers para los principales tipos de base de datos.

Oracle Designer

Es un juego de herramientas para guardar las definiciones que necesita el usuario y automatizar la construcción rápida de aplicaciones cliente/servidor flexibles y gráficas.

Integrado con Oracle Developer, Oracle Designer provee una solución para desarrollar sistemas empresariales cliente/servidor de segunda generación.

PowerDesigner

Aplicaciones de Powersoft para la construcción, diseño y modelado de datos a través. Es la herramienta para el análisis, diseño inteligente y construcción sólida de una base de datos y un desarrollo orientado a modelos de datos a nivel físico y conceptual, que dan a los desarrolladores de aplicaciones Cliente/Servidor la más firme base para aplicaciones de alto rendimiento.

System Architect

Posee un repositorio único que integra todas las herramientas, y metodologías usadas.

En la elaboración de los diagramas, conecta directamente al diccionario de datos, los elementos asociados, comentarios, reglas de validaciones, normalización, etc.

Posee control automático de diagramas y datos, normalizaciones y balanceo entre diagramas "Padre e Hijo", además de balanceo horizontal, que trabaja integrado con el diccionario de datos, asegurando la compatibilidad entre el Modelo de Datos y el Modelo Funcional.

SNAP

SNAP es un CASE para el desarrollo de aplicaciones en Sistemas AS/400 de IBM.

Proporciona el ambiente integral de trabajo, brindando la posibilidad de construir sistemas de inmejorable calidad, adheridos a los estándares S.A.A de IBM., totalmente documentados y ajustados a los requerimientos específicos de la organización, en una fracción del tiempo y coste del que se invertiría, si se utilizaran herramientas tradicionales.



Netbeans



Se clasifica como herramienta L-CASE, ya que son herramientas para el desarrollo de la aplicación.

NetBeans es un IDE libre creado principalmente para el lenguaje Java. (Permite crear aplicaciones Java de escritorio, móvil y web de forma fácil y sencilla).

NetBeans es un proyecto gratuito, de código abierto patrocinado por Sun Microsystems (que actualmente pertenece a Oracle Corporation).

La plataforma NetBeans permite que las aplicaciones sean desarrolladas a partir de un conjunto de componentes de software llamados módulos, (principalmente Date Table Visual) y también permite la creación de módulos por los desarrolladores.

Un módulo es un archivo Java que contiene clases de Java escritas para interactuar con las API de NetBeans y un archivo especial (*manifest file*) que lo identifica como módulo.

Las aplicaciones construidas a partir de módulos pueden ser extendidas agregándole nuevos módulos.

El NetBeans C/C++ Pack, soporta proyectos para C y C++ y el PHP Pack, soporta PHP 5. También permite utilizar HTML 5, JavaScript y CSS...

También permite importar proyectos realizados con Eclipse.

3.3.1. Herramientas de Análisis de Seguridad

SAST (Static Application Security Testing)

Se usa para el análisis del código fuente de una aplicación sin ejecutar. Examina el código buscando errores, debilidades y vulnerabilidades potencialmente explotables.

Es útil para encontrar problemas en fases tempranas antes de llevar la aplicación a producción. Detecta problemas como la exposición de datos sensibles, los errores de configuración o la vulnerabilidad a inyecciones SQL.

Entre otras herramientas existen Fortify, Checkmarx, SonarQube.



DAST (Dynamic Application Security Testing)

En este caso DAST se usa para identificar las vulnerabilidades en tiempo de ejecución, esto es en uso real. Subraya las deficiencias en la lógica de la aplicación y los problemas de configuración en tiempo de ejecución.

Entre otras herramientas encontramos Burp Suite, OWASP ZAP, Acunetix.

SCA (Software Composition Analysis)

Examina bibliotecas y componentes de terceros usados por la aplicación. Comprueba si las dependencias tienen vulnerabilidades conocidas o problemas legales asociados. Muy útil para la gestión del riesgo asociado con el uso de bibliotecas y componentes de terceros.

Herramientas habituales: Synk, WhiteSource, Black Duck.

SDLC (Secure Software Development Life Cycle)

Si bien no es una herramienta *per sé* si que es un enfoque que engloba las prácticas de seguridad en cada fase del desarrollo del software, desde el diseño hasta el mantenimiento, pasando por la implementación. Reduce la probabilidad de vulnerabilidades cubriendo todo el ciclo de vida del software.

4. Metodologías de desarrollo

La metodología es la forma establecida y detallada de llevar a cabo un trabajo.

El desarrollo de software tiene su propio ciclo de vida, denominado SDLC, siglas de Systems Development Life Cycle, también conocido como "System Design Life Cycle", en español ciclo vital del desarrollo/diseño de sistemas.

Existen diferentes modelos para la mejora y evaluación de procesos para el desarrollo, mantenimiento y operación de sistemas de software, como el modelo Agile y Capability Maturity Model Integration.

El resultado de la colaboración entre equipos autoorganizados y multifuncionales, son metodologías donde hay una evolución constante entre los requisitos y el desarrollo.

Las metodologías fomentan un enfoque disciplinado de administración de proyectos, donde se fomenta un conjunto de mejores prácticas, el método permite así, una entrega rápida de un producto o software de alta calidad, y, además, proporciona un enfoque comercial mejorado que alinea el desarrollo con las necesidades del cliente.



Las metodologías contrastan con la metodología de desarrollo tradicional, donde todos los requisitos se analizan y documentan inicialmente antes de que comience el desarrollo. Mientras tanto, en el enfoque de los requisitos son como los avances reales de desarrollo de proyectos o software dentro de cada iteración. Este enfoque proporciona flexibilidad para acomodar los cambios en los requisitos y prioridades de la empresa.

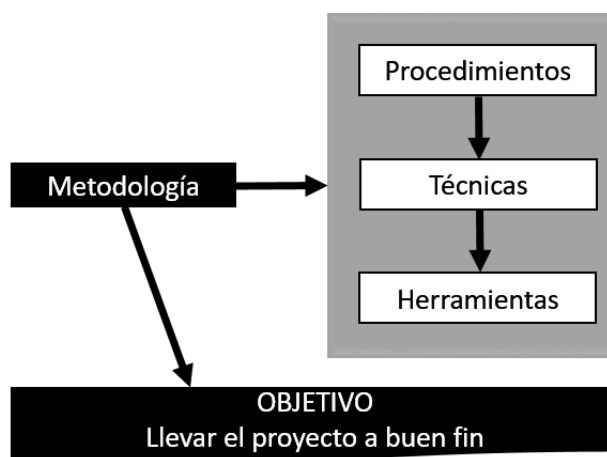


+ Info

Existen varias metodologías de desarrollo de proyectos o software. Dichas metodologías han sido influenciadas por el desarrollo iterativo e incremental.

Esto incluye programación extrema (XP), proceso unificado racional (RUP), Scrum y otros.

La metodología de desarrollo de software es un marco o forma determinada de trabajar estructurar, planificar y controlar el proceso de desarrollo en sistemas de información, con el objetivo de llegar a la realización correcta del proyecto planteado.



Esquema básico de una metodología

- Procedimientos: cada una de las fases de nuestro proyecto.
- Técnicas: diagramas y documentos de texto explicativos.
- Herramientas: de diferentes tipos, como realizar diagramas etc.



La gran complejidad de los sistemas informáticos hace necesario una vinculación entre diferentes desarrolladores (fabricantes) de software. Para que esta vinculación sea más cómoda y eficaz, se han creado una serie de metodologías para llevar a cabo el SDLC, con el objetivo de garantizar la creación de un producto correcto y libre de errores.

Existen multitud de modelos de desarrollo que pueden ser más o menos óptimos dependiendo del producto a desarrollar, la experiencia del equipo de trabajo, la estructura de la organización, etc.

- Modelo en cascada.
- Modelo en Espiral.
- Modelo en V.
- Modelo Iterativo Incremental.

También existen otras metodologías como son:

- **Las metodologías ágiles (Modelos AGILE).**

Se centran en los procesos de peso ligero, permitiendo la rápida evolución a lo largo del ciclo de desarrollo.

- **Metodologías iterativas, (como Rational Unified Process, RUP).**

Se centran en los ámbitos del proyecto limitado y la mejora o expansión de los productos de múltiples iteraciones.



+ Info

En la gestión de proyectos se definen el ciclo de vida del proyecto (PLC) y el SDLC, durante el cual suceden actividades.

Según Taylor (2004) «el ciclo de vida del proyecto abarca todas las actividades del proyecto, mientras que el ciclo de vida de desarrollo de sistemas se centra en el cumplimiento de los requisitos de los productos».



4.1. Gestión de proyectos de Tecnologías de la Información (TI)

4.1.1. Definición y características de los proyectos TI

La gestión de proyectos de Tecnologías de la Información (TI) consiste en la planificación, coordinación y ejecución de actividades orientadas a alcanzar objetivos específicos mediante el uso de recursos tecnológicos. Estos proyectos pueden involucrar el desarrollo, adaptación o implementación de sistemas informáticos, infraestructuras tecnológicas o soluciones digitales.

Una característica esencial de los proyectos TI es su naturaleza cambiante y compleja. Están sometidos a factores de incertidumbre técnica, dependencias interdisciplinarias y rápidos avances tecnológicos, lo que exige una planificación flexible y una supervisión constante. A diferencia de otros sectores, los proyectos TI pueden implicar tanto tareas tangibles (como la instalación de servidores) como intangibles (como el desarrollo de software), lo que complica su seguimiento y evaluación.

Entre los elementos comunes de este tipo de proyectos se encuentran:

- Objetivo delimitado en alcance y tiempo.
- Colaboración de perfiles técnicos diversos (desarrolladores, arquitectos, analistas, etc.).
- Gestión activa de riesgos, cambios y calidad.
- Participación de usuarios finales, cuya validación es crítica para el éxito.

4.1.2. Ciclo de vida del proyecto: inicio, planificación, ejecución, control y cierre

El ciclo de vida de un proyecto TI describe las fases por las que atraviesa desde su concepción hasta su finalización. Estas etapas proporcionan un marco estructurado para organizar el trabajo, facilitar su control y garantizar su mejora continua. Aunque pueden existir variaciones según la metodología utilizada, el modelo más común incluye cinco fases fundamentales:

- **Inicio:** En esta fase se establece el propósito del proyecto, se identifican las partes interesadas y se evalúa su viabilidad. El resultado principal es un documento de constitución que define el alcance preliminar, los recursos asignados y la aprobación formal para dar inicio al proyecto.
- **Planificación:** Durante esta etapa se especifican los requisitos técnicos y funcionales, se desglosan las tareas, se estiman los recursos necesarios y se establece el cronograma. Además, se definen los planes de comunicación, gestión de riesgos y control de calidad. Esta fase es determinante para el éxito del proyecto, ya que permite anticipar desafíos y establecer una hoja de ruta clara.



- **Ejecución:** Implica el desarrollo de las actividades previstas, coordinadas por el director del proyecto. Los equipos técnicos trabajan en la construcción de los entregables, mientras se mantiene una comunicación constante con los interesados y se gestionan las incidencias.
- **Seguimiento y control:** Esta fase ocurre en paralelo a la ejecución y consiste en monitorear el progreso del proyecto, identificar desviaciones respecto al plan inicial, implementar acciones correctivas y validar los resultados parciales. También incluye la gestión de cambios no previstos y la evaluación del desempeño del equipo.
- **Cierre:** Una vez completadas las tareas, se procede a validar los entregables, cerrar contratos, liberar recursos y documentar las lecciones aprendidas. Esta etapa asegura la finalización formal del proyecto y prepara el terreno para la sostenibilidad del producto o servicio desarrollado.

4.1.3. Metodologías de gestión de proyectos

Existen diferentes marcos metodológicos que proporcionan buenas prácticas y estructuras de trabajo para la gestión eficaz de proyectos. Entre los más utilizados en entornos TI destacan PMBOK y PRINCE2.

- **PMBOK (Project Management Body of Knowledge)**, desarrollado por el PMI (Project Management Institute), organiza la gestión de proyectos en diez áreas de conocimiento y cinco grupos de procesos. Este marco proporciona una visión holística que cubre aspectos como integración, alcance, cronograma, costes, calidad, recursos humanos, comunicación, riesgos, adquisiciones y gestión de interesados. Aunque no es una metodología rígida, su flexibilidad lo hace especialmente útil para proyectos complejos donde se requiere adaptabilidad.
- **PRINCE2 (Projects IN Controlled Environments, versión 2)** es una metodología basada en procesos, muy extendida en entornos europeos. Se centra en la justificación continua del negocio, la asignación clara de roles y responsabilidades, la gestión por fases y el aprendizaje a partir de cada etapa. Su enfoque escalable permite adaptarse tanto a proyectos grandes como a iniciativas más modestas.

Estas metodologías tradicionales no son excluyentes con enfoques ágiles como Scrum o Kanban. De hecho, es común implementar modelos híbridos, donde se emplean prácticas estructuradas (como PMBOK o PRINCE2) para la planificación estratégica, mientras se utilizan métodos ágiles para el desarrollo iterativo y la entrega incremental.

4.1.4. Herramientas de gestión de proyectos

El uso de herramientas digitales facilita enormemente la aplicación de estas metodologías, especialmente en contextos distribuidos, colaborativos o de alta complejidad. Existen plataformas que permiten gestionar tareas, cronogramas, recursos, comunicación y seguimiento del proyecto de manera centralizada.

Entre las más utilizadas se encuentran Trello y Jira, que ofrecen funcionalidades específicas para aplicar metodologías ágiles y visualizar el progreso de los trabajos. Estas herramientas permiten organizar tareas mediante tableros y columnas que representan estados del flujo de trabajo, asignar responsables, establecer plazos y generar métricas de rendimiento.



Aunque su funcionamiento detallado será tratado más adelante, conviene señalar aquí que herramientas como Trello resultan especialmente útiles en equipos que adoptan Kanban, gracias a su estructura visual y flexible, mientras que Jira está más orientado a entornos técnicos complejos, como el desarrollo de software, con control más estricto de versiones, seguimiento de incidencias y planificación de releases. En ambos casos, se trata de entornos escalables que contribuyen a mejorar la visibilidad del trabajo y la eficiencia del equipo.

4.2. Modelos de desarrollo

Vamos a ver el modelo de desarrollo en Cascada y en Espiral.

4.2.1. En Cascada

El modelo en cascada o Waterfall fue propuesto inicialmente por Winston W. Royce en 1970.

Es el modelo más antiguo, que fue considerado como "el Sistema para el Desarrollo del Ciclo de Vida", se define como:

"Una secuencia de etapas en las que la salida de cada etapa se convierte en la entrada para el siguiente".

Estas etapas suelen seguir los mismos pasos básicos, pero muchas metodologías diferentes en cascada suelen tener los mismos pasos con diferentes nombres y el número de pasos parecen variar entre 4 y 7. No hay un modelo definitivo para el desarrollo de ciclo de vida de un sistema, pero todas las opciones tienen el mismo propósito.

Es el modelo de vida clásico del software y el más básico. Este modelo es muy útil pues ayuda a los desarrolladores a comprender qué es lo que tienen que hacer en cada momento. Su simplicidad hace que resulte sencillo explicárselo a los clientes que no están familiarizados el proceso software.

Características del modelo en cascada:

- Fue el primero en popularizarse y es considerado el padre de los modelos de ciclo de vida del Software.
- Está formado por una serie de etapas que se suceden de forma estrictamente secuencial, es decir, hasta que una etapa no se ha completado no es posible pasar a la siguiente.
- Cada una de las etapas es atómica, tiene un inicio y un fin determinados y no es posible retomarlas una vez finalizadas o repetirlas.
- Requiere conocer de antemano todos los requisitos del sistema, lo cual es un inconveniente.
- Sólo es aplicable a pequeños desarrollos, ya que las etapas pasan de una a otra sin retorno posible.



Antes de poder avanzar a la siguiente etapa, es necesario haber finalizado completamente la etapa anterior (se presupone que no habrá errores ni variaciones del software). Si hay algún error durante el proceso hay que empezar desde el principio.

- En su definición formal, no permite modificaciones ni actualizaciones del software.

Inevitablemente, cuando se utiliza este modelo, hay que retomar a situaciones anteriores porque hay requisitos que hay que volver a restablecer, por ello existe una variación del modelo en cascada que es llamado modelo en cascada con retroalimentación.

Es un modelo más real puesto que se adapta a las situaciones reales de los desarrollos de software.

Modelo en cascada con realimentación

Es uno de los modelos más utilizados, surge a partir del modelo en Cascada, pero, pero se introduce una realimentación entre etapas, de forma que podamos volver atrás en cualquier momento para modificar o depurar algún aspecto.

Es un modelo más real puesto que se adapta a las situaciones reales de los desarrollos de software.

Una modificación consiste en la introducción de una revisión y vuelta atrás, con el fin de corregir las deficiencias detectadas durante las distintas etapas, o para completar o aumentar las funcionalidades del sistema en desarrollo.

No obstante, si se realizan muchos cambios durante el desarrollo de la aplicación, éste no es el modelo más idóneo.

Es el modelo perfecto si el proyecto es rígido (pocos cambios, poco evolutivo) y los requisitos están claros.

4.2.1.1. Ventajas e inconvenientes de los Modelos en Cascada

Las **ventajas** más destacadas son:

- Modelos sencillos y disciplinados.
- Fácil de aprender a utilizarlos y comprender sus funcionamientos.
- Dirigidos por los tipos de documentos y resultados que deben obtenerse al final de cada etapa.
- Muy usados y, por tanto, ampliamente contrastados.
- Ayuda a detectar errores en las primeras etapas a bajo costo.
- Ayuda a minimizar los gastos de planificación, pues se realiza sin problemas.



Los **inconvenientes** más destacados son:

- Los proyectos raramente siguen el proceso lineal tal como se definía originalmente el ciclo de vida.
- Es difícil que el cliente exponga explícitamente todos los requisitos al principio.
- El cliente debe tener paciencia pues obtendrá el producto al final del ciclo de vida.
- No refleja exactamente cómo se programa realmente el sistema, en el que suele haber un gran componente iterativo.
- Puede resultar complicado regresar a etapas anteriores (ya acabadas) para realizar correcciones.
- El producto final obtenido puede que no refleje todos los requisitos del usuario.

4.2.2. Modelo Espiral

El desarrollo en espiral es un modelo de ciclo de vida del software definido por primera vez por Barry Boehm en 1986 en su artículo «A Spiral Model of Software Development and Enhancement».

Básicamente consiste en una serie de ciclos que se repiten en forma de espiral, comenzando desde el centro.

Se suele interpretar como que dentro de cada ciclo de la espiral se sigue un Modelo Cascada, pero no necesariamente debe ser así.

El Espiral puede verse como un modelo evolutivo que conjuga dos conceptos:

- La naturaleza iterativa del modelo MCP.
(El Modelo de prototipos, pertenece a los modelos de desarrollo evolutivo. El prototipo debe ser construido en poco tiempo, usando los programas adecuados y no se debe utilizar muchos recursos).
- Y los aspectos controlados y sistemáticos del Modelo Cascada, agregando la gestión de riesgo.

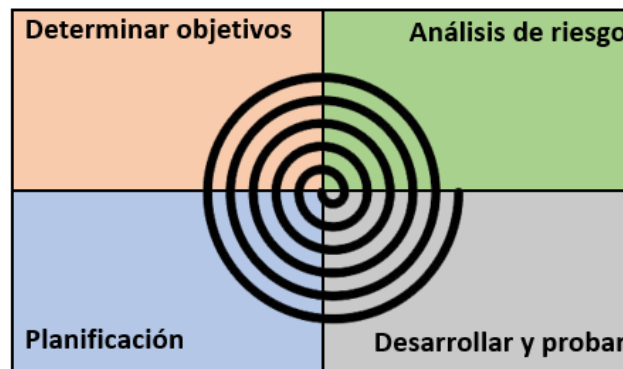
Este modelo tiene en cuenta fuertemente el riesgo que aparece a la hora de desarrollar software se comienza mirando las posibles alternativas de desarrollo, se opta por la de riesgo más asumible y se hace un ciclo de la espiral.

Si el cliente quiere seguir haciendo mejoras en el software, se vuelve a evaluar las distintas nuevas alternativas y riesgos y se realiza otra vuelta de la espiral, así hasta que llegue un momento en el que el producto software desarrollado sea aceptado y no necesite seguir mejorándose con otro nuevo ciclo.

Las actividades de este modelo se conforman en una espiral, en la que cada bucle o iteración representa un conjunto de actividades.



Las actividades no están fijadas a ninguna prioridad, sino que las siguientes se eligen en función del análisis de riesgo, comenzando por el bucle interior.



Ciclos o Iteraciones

En cada vuelta o iteración hay que tener en cuenta:

- **Los Objetivos:**

Qué necesidad debe cubrir el producto.

- **Alternativas:**

Las diferentes formas de conseguir los objetivos de forma exitosa, desde diferentes puntos de vista como pueden ser:

- Características: experiencia del personal, requisitos a cumplir, etc.
- Formas de gestión del sistema.
- Riesgo asumido con cada alternativa.

- **Desarrollar y Verificar:** Programar y probar el software.

Si el resultado no es el adecuado o se necesita implementar mejoras o funcionalidades:

- Se planificarán los siguientes pasos y se comienza un nuevo ciclo de la espiral. La espiral tiene una forma de caracola y se dice que mantiene dos dimensiones, la radial y la angular:
 - » Angular: Indica el avance del proyecto del software dentro de un ciclo.
 - » Radial: Indica el aumento del coste del proyecto, ya que con cada nueva iteración se pasa más tiempo desarrollando.



Este sistema es muy utilizado en proyectos grandes y complejos como la creación de un Sistema Operativo.



+ Info

Al ser un modelo de Ciclo de Vida orientado a la gestión de riesgo se dice que uno de los aspectos fundamentales de su éxito radica en que el equipo que lo aplique tenga la necesaria experiencia y habilidad para detectar y catalogar correctamente los riesgos.

Tareas

Para cada ciclo habrá cuatro actividades:

- **Determinar objetivos:**
 - Fijar también los productos definidos a obtener: requisitos, especificación, manual de usuario.
 - Fijar las restricciones.
 - Identificación de riesgos del proyecto y estrategias alternativas para evitarlos.
 - Hay una cosa que solo se hace una vez: planificación inicial.

- **Análisis del riesgo.**

Hay tener en cuenta los riesgos de cada uno de los ámbitos.

- Se realiza el estudio de las causas de las posibles amenazas y probables eventos no deseados y los daños y consecuencias que éstas puedan producir.
- Se evalúan alternativas.

Se debe tener un prototipo antes de comenzar a desarrollar y probar.

Dependiendo del resultado de la evaluación de los riesgos, se elige un modelo para el desarrollo, el que puede ser cualquiera de los otros existentes, como formal, evolutivo, cascada, etc.



Por ejemplo:

- Si los riesgos en la interfaz de usuario son dominantes, un modelo de desarrollo apropiado podría ser la construcción de prototipos evolutivos.
- Si los riesgos de protección son la principal consideración, un desarrollo basado en transformaciones formales podría ser el más apropiado.
- **Desarrollar y probar.**
 - Tareas de la actividad propia y de prueba.
 - Análisis de alternativas e identificación resolución de riesgos.
- **Planificación.**

Variaciones del modelo en espiral

Existen variaciones sobre el modelo en espiral, como son:

- **Modelo en Espiral Típico de seis regiones.**

El modelo en espiral puede adaptarse y aplicarse a lo largo de la vida del software, a diferencia del modelo de proceso clásico que termina cuando se entrega el software.

Las seis regiones que componen este modelo son las siguientes:

- **Comunicación con el cliente.**

Tareas necesarias para plantear la comunicación entre el desarrollador y el cliente.

- **Planificación.**

Tareas inherentes a la definición de recursos, el tiempo y otras informaciones relacionadas con el proyecto. Son todos los requerimientos.

- **Análisis de riesgos.**

Tareas para evaluar riesgos técnicos y otras informaciones relacionadas con el proyecto.

- **Ingeniería.**

Tareas para construir una o más representaciones de la aplicación.

- **Construcción y adaptación.**

Tareas requeridas para construir, probar, instalar y proporcionar soporte a los usuarios.



- **Evaluación del cliente.**

Tareas requeridas para obtener la reacción del cliente según la evaluación de las representaciones del software creadas durante la etapa de ingeniería e implementación durante la etapa de instalación.

- **Modelo en espiral WIN-WIN.**

El modelo Win-Win deriva su nombre del objetivo de las negociaciones entre cliente y desarrolladores, es decir, "ganar-ganar".

Es una adaptación del modelo espiral que se enfatiza en la participación del cliente en el proceso de desarrollo.

En un caso ideal, el desarrollador simplemente pregunta al cliente lo que se requiere y el cliente proporciona suficiente información y detalles para proceder.

Sin embargo, en la práctica esto no suele ocurrir, y es necesario que se establezcan negociaciones significativas entre ambas partes para equilibrar la funcionalidad y rendimiento con los costos y tiempo de salida al mercado del producto.

El cliente recibe el producto que satisface la mayoría de sus necesidades, y el desarrollador trabaja para alcanzar presupuestos y fechas de entrega. Para lograr este objetivo, se realizan varias actividades de negociación al principio de cada paso alrededor de la espiral.

Además del énfasis inicial puesto en la condición de ganar-ganar, el modelo también presenta tres etapas del proceso (puntos de anclaje), que son:

- **Objetivos del Ciclo de Vida (Life Cycle Objectives - LCO).**

Define una serie de objetivos para cada actividad de software más importantes (un conjunto de objetivos relacionados con la definición de los principales requisitos de nivel de producto).

- **Arquitectura del Ciclo de Vida (Life Cycle Architecture - LCA).**

Establece los objetivos que deben cumplirse como la arquitectura de software se define.

- **Initial Operational Capability (Initial Operational Capability - IOC).**

Representa un conjunto de objetivos asociados a la preparación del software para la instalación y distribución, la preparación previa a las instalaciones del sitio, asistencia requerida por todas las partes que utilizará o soporte técnico del software.

Estos tres puntos de anclaje ayudan a establecer la realización de un ciclo alrededor de la espiral y proporcionar los hitos de decisión antes de que el proyecto de producto de software.



4.2.2.1. Ventajas e inconvenientes del Modelo en Espiral

Las **ventajas** más destacadas son:

- El análisis del riesgo se hace de forma explícita y clara.
- Reduce riesgos del proyecto antes de que se conviertan en problemáticos.
- Une los mejores elementos de los restantes modelos.
- Es un enfoque realista del desarrollo del software.
- Incorpora objetivos de calidad.
- Integra el desarrollo con el mantenimiento, etc.
- Añade la posibilidad de tener en cuenta mejoras y nuevos requerimientos sin romper con la metodología, ya que este ciclo de vida no es rígido ni estático.
- Conjuga la naturaleza iterativa de los prototipos con los aspectos controlados y sistemáticos del modelo clásico.
- Proporciona el potencial para el desarrollo rápido de versiones incrementales.
- Puede adaptarse y aplicarse a lo largo de la vida del software.

Los **inconvenientes** más destacados son:

- Genera mucho tiempo en el desarrollo del sistema.
- Es un modelo costoso.
- Requiere experiencia en la identificación de riesgos.

4.2.3. Desarrollo ágil de software

El desarrollo ágil de software envuelve un enfoque para la toma de decisiones en los proyectos de software, que se refiere a métodos basados en el desarrollo iterativo e incremental, donde los requisitos y soluciones evolucionan con el tiempo según la necesidad del proyecto.



+ Info

Desarrollo iterativo y creciente (o incremental) es un proceso de desarrollo de para superar las debilidades del modelo tradicional de cascada.



Este modelo es un conjunto de tareas agrupadas en pequeñas etapas repetitivas (iteraciones), es uno de los más utilizados, empleado en metodologías diversas.

El modelo consta de diversas etapas de desarrollo en cada incremento, las cuales inician con el análisis y finalizan con la instauración y aprobación del sistema.

El trabajo es realizado mediante la colaboración de equipos auto organizados y multidisciplinarios, inmersos en un proceso compartido de toma de decisiones a corto plazo.

Cada iteración del ciclo de vida incluye planificación, análisis de requisitos, diseño, codificación, pruebas y documentación.

Adquiere una gran importancia el concepto de "finalizado" (done), ya que el objetivo de cada iteración no es agregar toda la funcionalidad para justificar el lanzamiento del producto al mercado, sino incrementar el valor por medio de "software que funciona" (sin errores).

Los métodos ágiles enfatizan las comunicaciones cara a cara en vez de la documentación.

La mayoría de los equipos ágiles están localizados en una simple oficina abierta, a veces llamadas "plataformas de lanzamiento" (bullpen en inglés).

Los métodos ágiles también enfatizan que el software funcional es la primera medida del progreso, combinado con la preferencia por las comunicaciones cara a cara, (en ocasiones los métodos ágiles son criticados y tratados como "indisciplinados" por la falta de documentación técnica).

Evolución del uso....

La definición moderna de desarrollo ágil de software evolucionó a mediados de la década de 1990 como parte de una reacción contra los métodos de "peso pesado", muy estructurados y estrictos, extraídos del modelo de desarrollo en cascada.

El proceso originado del uso del modelo en cascada era visto como burocrático, lento, degradante e inconsistente con las formas de desarrollo de software que realmente realizaban un trabajo eficiente.

Los métodos de desarrollo ágiles e iterativos pueden ser vistos como un retroceso a las prácticas observadas en los primeros años del desarrollo de software (aunque en ese tiempo no había metodologías para hacerlo).

En el año 2001, miembros prominentes de la comunidad se reunieron en Snowbird, Utah, y adoptaron el nombre de "métodos ágiles".



Poco después, algunas de estas personas formaron la "alianza ágil", una organización sin fines de lucro que promueve el desarrollo ágil de aplicaciones. Muchos métodos similares al ágil fueron creados antes del 2000.

Entre los métodos ágiles de desarrollo más destacados se encuentran:

- Agile Unified Process.
- Feature Driven Development (FDD).
- Lean Software Development (LSD): Lean startup.
- Kanban (desarrollo).
- Open Unified Process (OpenUP).
- Scrum (1986).
- Crystal Clear (transparente como el cristal).
- programación extrema (en inglés eXtreme Programming o XP, 1996).
- desarrollo de software adaptativo.
- feature-driven development.
- método de desarrollo de sistemas dinámicos (DSDM) (1995).

Agile

Metodología para el desarrollo de proyectos que precisan de rapidez y flexibilidad.

Es una filosofía que supone una forma distinta de trabajar y de organizarse, de forma que cada proyecto se divide en pequeñas partes que tienen que realizarse y entregarse en pocas semanas.

El objetivo es desarrollar software de calidad que respondan a las necesidades de unos clientes cuyas prioridades cambian rápidamente.

Las principales ventajas del "Agile" son:

- **Mejora la calidad:** Minimiza los errores en los entregables y mejora la experiencia y la funcionalidad para el cliente.
- **Mayor compromiso:** Mejora la satisfacción del empleado y genera conciencia de equipo.
- **Rapidez:** Acorta los ciclos de producción y minimiza los tiempos de reacción y toma de decisiones.
- **Aumento de la productividad:** Al asignar mejor los recursos, y de forma más dinámica, mejora la producción según las prioridades que tenga la empresa.

Agile describe un conjunto de principios rectores que utilizan un enfoque iterativo.



4.2.3.1. Marco de trabajo Scrum

Scrum es un marco de trabajo para desarrollo ágil de software.

Es un proceso en el que se aplican de manera regular un conjunto de buenas prácticas para trabajar colaborativamente, en equipo y obtener el mejor resultado posible de proyectos, caracterizado por:

Adoptar una estrategia de desarrollo incremental, en lugar de la planificación y ejecución completa del producto.

Basar la calidad del resultado más en el conocimiento tácito de las personas en equipos auto organizados, que en la calidad de los procesos empleados.

Solapar las diferentes fases del desarrollo, en lugar de realizar una tras otra en un ciclo secuencial o en cascada.

El núcleo central del marco de trabajo 'Scrum' es el 'sprint'.

Sprint es el nombre de cada uno de los ciclos o iteraciones que tenemos dentro de un proyecto que estamos construyendo, y que tiene que dar como resultado un incremento del producto que aporte valor al cliente, lo que se denomina un entregable al cliente, debe ser un producto que esté funcionando.

Un sprint, tiene un tiempo de duración prefijado, que suele ser entre una y cuatro semanas (aunque la metodología indica que debería estar entre dos semanas y un máximo de dos meses).



Atención

Puedes consultar información en su web oficial:

<https://www.scrum.org/>

Origen e historia

Este modelo de desarrollo fue identificado y definido por Ikujiro Nonaka y Takeuchi a principios de los 80, al analizar cómo desarrollaban los nuevos productos las principales empresas de manufactura tecnológica: Fuji-Xerox, Canon, Honda, NEC, Epson, Brother, 3M y Hewlett-Packard (Nonaka & Takeuchi, The New Product Development Game, 1986).



En su estudio, Nonaka y Takeuchi compararon la nueva forma de trabajo en equipo, con el avance en formación de melé (scrum en inglés) de los jugadores de Rugby, a raíz de lo cual quedó acuñado el término "scrum" para referirse a ella.

Aunque esta forma de trabajo surgió en empresas de productos tecnológicos, es apropiada para cualquier tipo de proyecto con requisitos inestables y para los que requieren rapidez y flexibilidad, situaciones frecuentes en el desarrollo de determinados sistemas de software.

En 1995, Ken Schwaber presentó "Scrum Development Process" en OOPSLA 95 (Object-Oriented Programming Systems & Applications conference) (SCRUM Development Process), un marco de reglas para desarrollo de software, basado en los principios de Scrum, y Jeff Sutherland en su empresa Easel Corporation (compañía que, en los macrojuegos de compras y fusiones, se integraría en VMARK, y luego en Informix y finalmente en Ascential Software Corporation).

4.2.3.1.1. Características, metodología y roles

Scrum es un marco de trabajo que define un conjunto de prácticas y roles, y que puede tomarse como punto de partida para definir el proceso de desarrollo que se ejecutará durante un proyecto.

Las principales características que hacen que Scrum sea utilizado de manera regular en un conjunto de buenas prácticas para el trabajo en equipo y de esa manera obtener resultados posibles son:

- Gestión regular de las expectativas del cliente, resultados anticipados, flexibilidad y adaptación, retorno de inversión, mitigación de riesgos, productividad y calidad, o, equipo motivado.
- Se hace uso de equipos auto-dirigidos y auto-organizados.
- Se realiza a diario una reunión de Scrum, que es una reunión de avance diaria que no dura más de 15 minutos con el objetivo de obtener realimentación sobre las tareas del equipo y los obstáculos que se presentan.
- Existen varias implementaciones de sistemas para gestionar el proceso de Scrum que permite que una persona, de un solo vistazo, pueda ver en qué están trabajando los demás en un momento determinado como son:
 - **Notas amarillas "post-it".**
 - **Pizarras.**
 - **Paquetes de software** que requieren poco esfuerzo para comenzarse a utilizar.

De esta forma, utilizando una pizarra con notas autoadhesivas cualquier miembro del equipo podrá ver tres columnas: trabajo pendiente ("backlog"), tareas en proceso ("in progress") y hecho ("done").

La metodología utilizada, permite la creación de equipos auto organizados impulsando la co-localización de todos los miembros del equipo, y la comunicación verbal entre todos los miembros y disciplinas involucrados en el proyecto.



La metodología se basa en:

- El desarrollo incremental de los requisitos del proyecto en bloques temporales cortos y fijos.
- Se da prioridad a lo que tiene más valor para el cliente.
- El equipo se sincroniza diariamente y se realizan las adaptaciones necesarias.
- Tras cada iteración (un mes o menos entre cada una) se muestra al cliente el resultado real obtenido, para que éste tome las decisiones necesarias en relación a lo observado.
- Se le da la autoridad necesaria al equipo para poder cumplir los requisitos.
- Fijar tiempos máximos para lograr objetivos.
- Equipos pequeños (de 3 a 9 personas cada uno).

En Scrum, se ejercen diferentes roles que pueden clasificarse como:

- **Roles principales:**
 - **Product Owner (o Propietario del producto).**

Representa a los stakeholders (interesados externos o internos).

El Product Owner se asegura de que el equipo Scrum trabaje de forma adecuada desde la perspectiva del negocio, y ayuda al usuario a escribir las historias de usuario, las prioriza, y las coloca en el Product Backlog.

Una historia de usuario es una representación de un requisito escrito en una o dos frases utilizando el lenguaje común del usuario. Las historias de usuario son utilizadas en las metodologías de desarrollo ágiles para la especificación de requisitos. Cada historia de usuario debe ser limitada, esta debería poderse escribir sobre una nota adhesiva pequeña. (Dentro de la metodología XP las historias de usuario deben ser escritas por los usuarios).

- **Scrum Master (o Facilitador).**

Es el responsable del cumplimiento de las reglas del marco scrum, es decir que procura facilitar la aplicación de Scrum y gestionar cambios.

Se asegura que éstas son entendidas por la organización y de que se realiza el trabajo conforme a ellas, eliminando los obstáculos que impiden que se desarrolle el objetivo del *sprint*.

Asesora y da la formación necesaria al propietario del producto y al equipo de desarrolladores.



- **Desarrollador.**

Cada uno de los profesionales que realizan la entrega del incremento de producto generado en cada sprint (denominado incremento).

Es recomendable un pequeño equipo de 3 a 9 personas con las habilidades transversales necesarias para realizar el trabajo (análisis, diseño, desarrollo, pruebas, documentación, etc.).

- **Roles Auxiliares.**

Los roles auxiliares en los "equipos Scrums" son los que ejercen los miembros del equipo (Team), que ejecutan el desarrollo y demás elementos relacionados.

Son aquellos que no tienen un rol formal y no se involucran frecuentemente en el "proceso Scrum", sin embargo, deben ser tomados en cuenta, ya que, en una metodología ágil, un aspecto importante es la práctica de involucrar en el proceso a los usuarios, expertos del negocio y otros interesados. Es importante que todos participen y entreguen retroalimentación con respecto a la salida del proceso a fin de revisar y planear cada sprint.

Se les denomina Stakeholders a Clientes, Proveedores, Vendedores, etc. que son las personas que hacen posible el proyecto y para quienes el proyecto producirá el beneficio acordado que justifica su desarrollo. Solo participan directamente durante las revisiones del "sprint".

4.2.3.1.2. Funcionamiento

Durante cada *sprint*, un periodo entre una y cuatro semanas (la magnitud es definida por el equipo y debe ser lo más corta posible), el equipo crea un incremento de software *potencialmente entregable* (utilizable).

El conjunto de características que forma parte de cada sprint viene del Product Backlog, que es un conjunto de requisitos de alto nivel priorizados que definen el trabajo a realizar (PBI, Product Backlog Item).

Procesos:

- Los elementos del *Product Backlog* que forman parte del sprint se determinan durante la reunión de *Sprint Planning*, en la que el *Product Owner* identifica los elementos del *Product Backlog* que quiere ver completados y los da a conocer al equipo.
- Entonces, el equipo conversa con el Product Owner buscando la claridad y magnitud adecuadas (Cumpliendo el INVEST).
- Luego se determina la cantidad de ese trabajo que puede comprometerse a completar durante el siguiente sprint.
 - Durante el sprint, nadie puede cambiar el Sprint Backlog, lo que significa que los requisitos están congelados durante el sprint.



Artefactos

- **Pila del producto (o product backlog).**

Registra y prioriza los requisitos del usuario. Empieza con una visión inicial del producto y crece y evoluciona durante el desarrollo del producto.

- **Pila del sprint (o sprint backlog).**

Registro de los requisitos desde el punto de vista de los desarrolladores. Es la lista de tareas que se deben realizar durante un sprint para lograr el incremento previsto.

- **Incremento.**

Es el resultado de cada sprint.

Flujo de trabajo

- **Sprint.**

Es el período en el cual se lleva a cabo el trabajo en sí.

Es recomendado que la duración de los sprints sea constante y definida por el equipo con base en su propia experiencia.

Al final de cada sprint, el equipo deberá presentar los avances logrados, y el resultado obtenido es un producto que, potencialmente, se puede entregar al cliente.

Se recomienda no agregar objetivos al sprint o *sprint backlog* a menos que su falta amenace al éxito del proyecto.

La constancia permite la concentración y mejora la productividad del equipo de trabajo.

- **Planificación de sprint.**

Al comienzo de un sprint, el equipo de scrum tiene un evento de planificación de sprint.

Uno de los objetivos de la reunión es identificar y comunicar cuánto del trabajo es probable que se realice durante el actual Sprint.

- **Scrum diario.**

También llamado Daily Standup.

Cada día durante la iteración, tiene lugar una reunión de estado del proyecto.



Su objetivo es que los miembros del equipo se mantengan actualizados unos a otros sobre el trabajo de cada uno desde el último standup, qué problemas han encontrado o prevén encontrar, y qué planean hacer.

- » La reunión tiene una duración fija de entre 5 y 15 minutos.
 - » Se recomienda hacerla de pie para recordar que debe ser una reunión breve y centrada en su objetivo, sin divagaciones. Es obligatorio parar todo lo que se está haciendo para concentrarse en la reunión.
 - » Si se requiere ampliar un tema, se hará tras el Daily Standup, pero no se interrumpe la dinámica del Standup para elaborar una discusión.
 - » Se hace siempre a la misma hora y en el mismo lugar. Si falta alguien, no se pospone la reunión.
- **Revisión de sprint.**

Al final de un sprint, el equipo realiza dos eventos: la revisión del sprint y la retrospectiva del sprint.

En la reunión de revisión de sprint se presentan los trabajos completados y su duración no debería ser superior a 4 horas para un Sprint de 1 mes.

- **Retrospectiva del sprint.**

Después de cada sprint, se lleva a cabo una retrospectiva del sprint (*Sprint Retrospective*), en la cual todos los miembros del equipo dejan sus impresiones sobre el sprint recién superado (qué ha ido bien, qué es lo que ha fallado, si ha habido malas prácticas, etc.).

El propósito de la retrospectiva es realizar una mejora continua del proceso.

Esta reunión tiene un tiempo fijo de cuatro horas.

Documentos

- **Product backlog.**

El product backlog se trata como un documento de alto nivel para todo el proyecto:

- Es el conjunto de todos los requisitos de proyecto, el cual contiene descripciones genéricas de funcionalidades deseables, priorizadas según su retorno sobre la inversión (ROI).
- Representa el qué va a ser construido en su totalidad. Es abierto y solo puede ser modificado por el product owner.
- Contiene estimaciones realizadas a grandes rasgos, tanto del valor para el negocio, como del esfuerzo de desarrollo requerido.



Esta estimación ayuda al product owner a ajustar la línea temporal (KEV) y, de manera limitada, la prioridad de las diferentes tareas.

Por ejemplo, si dos características tienen el mismo valor de negocio la que requiera menor tiempo de desarrollo tendrá probablemente más prioridad, debido a que su ROI será más alto.

- **Sprint backlog.**

El sprint backlog es el subconjunto de requisitos que serán desarrollados durante el siguiente sprint.

Al definir el sprint backlog, se describe el cómo el equipo va a implementar los requisitos durante el sprint.

Por lo general los requisitos se subdividen en tareas, a las cuales se asignan ciertas horas de trabajo, pero ninguna tarea con una duración superior a 16 horas. Si una tarea es mayor de 16 horas, deberá ser dividida en otras menores.

Las tareas en el sprint backlog nunca son asignadas, son tomadas por los miembros del equipo del modo que les parezca adecuado.

- **Burn down chart.**

La burn down chart es una gráfica mostrada públicamente que mide la cantidad de requisitos en el Backlog del proyecto pendientes al comienzo de cada Sprint.

Dibujando una línea que conecte los puntos de todos los Sprints completados, podremos ver el progreso del proyecto.

Lo normal es que esta línea sea descendente (en casos en que todo va bien en el sentido de que los requisitos están bien definidos desde el principio y no varían nunca) hasta llegar al eje horizontal, momento en el cual el proyecto se ha terminado (no hay más requisitos pendientes de ser completados en el Backlog).

Si durante el proceso se añaden nuevos requisitos la recta tendrá pendiente ascendente en determinados segmentos, y si se modifican algunos requisitos la pendiente variará o incluso valdrá cero en algunos tramos.

- **Definition of Done** (Definición de terminado).

El Definition of Done es un documento con una serie de criterios comunes para determinar cuándo una tarea está completamente hecha.



4.2.3.2. Herramientas y filosofías compatibles

4.2.3.2.1. Kanban

Kanban es un método ágil de gestión del trabajo que se centra en tres pilares fundamentales:

- **Visualización del flujo de trabajo.** El método Kanban utiliza un sistema visual basado en tarjetas y columnas que representan cada etapa del proceso productivo. Esta representación gráfica permite a los equipos identificar de un vistazo qué tareas están pendientes, en proceso o finalizadas. La disposición visual facilita la coordinación entre miembros del equipo y ayuda a detectar posibles bloqueos o retrasos en el flujo de trabajo.
- **Limitación del trabajo en progreso (WIP: Work In Progress).** Kanban establece límites estrictos sobre cuántas tareas pueden estar activas simultáneamente en cada fase del proceso. Esta restricción evita la sobrecarga del equipo y previene la acumulación de trabajos a medio terminar. Al completar una tarea antes de iniciar otra nueva, se mejora el enfoque y se optimiza el tiempo dedicado a cada actividad.
- **Optimización continua de la eficiencia.** El sistema promueve la mejora progresiva mediante el análisis periódico de métricas como el tiempo de ciclo o la tasa de entrega. Los equipos realizan ajustes incrementales para agilizar procesos, eliminar pasos innecesarios y resolver cuellos de botella. Esta filosofía de mejora continua permite adaptar el método a las necesidades cambiantes del proyecto.

Utiliza un tablero visual (físico o digital) organizado en columnas que representan las distintas etapas del proceso, donde cada tarea se representa mediante una tarjeta individual.

El principio esencial de Kanban radica en establecer límites estrictos al WIP, lo que previene la sobrecarga del equipo y garantiza un flujo de trabajo equilibrado.

Asimismo, promueve la mejora continua a través del análisis de métricas clave, como el tiempo de ciclo y la identificación sistemática de cuellos de botella.

A diferencia de metodologías más estructuradas como Scrum, **Kanban destaca por su flexibilidad inherente:** no requiere sprints predefinidos ni roles específicos, lo que lo convierte en la opción ideal para entornos con prioridades cambiantes.

Sin embargo, su efectividad depende crucialmente del compromiso del equipo para respetar los límites de WIP y mantener una total transparencia en el proceso.

Para su implementación, existen herramientas digitales como Trello o Jira, aunque incluso un simple tablero físico con post-its puede ser igualmente efectivo. En esencia, Kanban ofrece un enfoque ágil y altamente adaptable, particularmente valioso para equipos que buscan mejorar sus procesos de manera incremental sin necesidad de transformaciones radicales.



Trello

Trello es una plataforma digital diseñada para **gestionar proyectos y tareas de forma visual** e intuitiva. Su estructura se basa en un sistema de tableros, listas y tarjetas que permiten organizar el trabajo de manera clara y flexible.

Cada proyecto se representa mediante un tablero, donde las listas funcionan como columnas que dividen las etapas del proceso, como **"Pendiente", "En progreso" o "Finalizado"**. Dentro de estas listas, las tarjetas actúan como tareas individuales, las cuales pueden incluir descripciones, archivos adjuntos, fechas límite y asignaciones de responsables.

Una de las características más valoradas de Trello es su capacidad para adaptarse a diferentes necesidades, desde el trabajo en equipo hasta la planificación personal. Su **interfaz sencilla y su enfoque visual** lo hacen ideal para metodologías ágiles, gestión de proyectos profesionales o incluso para organizar actividades cotidianas.

Además, Trello ofrece funciones colaborativas, como la posibilidad de añadir comentarios, etiquetas de colores para priorizar tareas y recordatorios automáticos. Su **integración con otras herramientas populares**, como Google Drive o Slack, amplía su utilidad en entornos de trabajo diversos.

Jira

Jira es una plataforma de gestión de proyectos que **organiza el trabajo mediante proyectos** independientes con configuraciones personalizables. Cada proyecto contiene "issues" (tareas) de diversa complejidad, asignables a usuarios o equipos para distribuir responsabilidades.

El sistema permite gestionar **permisos de acceso con roles personalizados**, desde administradores hasta usuarios básicos, garantizando seguridad y control. Además, ofrece herramientas como etiquetas para clasificar tareas, subtareas para dividir trabajos complejos y "épicas" para agrupar objetivos estratégicos.

Para desarrollo de software, Jira incluye funciones como versiones, que vinculan tareas a releases específicos, facilitando el seguimiento del ciclo de vida del producto. Su flexibilidad permite adaptar flujos de trabajo, relaciones entre tareas y métricas de progreso, ofreciendo un entorno escalable para equipos que requieren precisión en la gestión. La plataforma integra todas estas funcionalidades en un sistema cohesivo, equilibrando estructura y adaptabilidad para proyectos de cualquier envergadura.

4.2.3.2.2. Lean Software Development

LEAN es una metodología de gestión que busca **maximizar el valor para el cliente** eliminando desperdicios (actividades que no agregan valor) y optimizando procesos. Su origen se remonta al Sistema de Producción de Toyota, pero hoy se aplica en manufactura, servicios, logística e incluso desarrollo de software.



El enfoque LEAN se sustenta en cuatro principios fundamentales

- **1 Definir el valor desde la perspectiva del cliente:** No se trata de lo que la empresa cree que es valioso, sino de lo que el cliente está dispuesto a pagar. Esto implica analizar necesidades reales y eliminar características o procesos superfluos.
- **2 Garantizar un flujo de trabajo continuo:** El objetivo es que los procesos fluyan sin interrupciones, evitando cuellos de botella, esperas o acumulaciones innecesarias entre etapas. Técnicas como el balanceo de líneas y la estandarización son clave.
- **3 Producción bajo demanda (pull):** En lugar de producir en base a pronósticos, se fabrica solo cuando hay una demanda real, reduciendo inventarios y sobreproducción. Sistemas como Kanban ayudan a visualizar y controlar este flujo.
- **4 Mejora continua (Kaizen):** La perfección no es un destino, sino un proceso constante. Se fomenta la participación de todos los colaboradores en la identificación de oportunidades de mejora, por pequeñas que sean.

Los 7 desperdicios (Muda) en LEAN

- **Sobreproducción:** producir más de lo necesario o antes de que se requiera. Esto genera excedentes que ocupan espacio, aumentan costos de almacenamiento y pueden volverse obsoletos. Es considerado el peor desperdicio porque ocasiona los demás.
- **Esperas:** tiempos muertos por descoordinación entre procesos, falta de materiales o espera de aprobaciones. Esto incluye tanto a personas inactivas como máquinas paradas.
- **Transporte innecesario:** movimiento excesivo de materiales o productos entre áreas, que no agrega valor y puede generar daños, pérdidas o retrasos. Ejemplo: almacenamiento en lugares distantes del punto de uso.
- **Exceso de inventario:** acumular materias primas, productos en proceso o terminados más allá de lo estrictamente necesario. Esto incrementa costos de almacenamiento, riesgos de obsolescencia y oculta problemas de calidad o planificación.
- **Movimientos redundantes:** acciones innecesarias de operarios, como desplazamientos excesivos, búsqueda de herramientas o posturas incómodas. Esto reduce la eficiencia y puede causar fatiga o lesiones.
- **Defectos:** errores que generan retrabajo, desperdicio de materiales o productos no conformes. Además del costo directo, afectan la reputación y la confianza del cliente.
- **Sobreprocesamiento:** añadir pasos, características o recursos que el cliente no valora. Ejemplos: inspecciones redundantes, acabados innecesarios o uso de materiales de mayor calidad que la requerida.



Herramientas clave en LEAN

El enfoque LEAN se apoya en diversas herramientas prácticas diseñadas para optimizar procesos y eliminar desperdicios. La metodología **5S** proporciona un marco sistemático para organizar los espacios de trabajo, basado en cinco pasos fundamentales: **clasificación, orden, limpieza, estandarización y mantenimiento de las mejoras**.

Para la gestión visual del flujo de trabajo, Kanban emerge como una solución efectiva, permitiendo controlar la producción mediante señales visuales que evitan la sobreproducción y equilibran la carga de trabajo. En el ámbito de la flexibilidad operativa, **Single-Minute Exchange of Dies (SMED)** se enfoca en minimizar los tiempos de transición entre diferentes producciones, agilizando así los cambios de línea.

El **Value Stream Mapping (VSM)** ofrece una perspectiva global del flujo de valor, identificando tanto las actividades que aportan valor como los desperdicios presentes en el proceso. Complementariamente, los KPIs (Indicadores Clave de Desempeño) como el tiempo de ciclo, la eficiencia global de los equipos (OEE) y los índices de defectos, proporcionan mediciones objetivas para evaluar y mejorar el rendimiento.

Como marco de mejora continua, el ciclo **Planificar-Hacer-Verificar-Actuar (PDCA)** establece un enfoque iterativo para la resolución de problemas y la implementación de mejoras sostenibles.

Integración y beneficios

La naturaleza flexible de LEAN permite su integración sinérgica con otras metodologías como Six Sigma, que aporta herramientas estadísticas para reducir la variabilidad, o Agile, particularmente útil en entornos de desarrollo de software. La implementación exitosa requiere más que la aplicación de técnicas; exige un cambio cultural donde la mejora continua se convierta en un compromiso compartido por todos los niveles organizacionales.

Entre los beneficios tangibles se destacan la reducción significativa de costos y tiempos de producción, el incremento en la calidad entregada al cliente, la optimización del uso de espacios y recursos, así como el fortalecimiento del compromiso de los equipos y la agilidad de los procesos.

En su esencia, LEAN trasciende el concepto de simple metodología operativa para convertirse en una filosofía gerencial orientada a maximizar el valor entregado al cliente mientras se minimiza el uso de recursos. Esta dualidad entre eficiencia y creación de valor constituye el núcleo de su propuesta transformadora.

4.2.3.3. Culturas técnicas alineadas

4.2.3.3.1. DevOps

DevOps es una **metodología de desarrollo** de software que integra Desarrollo y Operaciones para acelerar la entrega de aplicaciones, mejorar su calidad y fomentar la colaboración entre equipos. Surgió como respuesta a los problemas de los modelos tradicionales, donde los desarrolladores y los administradores de sistemas trabajaban en silos, generando cuellos de botella, lentitud en los despliegues y falta de comunicación. DevOps promueve una cultura de colaboración, automatización y mejora continua, eliminando barreras entre equipos y optimizando el ciclo de vida del software.



El enfoque DevOps sigue un **flujo continuo e iterativo** que incluye varias etapas clave. La primera es la **planificación**, donde se definen requisitos, priorizan tareas y se alinean los equipos, usando herramientas como Jira o Trello. Luego viene la **codificación**, donde se desarrolla el software con control de versiones en repositorios compartidos usando Git o GitHub.

La **construcción** automatiza la compilación y empaquetado del código con herramientas como **Jenkins o GitHub Actions**. Las pruebas ejecutan verificaciones automatizadas con **Selenium o JUnit**. El despliegue implementa los cambios en diferentes entornos usando **Docker, Kubernetes o Terraform**. La operación gestiona la infraestructura y el escalado en producción con Kubernetes o servicios en la nube. Finalmente, el monitoreo supervisa el rendimiento y disponibilidad con Prometheus, Grafana o ELK Stack.

Los principios fundamentales de DevOps incluyen la **automatización** para eliminar tareas manuales repetitivas, la infraestructura como código para gestionar servidores y entornos de forma programática, y la integración continua y despliegue continuo para entregas rápidas y confiables. También se enfoca en la **colaboración** entre equipos multidisciplinarios y la **integración de seguridad** desde el inicio, conocido como **DevSecOps**, usando herramientas como SonarQube o OWASP ZAP.

Comparado con modelos tradicionales como el cascada, DevOps es más rápido, pasando de ciclos de meses o años a horas o días. Mientras que los modelos antiguos son manuales y operan en silos, DevOps automatiza procesos y fomenta equipos unificados. Además, en lugar de feedback escaso después del lanzamiento, **DevOps ofrece retroalimentación constante mediante monitoreo en tiempo real**. Su flexibilidad permite adaptarse frente a enfoques rígidos y secuenciales.

Los beneficios de DevOps incluyen **mayor velocidad de entrega** con lanzamientos frecuentes y confiables, **mejor calidad del software** gracias a pruebas automatizadas y detección temprana de errores, **reducción de costos** al minimizar fallos en producción y optimizar recursos, escalabilidad eficiente con gestión de infraestructura en la nube, y **seguridad mejorada** mediante prácticas integradas desde el diseño.

Sin embargo, DevOps también presenta retos. Requiere un cambio cultural para romper silos y fomentar colaboración, tiene una curva de aprendizaje por la necesidad de dominar herramientas complejas como Kubernetes o Terraform, y exige integrar seguridad y cumplimiento desde el inicio en lugar de tratarlos como una idea tardía.

Herramientas como las siguientes son pilares de su implementación:

- **Git:** sistema de control de versiones distribuido que permite rastrear cambios en el código fuente durante el desarrollo de software. Facilita la colaboración entre equipos mediante ramas, fusiones y repositorios remotos.
- **Jenkins:** herramienta de automatización de código abierto para CI/CD. Permite automatizar la construcción, prueba y despliegue de aplicaciones mediante pipelines definidos en scripts o interfaces gráficas.
- **GitHub Actions:** plataforma de automatización integrada en GitHub que permite crear flujos de trabajo (workflows) para CI/CD directamente en repositorios. Ejecuta tareas como pruebas, despliegues y notificaciones basadas en eventos de GitHub.



- **Docker:** plataforma de contenedorización que permite empaquetar aplicaciones y sus dependencias en contenedores ligeros y portables. Los contenedores se ejecutan de manera aislada en cualquier entorno con Docker Engine.
- **Kubernetes:** orquestador de contenedores open-source para automatizar el despliegue, escalado y gestión de aplicaciones en contenedores. Organiza contenedores en pods, maneja balanceo de carga, auto-reparación y actualizaciones sin downtime.
- **Terraform:** herramienta de infraestructura como código de HashiCorp que permite definir y provisionar recursos en la nube o locales usando archivos de configuración declarativos. Gestiona el ciclo de vida de la infraestructura de forma versionable.
- **Prometheus/Grafana:** prometheus: Sistema de monitoreo y alerta open-source que recopila métricas de aplicaciones y infraestructura en tiempo real, usando un modelo de almacenamiento basado en series temporales.
- **Grafana:** plataforma de visualización de datos que se integra con Prometheus (y otras fuentes) para crear dashboards interactivos con gráficos, alertas y análisis de métricas.

Plataformas en la nube también juegan un papel clave al proporcionar escalabilidad bajo demanda.

4.2.3.3.2. DevSecOps

DevSecOps representa un cambio de paradigma en el desarrollo de software, donde **la seguridad se integra de forma nativa** y continua en todo el ciclo de vida DevOps. Este enfoque transforma la seguridad de ser un obstáculo a convertirse en un facilitador estratégico, implementando controles de manera programable y automatizada que evolucionan junto con la infraestructura y aplicaciones.

En sectores como banca y cloud, DevSecOps permite implementar arquitecturas adaptativas con **controles de cumplimiento integrados y mecanismos de autodefensa nativos**. Su esencia radica en tres pilares clave:

- Automatización avanzada de controles que va más allá del simple escaneo.
- El principio "Shift Left" aplicado mediante prácticas como threat modeling colaborativo.
- Una monitorización proactiva que evoluciona hacia observabilidad de seguridad predictiva.

Este modelo exige una transformación cultural hacia la "propiedad compartida de seguridad", donde los equipos adoptan responsabilidad colectiva. Se implementa mediante plataformas internas que ofrecen seguridad como servicio autoservicio, permitiendo autonomía con guardrails automatizados. La verdadera innovación no está solo en prevenir vulnerabilidades, sino en crear sistemas que mejoren continuamente su postura de seguridad mediante aprendizaje automático y retroalimentación en tiempo real, todo ello manteniendo la agilidad y velocidad que caracterizan a DevOps.



Un pilar clave de DevSecOps es abordar los cinco riesgos críticos:

- **Código vulnerable:** son errores o debilidades en el código fuente de una aplicación que pueden ser explotados por atacantes para comprometer su seguridad.
- **Configuraciones erróneas:** ocurren cuando los sistemas, servicios o aplicaciones tienen ajustes de seguridad incorrectos, como permisos demasiado abiertos, servicios innecesarios activados o contraseñas predeterminadas, facilitando accesos no autorizados.
- **Secretos expuestos:** sucede cuando información sensible como contraseñas, claves API o tokens de acceso quedan visibles en repositorios de código, logs o mensajes de error, permitiendo que actores maliciosos los aprovechen.
- **Falta de gobernanza:** es la carencia de políticas, controles y procesos definidos para gestionar la seguridad de manera organizada, lo que lleva a riesgos no identificados, cumplimiento deficiente y respuestas lentas ante incidentes.

Herramientas para combatir los riesgos:

- **SonarQube:** es una herramienta de análisis estático de código abierto que examina el código fuente para detectar errores, vulnerabilidades de seguridad, malas prácticas y falta de cobertura de pruebas. Soporta múltiples lenguajes como Java, Python y JavaScript, y se integra en pipelines CI/CD para evaluar automáticamente la calidad del código. Genera informes detallados y métricas que ayudan a los desarrolladores a mejorar su trabajo, bloqueando cambios problemáticos antes de que se fusionen. Es ideal para mantener estándares de código limpio y mantenible durante el desarrollo continuo.
- **Checkmarx** es una solución comercial especializada en seguridad de aplicaciones, enfocada en identificar vulnerabilidades avanzadas en el código y sus dependencias. Mediante análisis SAST y SCA, detecta riesgos como los del OWASP Top 10 o fallos en bibliotecas externas (ej: log4j). A diferencia de SonarQube, profundiza en amenazas complejas con menos falsos positivos, usando IA, y es clave para cumplir normativas como PCI-DSS. Se integra en pipelines DevSecOps para auditorías automáticas, siendo usada por equipos de seguridad para prevenir despliegues inseguros.
- **OWASP ZAP** es una herramienta de seguridad de código abierto mantenida por la fundación OWASP, diseñada para identificar vulnerabilidades en aplicaciones web mediante pruebas de penetración activas y pasivas. Funciona como un proxy intermediario que intercepta y analiza el tráfico entre el navegador y la aplicación, detectando vulnerabilidades comunes como inyecciones SQL, XSS, CSRF y fallos de autenticación. A diferencia de SonarQube y Checkmarx, ZAP se enfoca en pruebas dinámicas, simulando ataques reales contra aplicaciones en ejecución. Es altamente configurable, se integra con pipelines CI/CD, y es ideal para desarrolladores y pentesters que buscan una solución gratuita para mejorar la seguridad de sus aplicaciones web.
- **Snyk** es una plataforma de seguridad especializada en identificar y corregir vulnerabilidades en dependencias de código abierto y en la configuración de infraestructura como código. A diferencia de herramientas como SonarQube o Checkmarx, Snyk se enfoca específicamente en analizar las librerías y paquetes de terceros que utiliza un proyecto, detectando vulnerabilidades conocidas en dependencias directas y transitivas. Además, ofrece capacidades para escanear imágenes de Docker y configuraciones de Kubernetes o Terraform, buscando configuraciones inseguras integradas directamente en los pipelines CI/CD.



A diferencia de modelos tradicionales donde la seguridad se aplica al final, DevSecOps exige colaboración entre desarrolladores, operaciones y equipos de seguridad, con formación continua en estándares como OWASP Top 10. Su implementación requiere compromiso organizacional y automatización robusta, pero ofrece beneficios como reducción de brechas de seguridad, cumplimiento normativo más ágil y mayor confianza del cliente.

4.2.3.4. Beneficios

- **Flexibilidad a cambios.**

Gran capacidad de reacción ante los cambiantes requerimientos generados por las necesidades del cliente o la evolución del mercado.

El marco de trabajo está diseñado para adecuarse a las nuevas exigencias que implican proyectos complejos.

- **Reducción del Time to Market.**

El cliente puede empezar a utilizar las características más importantes del proyecto antes de que esté completamente terminado.

- **Mayor calidad del software.**

El trabajo metódico y la necesidad de obtener una versión de trabajo funcional después de cada iteración, ayuda a la obtención de un software de alta calidad.

- **Mayor productividad.**

Se logra, entre otras razones, debido a la eliminación de la burocracia y la motivación del equipo proporcionado por el hecho de que pueden estructurarse de manera autónoma.

- **Maximiza el retorno de la inversión (ROI).**

Creación de software solamente con las prestaciones que contribuyen a un mayor valor de negocio gracias a la priorización por retorno de inversión.

- **Predicciones de tiempos.**

A través de este marco de trabajo se conoce la velocidad media del equipo por sprint, con lo que es posible estimar de manera fácil cuando se podrá hacer uso de una determinada funcionalidad que todavía está en el Backlog.

- **Reducción de riesgos.**

El hecho de desarrollar, en primer lugar, las funcionalidades de mayor valor y de saber la velocidad a la que el equipo avanza en el proyecto, permite despejar riesgos efectivamente de manera anticipada.



Comparativa

Diferencia entre Agile y Scrum.

Scrum es un marco específico de desarrollo de proyectos y software., mientras que, Agile es una metodología de desarrollo que engloba el marco, pero que va más allá.

4.3. Capability Maturity Model Integration (CMMI)

(Integración de sistemas modelos de madurez de capacidades).

Es un modelo para la mejora y evaluación de procesos para el desarrollo, mantenimiento y operación de sistemas de software.

Administrado por el Instituto CMMI, una subsidiaria de ISACA (acrónimo de Information Systems Audit and Control Association, una asociación internacional que apoya y patrocina el desarrollo de metodologías y certificaciones para la realización de actividades de auditoría y control en sistemas de información) se desarrolló en la Universidad Carnegie Mellon (CMU).

Es requerido por muchos contratos del Departamento de Defensa de los Estados Unidos (DoD) y del Gobierno de los Estados Unidos, especialmente en el desarrollo de software.

CMU pretende que CMMI pueda ser usado para guiar la mejora de procesos en un proyecto, división o una organización completa.

CMMI define los siguientes niveles de madurez para los procesos:

- Inicial.
- Repetible.
- Definido.
- Gestionado.
- Optimizado.

CMMI está registrada en la Oficina de Patentes y Marcas de Estados Unidos por CMU.



La versión 2.0 se publicó en 2018 y en el año 2010 la versión 1.3, que tiene 3 divisiones disponibles:

- **CMMI para el Desarrollo (CMMI-DEV o CMMI for Development).**

En él se trata la mejora de los procesos para el desarrollo de mejores productos y servicios.

- **CMMI para la adquisición (CMMI-ACQ o CMMI for Acquisition).**

En él se tratan la gestión de la cadena de suministro, adquisición y contratación externa en los procesos del gobierno y la industria.

- **CMMI para servicios (CMMI-SVC o CMMI for Services).**

Está diseñado para cubrir todas las actividades que requieren gestionar, establecer y entregar Servicios.

Evaluación (appraisal)

Muchas organizaciones valoran el medir su progreso llevando a cabo una evaluación (appraisal) y ganando una clasificación del nivel de madurez o de un nivel de capacidad de logro. Este tipo de evaluaciones son realizadas normalmente por una o más de las siguientes razones:

- Para determinar que tan bien los procesos de la organización se comparan con las mejores prácticas CMMI y determinar qué mejoras se pueden hacer.
- Como requisito del cliente en licitaciones públicas o concursos privados.

Las valoraciones de las organizaciones utilizando un modelo CMMI deben ajustarse a los requisitos definidos en el documento "Appraisal Requirements for CMMI" (ARC).

La evaluación se enfoca en identificar oportunidades de mejora, y comparar los procesos de la organización con las mejores prácticas CMMI.

Los equipos de evaluación usan el modelo CMMI y un método conforme a ARC para guiar su evaluación y reporte de conclusiones, y los resultados de la evaluación son usados para planear mejoras en la organización.

Hay tres clases de evaluación: Clase A, B, C.

El Standard CMMI Appraisal Method for Process Improvement (SCAMPI) es un Método de evaluación que cumple todos los requerimientos ARC. Una evaluación de clase A es más formal y es la única que puede resultar en una clasificación de nivel.

El Standard CMMI Appraisal Method for Process Improvement (SCAMPI) es el método oficial SEI para proveer puntos de referencia de sistemas de calificación en relación con los modelos CMMI. SCAMPI se usa para identificar fortalezas y debilidades de los procesos, revelar riesgos de desarrollo/adquisición, y determinar niveles de capacidad y madurez. Se utilizan ya sea como parte de un proceso o programa de mejoramiento, o para la calificación de posibles proveedores.



El método define el proceso de evaluación constando de preparación; las actividades sobre el terreno; observaciones preliminares, conclusiones y valoraciones; presentación de informes y actividades de seguimiento.

5. Pruebas. SCM (Gestión de Configuración de Software)

SCM son las siglas de Software Configuration Management. SCM, es una especialización de la gestión de configuración a todas las actividades en el sector del desarrollo de *software*.

SCM trata y controla:

- La elaboración de código fuente por varios desarrolladores simultáneamente.
- El seguimiento del estado de las fases del desarrollo de software (versiones) y sus cambios (control de versiones).
- La conducción de la integración de las partes del software en un solo producto de software.

Para la realización de la SCM hay diferentes herramientas. Pero herramientas que pretenden ofrecer una solución total al problema, a menudo no cumplen con los requisitos técnicos como:

- Apoyo a diferentes plataformas.
- Iniciar el proceso de build.
- Conexión a los bancos de datos existentes.
- Integración a la organización existente.

Por esa razón ofrece una mayor flexibilidad una solución que integre herramientas parciales que sean más fáciles de integrar en el proceso existente.

Por ejemplo:

- Uso de un software de administración de versiones como IBM Rational Team Concert, CVS, Subversion, SourceSafe, ClearCase, Darcs, Plastic SCM.
- Introducción de una herramienta para la documentación comunitaria con una administración de cambios, acceso interactivo y foro o alguna plataforma para la comunicación.
- Determinar un entorno para el build automático.



Algunos ejemplos de sistemas de SCM son:

- AccuRev.
- Rational ClearCase.
- Fossil.
- Git.
- Harvest (CA).
- Perforce.
- Plastic SCM.
- Rational Synergy (Telelogic Synergy/CM).
- Sablime.
- SET-LIBER.
- Smart Bear.
- SpectrumSCM.
- Surround SCM.
- Subversion.
- Trac.
- Visual Source Safe.
- Microsoft Visual Studio 2010 ALM.
- Microsoft Team Foundation Server 2010.

"Plataforma de control de versiones de Microsoft".

5.1. Desarrollo Orientado a Pruebas (TDD)

TDD (en inglés Test-Driven Development) es una metodología de desarrollo de software que integra las pruebas dentro del propio ciclo de desarrollo. En TDD, los desarrolladores escriben primero las pruebas automatizadas antes de desarrollar el código funcional. Esto asegura que cada nueva funcionalidad esté probada y validada desde el inicio del proceso de desarrollo.



La metodología TDD tiene una relación importante con la Gestión de Configuración de Software (SCM), ya que proporciona una base sólida para:

- Controlar la calidad del código: A través de la ejecución continua de pruebas, se asegura que cada versión o cambio en el código cumpla con los requisitos establecidos.
- Facilitar la integración continua: El uso de TDD permite realizar una integración más segura en sistemas de control de versiones como Git, Subversion o Microsoft Team Foundation Server, ya que se puede verificar automáticamente que cada cambio pase todas las pruebas antes de ser aceptado en la rama principal del proyecto.
- Automatización del proceso de build: En conjunto con herramientas de SCM, TDD se integra perfectamente en entornos de Integración Continua (CI) y Entrega Continua (CD), donde las pruebas escritas impulsan la generación de builds automáticos y la validación constante del código en diversas fases del ciclo de vida del software.

Las herramientas de SCM como Git, Subversion y Microsoft Team Foundation Server (TFS), combinadas con TDD, permiten:

- Versionado del código: Mantener un historial detallado de cada cambio en el código, junto con sus pruebas asociadas.
- Ejecución automatizada de pruebas: Cada vez que se realiza un commit o push, las pruebas se ejecutan automáticamente para asegurar la estabilidad del software.

Beneficios de TDD dentro de SCM:

Existe un menor riesgo de errores en producción pues las pruebas están integradas en cada fase, reduciendo significativamente los errores en las versiones finales.

El código escrito bajo TDD suele ser más modular, es más mantenible y flexible ya que debe ajustarse a las pruebas específicas facilitando modificación y mantenimiento.

Los desarrolladores pueden colaborar eficientemente, porque cada cambio está respaldado por pruebas y gestionado de manera controlada dentro de un sistema SCM.

6. Monitorización de infraestructuras informáticas

El sistema de monitorización o subsistema de monitorización es el encargado de hacer un seguimiento del estado del sistema completo, para asegurar la fiabilidad y estabilidad de los servicios que provee el conjunto.

Evalúa la salud y el rendimiento del sistema completo, tanto de la infraestructura como del resto de subsistemas. Para ello se basa en la recogida de métricas, procesamiento y visualización de los datos, y también con la generación de alertas cuando sucede algo que puede ser un síntoma de un riesgo o mal funcionamiento.



La base del sistema de monitorización es la recopilación de valores, qué al analizarlos, permiten entender el comportamiento, las tendencias, los riesgos y poder prever el impacto que tendrán posibles futuros cambios.

6.1. Métrica

Se llama métrica a aquellas características del sistema, que, al ser medidas, proporcionan una secuencia de valores registrados con su sello de tiempo. Por ejemplo, en un servidor web, el conjunto total de peticiones recibidas es una métrica.

En un mundo ideal deberíamos controlar con una métrica todo aquello que pudiera en un momento ser relevante. Sin embargo, puede que esto no sea posible o incluso deseable por algunos/s de los siguientes motivos.

Lo ideal sería poder controlar con un tipo de métrica todo lo relevante, pero esto no es posible por los siguientes motivos:

- Recursos disponibles para realizar el seguimiento (personas, infraestructura, presupuesto).
- La complejidad y propósito de la aplicación.
- Entorno de despliegue.

Es más interesante tener una monitorización robusta en los entornos de producción que en los de desarrollo o testing. Por esta razón suele haber diferencias en la severidad, granularidad y cantidad de diferentes métricas medidas.

- La probabilidad de que la métrica sea útil.

Cada métrica adicional incrementa la complejidad y gasta recursos.

- La importancia de la estabilidad del sistema.

Por estos motivos, se asocian distintas etiquetas a las métricas para permitir definir un modelo de datos con distintas dimensiones.

Ejemplo para un servidor web:

- Para la métrica del conjunto total de peticiones recibidas podemos definir etiquetas para identificar que método HTTP se usa.
- Por otro lado, definir etiquetas para identificar el directorio o subdirectorío sobre el que se realiza la petición. Con esta información, podríamos obtener datos sobre las peticiones HTTP que usan el método POST.



Tipos de información usada en las métricas

Las métricas sobre las que se recogen los valores son distintas para cada sistema, por ello distinguimos entre distintos niveles al planificar la estrategia de monitorización:

- **Métricas basadas en las máquinas.**

En lo más bajo de la jerarquía de métricas están los indicadores de las máquinas. Aquí se incluyen cualquier métrica involucrada en evaluar el estado o rendimiento de la máquina individual ignorando por el momento su pila de aplicación y servicios. Ejemplo Uso de CPU, Memoria, Espacio de disco, uso de memoria de intercambio.

- **Métricas de aplicación.**

Determinan si una aplicación está funcionando correctamente y con eficiencia, son indicadores de la salud, rendimiento o carga de una aplicación. Ejemplos: Tasas de error y éxito, fallos de servicio y reinicios, rendimiento y latencia de las respuestas, uso de recursos.

- **Métricas de conectividad y red.**

Son las relacionadas con indicadores de red y conectividad, muy importantes para evaluar la conectividad desde el exterior y para asegurar que los servicios son accesibles a otras máquinas del sistema. Ejemplos: conectividad, tasa de error y paquetes perdidos, latencia, utilización del ancho de banda.

- **Métricas de pool de servidores.**

Las métricas sobre servidores individuales son importantes. Sin embargo, en sistemas grandes es mejor evaluar la habilidad de una colección de máquinas para desarrollar un trabajo y responder adecuadamente a las peticiones. Ejemplos: uso de los recursos del pool, indicadores de ajuste de escala, instancias degradadas.

- **Métricas de dependencias externas.**

Es frecuente que los servicios provean páginas de estado o APIs para detectar problemas. El seguimiento de esta información desde nuestro sistema, junto con el registro de las interacciones con el servicio, puede ayudar a identificar problemas con los proveedores que puede afectar a las operaciones. Ejemplos: Estado del servicio y disponibilidad, tasas de error y éxito, agotamiento de recursos, tasa de ejecución y costes operacionales.

Tipos de comportamiento de métricas

Podemos distinguir distintos tipos de métricas, dependiendo de sus características:

- **Contador.**

Es una métrica acumulativa que representa un valor numérico que solo puede subir. Por ejemplo, un contador de peticiones servidas, tareas completadas, errores ocurridos, etc.



- **Calibrador.**

Es una métrica que representa un valor numérico que puede arbitrariamente subir o bajar. Por ejemplo, una medida de la temperatura, una medida de la memoria usada o el número de procesos en ejecución.

- **Histograma.**

Muestra observaciones (normalmente cosas con duraciones de peticiones o tamaños de respuesta) y las cuenta en tipos base previamente configurados. Se suele proporcionar una suma de todos los valores observados y de cada uno de los tipos.

- **Resumen.**

De forma similar a los histogramas, muestra observaciones (normalmente cosas como duración de peticiones y tamaños de respuestas). Provee el número total de observaciones, una suma de todos los valores observados y calcula cuantiles configurables sobre una ventana de tiempo deslizante.

Los cuantiles, son aquellos valores de la variable, que, ordenados de menor a mayor, dividen a la distribución en partes, de tal manera que cada una de ellas contiene el mismo número de frecuencias.

6.1.1. Metodología métrica v3

Se concibe como una Metodología de Planificación, Desarrollo y Mantenimiento de Sistemas de Información.

Puede ser utilizada libremente con la única restricción de citar la fuente de su propiedad intelectual: el **Ministerio de Administraciones Públicas**.



Importante

La metodología MÉTRICA Versión 3 ofrece a las Organizaciones un instrumento útil para la sistematización de las actividades que dan soporte al ciclo de vida del software.

[Métrica v. 3](#)



Este Ministerio, ofrece a las Organizaciones, desde el Consejo Superior de Informática, un método para la sistematización de las actividades que dan soporte al ciclo de vida del software en el desarrollo de Sistemas de Información, y un marco de gestión para asegurar que los proyectos cumplen sus objetivos en términos de calidad, coste y plazos.



+ Info

En su página web está la documentación con una descripción completa y detallada de esta metodología.

Los contenidos de dicha página, la que publica el Ministerio de Administraciones Públicas, es un contenido libre que se puede utilizar con total libertad citando la fuente.

<http://www.map.es/portal/index.html;jsessionid=B072582A935A16749177D6FEA7DA9DF9>

https://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html

Métrica v. 3. ofrece a las Organizaciones un instrumento útil para la sistematización de las actividades que dan soporte al ciclo de vida del software a través de varios documentos PDF.

Documentación extensa que el alumno deberá leer, pues preguntas de examen como la del año 22:

¿Qué participantes están presentes en la tarea de "Elaboración de los Manuales de Usuario"?

Son resueltas en estos mismos, en este caso en el documento "Construcción del Sistema de Información (Proceso CSI)", página 15.

6.1.1.1. Conceptos generales

Vamos a tener una serie de fases o procedimientos por los que vamos a pasar durante el desarrollo de la aplicación.

En cada una de estas fases vamos a hacer una serie de tareas que tenemos que hacer.

Para hacer cada tarea dispondremos de unas herramientas, y técnicas definidas.



Cada una de estas tareas normalmente van a generar un producto. Este producto lo utilizaremos para tareas siguientes y así vamos evolucionando en el desarrollo de la aplicación.

Hay muchos tipos de metodología e incluso en muchas ocasiones uno utiliza una metodología no definida concretamente si no qué va usando y aplicando las diferentes fases para desarrollar una aplicación de sistemas.

Las más cercanas a España, tenemos:

- Metodología **MERISE**. Francia que surge en 1977.
- Metodología **SSADM**. (Método Estructurado de Análisis y Diseño de Sistemas). Aparece en Gran Bretaña se establece como obligatoria para la Administración Pública a partir de 1983.
- Metodología **Métrica**. España.

No obstante, existen metodologías perfectamente definidas dónde se describen las diferentes fases tareas herramientas técnicas resultados de cada una de esas tareas que hay que realizar en el desarrollo de una aplicación.

Ejemplos de metodologías más actuales y de ámbito empresarial más que de instituciones públicas, tendríamos:

- Metodología de Prototipo.
- Desarrollo Rápido de Aplicaciones (RAD).
- Metodología de Programación Extrema (XP).

La metodología de España a nivel de administración pública que se conoce como **Métrica**. La metodología **Métrica** es una metodología que ha ido evolucionando a lo largo del tiempo actualmente la versión con la que se trabaja es la versión 3.

Existen unos estándares de diferentes asociaciones organizaciones. Estas organizaciones corresponden a organizaciones no gubernamentales ONG's, sin ánimo de lucro que establecen estándares son la base para los diferentes productos, que van saliendo, ejemplos de este tipo de organizaciones sería ISO, IEC, IEEE, OMG entre otras.

Métrica para su creación está basada en estos estándares. los principales que podríamos enumerar serían:

- **ISO 12207 Information technology -Software life cycle processes.**

Esta norma propone un Modelo de Ciclo de Vida de Desarrollo, el cual se ha seguido en la elaboración de la estructura de Métrica versión 3.

- **ISO/IEC TR 15.504 (SPICE)** Software Process Improvement and assurance standards Capability Determination.



- **ISO 9000-3 Quality Management and Quality".** Part 3: Guidelines for the application of ISO 9001 – "Model for Quality Assurance in Design/Development, Production, Installation and Servicing.
- **IEEE Standard Glossary of Software Engineering Terminology.** Std. 610.12-1998.
- **IEEE Std. 1074-1998:** Software life-cycle processes.
- **OMG standard UML.**

La metodología métrica facilita interfaces de realización de los procesos tanto estructurados como orientado a objetos para ello utiliza:

- **Procesos principales.**
- **Interfaces** (Las interfaces son tareas comunes a todos los procesos).

Procesos principales

Los procesos principales corresponden a las fases del proceso de desarrollo y dentro de cada una de ellas vamos a tener una serie de actividades o subprocesos que van a establecer una determinada documentación y constituir hitos (momentos concretos con parte del producto terminados).

En cada proceso detalla las Actividades y Tareas a realizar.

Para cada tarea se indican:

- Las técnicas y prácticas a utilizar.
- Los responsables de realizarla.
- Sus productos de entrada y salida.

Estructura de procesos:

- Planificación del Sistema de Información **PSI**.
- Desarrollo:
 - Estudio de viabilidad **EVS**.
 - Análisis **ASÍ**.
 - Diseño **DSI**.
 - Construcción **CSI**.
 - Implantación y aceptación **IAS**.
- Mantenimiento **MSI**.



Interfaces

- Aseguramiento de la Calidad.
- Seguridad.
- Gestión de Configuración.
- Gestión de Proyectos.

6.2. Monitorización

Las métricas obtenidas desde varias partes del sistema son recopiladas dentro del sistema de monitorización el cual es responsable de:

- Almacenamiento tanto de los valores actuales con de los datos históricos. Debido a la naturaleza de los datos que gestionan usan bases de datos de series históricas que son un tipo de bases de datos no relacionales.
- Análisis de la información para realizar muestreos e información agregada sobre las métricas (Frecuencias de suceso, tiempo medio).
- Visualización para un mejor entendimiento e interacción (listas de valores, tablas, gráficos, panel de control).
- Organización para correlación de entradas. Por ejemplo, para descubrir si un evento está relacionado con que cierta métrica tenga valores muy altos.
- Inicio de respuestas automatizadas (alertas) cuando los valores cumplen ciertas condiciones. Cuando los valores obtenidos en las métricas caen fuera de los rangos esperados, los sistemas de monitorización envían notificaciones para avisar a un operador para que revisen la situación. El sistema de monitorización asistirá a dicho operador, haciendo disponible la información que gestiona, para que pueda identificar la(s) causa(s). La definición de las alertas tiene dos componentes: una condición basada en las métricas y una acción a desarrollar cuando los valores caen fuera de condiciones aceptables.

La notificación de alerta debería contener suficiente información para diagnosticar que es lo que está pasando. Dependiendo de la importancia de las alertas se puede usar un sistema distinto de notificación (correo electrónico, llamadas, etcétera). Las alertas permiten a los operadores no estar tan pendiente de la monitorización del sistema.

Algunas de las mejores herramientas para la monitorización son:

- Acronis Monitoring Service.
- New Relic.
- LogicMonitor.



- Nagios.
- Icinga.
- Sensu.
- Zabbix.
- Paessler.
- SolarWinds.
- ManageEngine.

7. Programas para Control de Versiones

Se le da el nombre de SVC (System Version Control), en castellano SCV (Sistema de Control de Versiones).

Se llama control de versiones a la gestión de los diversos cambios que se realizan sobre los elementos de algún producto o una configuración del mismo. Una versión, revisión o edición de un producto, es el estado en el que se encuentra el mismo, en un momento dado de su desarrollo o modificación.

Un sistema de control de versiones es una herramienta que registra los cambios realizados sobre un archivo o conjunto de archivos de un proyecto a lo largo del tiempo.

El sistema de control de versiones es de gran utilidad para entornos de desarrollo colaborativo donde varios diseñadores, maquetadores y/o programadores trabajan sobre un mismo proyecto.

Los cambios que se realizan sobre cada elemento pueden verlos todos los integrantes.

7.1. Características

Un SVC posee tres capacidades principales:

- **Reversibilidad:** posibilidad de volver a un estado anterior del proyecto en caso de fallos.
- **Concurrencia:** puede haber varias personas modificando el mismo elemento al mismo tiempo.
- **Anotación:** se debe permitir adjuntar información relevante a los cambios realizados.

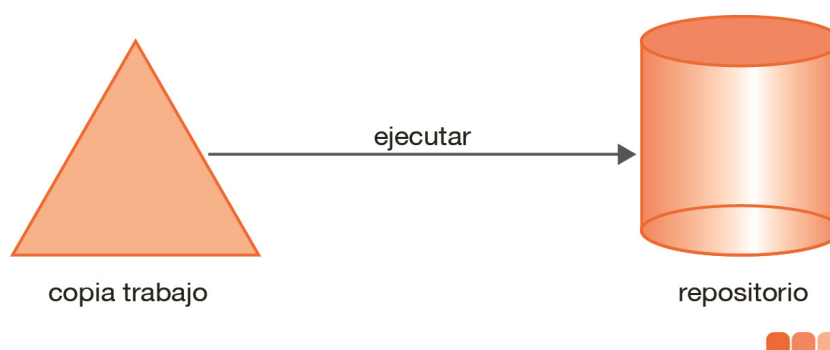


Un sistema de control de versiones debe proporcionar:

- **Mecanismo de almacenamiento:** debe permitir almacenar los elementos que deba gestionar (documentos, textos, código, esquemas, etcétera).
- **Posibilidad de realizar cambios:** debe permitir realizar cambios sobre los datos almacenados (modificación, añadir, borrar, renombrar, mover elementos, etcétera).
- **Registro histórico:** debe mantener un registro de las acciones realizadas sobre cada elemento. Permitirá volver a un estado anterior del elemento.

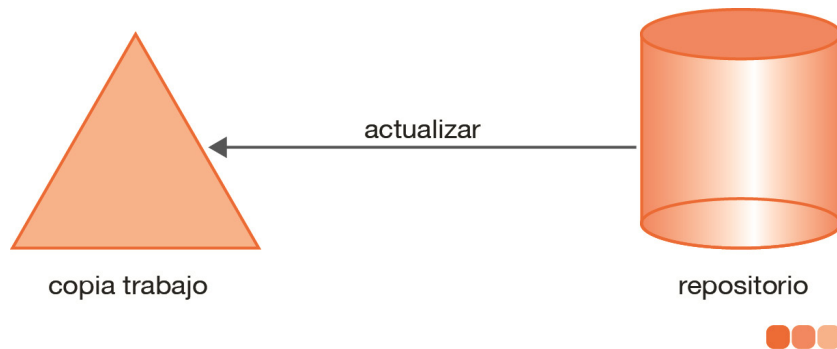
Conceptos básicos (Repaso)

- **Repositorio:** lugar en el que se almacenan los datos actualizados e históricos de cambios (sistema de archivos en un disco duro, un banco de datos, etcétera).
- **Revisión:** versión determinada de la información que se gestiona.
- **Tags:** permiten identificar de forma fácil revisiones importantes en el proyecto.
- **Módulo:** conjunto de directorios y/o archivos dentro del repositorio que pertenecen a un proyecto común.
- **Branch:** es una copia del proyecto aislada, de forma que los cambios realizados no afecten al resto del proyecto y viceversa, excepto cuando los cambios sean *unidos* de un lado al otro.
- **Línea base (baseline):** una revisión aprobada de un documento o fichero fuente, a partir del cual se pueden realizar cambios subsiguientes.
- **Checkout:** crea una copia de trabajo local desde el repositorio.
- **Merge:** une dos grupos de cambios en un archivo (o grupo de archivos), generando una revisión unificada.
- **Conflicto:** sucede cuando dos o más personas intentan realizar diferentes cambios en la misma porción de código.
- **Commit:** consiste en realizar un cambio local en el proyecto y luego almacenar dicho cambio en el repositorio.





- **Change set:** conjunto de cambios realizados en un único *commit*.
- **Update:** integra los cambios que han sido realizados en el repositorio en la copia de trabajo local.



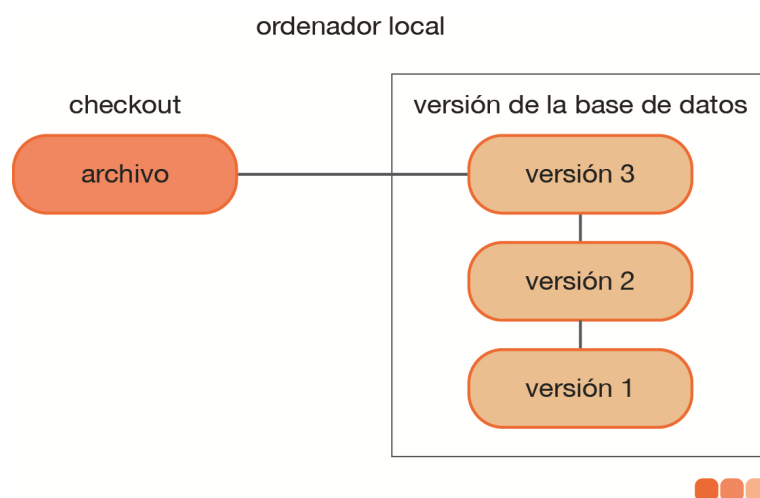
7.2. Clasificación

Podemos clasificar los sistemas de control de versiones según la arquitectura para almacenar la información en:

- Locales.
- Centralizados.
- Distribuidos.

7.2.1. Locales

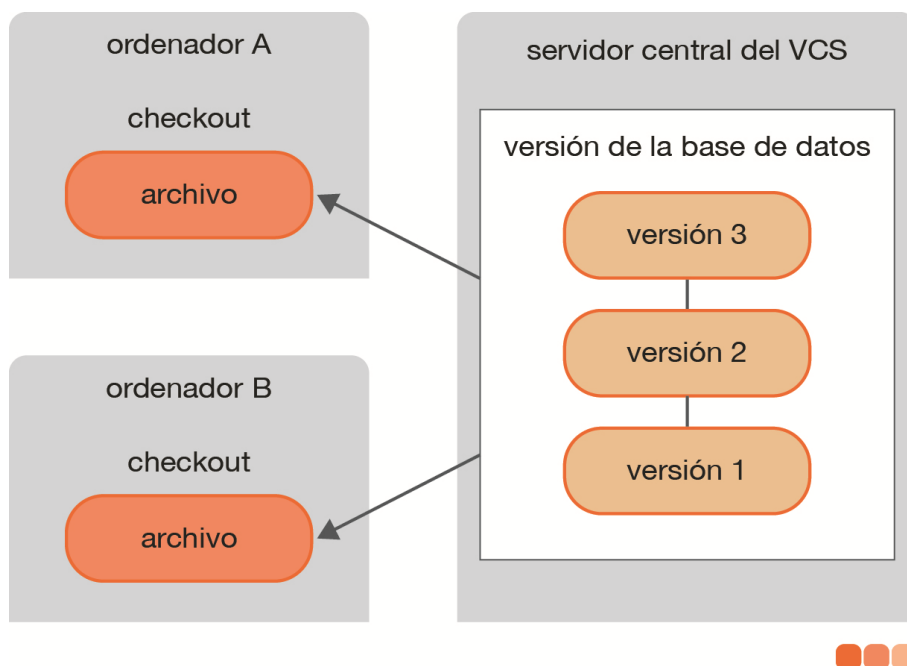
La información se guarda en un ordenador o repositorio local, con lo que no sirve para trabajar en forma colaborativa.





7.2.2. Centralizados

En los sistemas de control de versiones centralizados (*Centralized Version Control System* o CVCS) la información se guarda en un servidor dentro de un repositorio centralizado.



Existe un usuario o usuarios responsables con capacidad de realizar tareas administrativas.

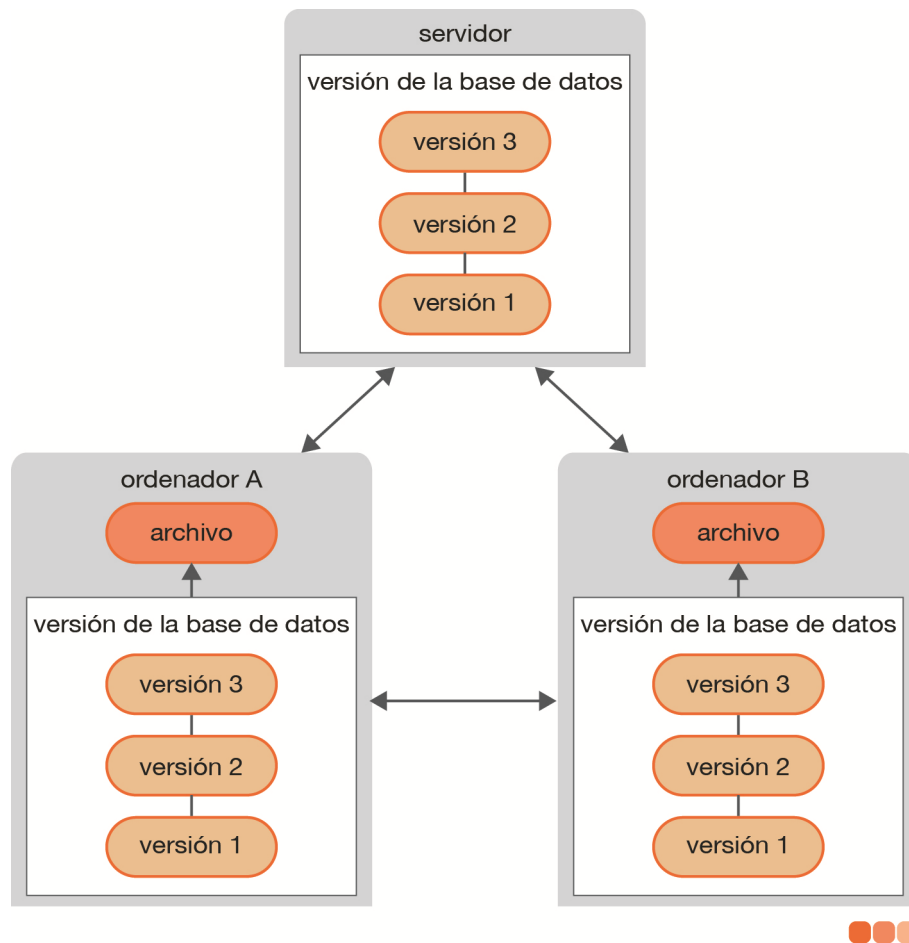
Se reduce la flexibilidad, ya que se necesita la aprobación del responsable para realizar acciones.

7.2.3. Distribuidos

Los sistemas de control de versiones distribuidos (*Distributed Version Control System* o DVCS) son aquellos en los que cada usuario tiene su propio repositorio.

Los distintos repositorios pueden intercambiar y mezclar revisiones entre ellos.

Es frecuente el uso de un repositorio, que está normalmente disponible, que sirve de punto de sincronización de los distintos repositorios locales.



7.3. Software para el Control de Versiones

Software RCS (Revision Control System)

Podemos clasificarlo en control local.

Revision Control System o RCS es una implementación en *software* del control de versiones que automatiza las tareas de guardar, recuperar, registrar, identificar y mezclar versiones de archivos. Es útil para archivos que son modificados frecuentemente. También puede ser utilizado para manejar archivos binarios, pero con eficacia y eficiencia reducidas. Las distintas versiones son archivadas mediante la ayuda de la herramienta *diff*.

No permite:

- Trabajar con proyectos enteros (solo con ficheros).
- Varios usuarios.

RCS es muy simple, por lo que se utiliza en *wikis* para guardar versiones de documentos.



Software Subversion



Fuente: https://commons.wikimedia.org/wiki/File:Subversion_logo.svg

Podemos clasificarlo como un software de control Centralizado.

Apache Subversion (SVN) es una herramienta de control de versiones *open source* basada en un repositorio cuyo funcionamiento se asemeja enormemente al de un sistema de ficheros. Es *software* libre bajo una licencia de tipo Apache/BSD. Utiliza el concepto de revisión para guardar los cambios producidos en el repositorio.

Entre dos revisiones guarda solo el conjunto de modificaciones, optimizando así al máximo el uso de espacio en disco. SVN permite al usuario crear, copiar y borrar carpetas con la misma flexibilidad con la que lo haría si estuviese en su disco duro local.

Subversion puede acceder al repositorio a través de redes, lo que le permite ser usado por personas que se encuentran en distintas computadoras.

Visual SourceSafe

También conocido por sus siglas VSS, es una herramienta de Control de versiones que forma parte de Microsoft Visual Studio, aunque está siendo sustituido por Visual Studio Team Foundation Server.

SourceSafe es un sistema basado en un equipo anfitrión a diferencia de la mayoría de los programas de control de versiones que son basados en Cliente-Servidor donde el repositorio de control de cambios reside en el equipo servidor y los clientes toman de allí la última versión para modificarla y posteriormente ingresarla con las modificaciones realizadas. Como en todos los programas de control de versiones, se basa en obtener una copia de trabajo ("check-out" o "desproteger"), realizar cambios sobre la copia y reingresarla al repositorio. Para lograr el acceso compartido al repositorio, VSS emplea el protocolo de archivos compartidos SMB (lo cual crea algunos inconvenientes).

SMB, Server Message Block, protocolo de red utilizado en el sistema operativo Windows, que permite compartir archivos, impresoras, etcétera, entre nodos de una red de computadoras que usan el sistema operativo Microsoft Windows.

- **Ventajas de Visual SourceSafe:**

Para las personas que desarrollan programas en el sistema operativo Windows, resulta una herramienta útil ya que se integra fuertemente con el entorno de desarrollo integrado o IDE de Visual Studio permitiendo un manejo relativamente simple de versiones sobre una computadora individual y en equipos de trabajo relativamente pequeños.



- **Desventajas de Visual SourceSafe:**

La principal desventaja de Visual SourceSafe reside en el método de acceso a los archivos compartidos que constituyen su repositorio mediante el protocolo SMB que no impide que éstos sean manipulados de manera externa al producto por cualquier persona que tenga acceso al mismo, provocando corrupción de datos. Este mismo tipo de acceso a archivos compartidos provoca que en equipos de trabajo grandes, el acceso concurrente pueda ser particularmente lento.

El SourceSafe es configurable, permitiendo que un solo programador modifique el código fuente (recomendado) o que lo hagan varios.

Las herramientas de gestión de diferencias para reunificar el código fuente modificado por varios programadores no son demasiado buenas comparadas con las de otros gestores de código fuente.

El SourceSafe es inestable cuando se suben ficheros binarios de gran tamaño, ya que espera solo ficheros de texto. Así que no es útil para almacenar documentación, sólo código fuente.

Mercurial



Fuente:

https://commons.wikimedia.org/wiki/File:New_Mercurial_logo.svg

Mercurial es un sistema de control de versiones multiplataforma. Está implementado principalmente en Python, pero incluye una implementación binaria de diff escrita en C. Mercurial es, sobre todo, un programa para la línea de comandos.

Las principales metas de desarrollo de Mercurial incluyen:

- Gran rendimiento.
- Escalabilidad.
- Desarrollo completamente distribuido (sin necesidad de servidor).



- Gestión robusta de archivos (de texto o binarios).
- Capacidades avanzadas de ramificación e integración.
- Sencillez conceptual.

Incluye una interfaz web integrada. Es *software* libre y el código fuente se encuentra disponible bajo los términos de la licencia GNU GPL versión 2.

GIT



Fuente:
<https://commons.wikimedia.org/wiki/File:Git-logo-orange.svg>



El experto opina

En nuestra opinión es el más importante. Se usa en la mayoría de grandes empresas y es el que nosotros aconsejamos usar para proyectos de cierta envergadura.

Git (pronunciado "guit") es un *software* de control de versiones diseñado por Linux Torvalds

Está orientado a la eficiencia y la confiabilidad del mantenimiento de versiones de aplicaciones cuando estas tienen un gran número de archivos de código fuente.

Su propósito es llevar registro de los cambios en archivos de computadora y coordinar el trabajo que varias personas realizan sobre archivos compartidos.

Git, se pensó como un motor de bajo nivel sobre el cual otros pudieran escribir la interfaz de usuario o front end, pero actualmente es un sistema de control de versiones con plena funcionalidad, por lo que es usado en algunos proyectos de mucha relevancia como el grupo de programación del núcleo Linux.

En cuanto a derechos de autor, Git es un software libre distribuible bajo los términos de la versión 2 de la Licencia Pública General de GNU.



Órdenes básicas

Vamos a describir las ordenes básicas de Git:

- **git init:**

Se utiliza para iniciar un nuevo repositorio de Git en un directorio existente o vacío. Se crea un subdirectorio oculto y nuevo llamado `.git`, el cual contiene todos los archivos necesarios del repositorio: un esqueleto de un repositorio de Git. Todavía no hay nada en el proyecto que esté bajo seguimiento. Git crea un nuevo repositorio en esa ubicación y comienza a rastrear los archivos en ese directorio.

- **git fetch:**

Descarga los cambios realizados en el repositorio remoto, sin realizar fusión alguna con las ramas locales.

- **git merge:**

Impacta en la rama en la que te encuentras situado, fusiona los cambios realizados en la rama "nombre_rama": `git merge "nombre_rama"`.

- **git pull:**

Unifica los comandos fetch y merge en un único comando.

- **git commit:**

Te da el control sobre qué cambios quieres incluir en tus commits, lo que te permite mantener un historial de cambios limpio y significativo en tu proyecto.

- **git commit -am "mensaje":**

Agrega los archivos modificados y eliminados al área de preparación y tras ello ejecuta un commit con el mensaje especificado.

- **git push <origin>:**

Sube los commits de la rama en la que nos encontramos al directorio remoto.

- **git status:**

Con este comando el sistema nos brindará información sobre los cambios pendientes en el directorio de trabajo, tras un commit debería de indicar que la rama está "limpia".

- **git add:**

Prepara los cambios realizados en los ficheros del proyecto para que puedan ser incluidos en el próximo commit.



- **git checkout -b** *<nombre_de_la_rama>*:

El comando se utiliza para crear una nueva rama y cambiar a ella en un solo paso.

- **git checkout -t** *<origin>*:

Establece una nueva rama local para rastrear automáticamente una rama remota en el repositorio remoto llamado "origin".

- **git branch**:

Lista de todas las ramas locales presentes en tu repositorio incluida la rama en la que te encuentras actualmente (que marca en esa lista con un asterisco *).

- **git branch -a**:

Lista todas las ramas locales y remotas.

- **git branch -d** *<nombre de la rama>*:

Elimina una rama local especificada en *<nombre_de_la_rama>*.

- **git remote prune** *<origin>*:

Elimina referencias locales a ramas remotas que ya no existen en el repositorio remoto *<origin>*.

- **git reset --hard HEAD**:

Eliminar todos los cambios locales en tu directorio de trabajo y en el área de preparación (staging area) y revertirlo al estado en el que se encuentra el commit más reciente en la rama actual.

- **git revert** *<hash_del_commit>*:

Se creará un nuevo commit que deshace los cambios introducidos por el commit especificado *<hash_del_commit>*.

Etiquetado

Git tiene la posibilidad de etiquetar puntos específicos del historial como importantes. Esta funcionalidad se usa típicamente para marcar versiones de lanzamiento (v1.0, por ejemplo).

- Listar las etiquetas disponibles en orden alfabético: **git tag**.

- Crear Etiquetas:

- Etiqueta ligera: **git tag -a** *<nombre_etiqueta>*

Creación de una etiqueta: **git tag -a** [v1.0]. Esta etiqueta puede ser utilizada para marcar puntos importantes en la historia de tu repositorio, como lanzamientos de versiones o hitos significativos.



- Etiqueta anotada: `git tag -a <nombre_etiqueta> -m "Mensaje de la etiqueta"`

Es la misma instrucción que la anterior, pero a ella se añade el `-m` que permitirá añadirle un mensaje, p.e: `git tag -a V2.0 -m "Nueva Versión 2.0"`.



+ Info

Existen más opciones dentro de GIT, te hemos mostrado las más destacadas.

Puedes consultar más en su página oficial ((Usa el traductor para verla en español)).

<https://git-scm.com>

Integrar cambios de una rama a otra

Git tiene dos utilidades que se especializan en integrar cambios de una rama a otra:

- **Git merge.**

Merge es siempre un registro de cambios de avance.

- **Rebase.**

git rebase <rama_objetivo>

Toma todos los commits de la rama actual que no están en <rama_objetivo>, deshace temporalmente esos commits, mueve la rama actual al punto en el que se bifurcó de <rama_objetivo>, aplica los commits uno por uno y finalmente mueve la rama actual al final de <rama_objetivo>.

git rebase -i <rama_objetivo>

Tiene potentes funciones de rescritura del historial.

Abrirá un editor de texto con una lista de commits donde se pueden cambiar el orden de los mismos, combinar varios en uno solo, editar mensajes de los commit o eliminarlos. Una vez realizados los cambios y guardados, el sistema aplicará los commits según lo especificado.



Rebase tiene 2 modos principales:

- Manual.
- Interactivo.

El archivo "gitignore"

Es una forma para indicar a Git, que archivos no queremos que rastree, como archivos de copia de seguridad creados por nuestro editor o archivos intermedios creados durante el análisis de datos.

En el archivo gitignore, se especifican todas las rutas y archivos que no se requiere comprobar, y de esta forma el proceso de control de versiones ignorará esos archivos.

Está disponible una herramienta online llamada *gitignore.io*.

En el cuadro de búsqueda, hay que poner los nombres de todas las herramientas, sistemas, frameworks, lenguajes, etc. que puedas estar usando, se selecciona todos los valores y al pulsar el botón Create, se genera el archivo automáticamente.



Fuente: <https://commons.wikimedia.org/wiki/File:Gitignore-io.svg>

8. Plataformas de desarrollo colaborativo de *software*

Una plataforma de desarrollo colaborativo de *software*, también conocida como forja, está enfocada a la cooperación entre desarrolladores para la difusión de *software* y el soporte al usuario.

En este tipo de plataformas se albergan múltiples proyectos de *software*, en los que los desarrolladores han de registrarse para poder contribuir.

Consta de numerosas aplicaciones normalmente con interfaz web para la administración y desarrollo de estos proyectos en común.

En las plataformas de desarrollo colaborativo es muy importante que los requisitos estén muy bien definidos, para poder crear un plan de desarrollo y determinar qué tareas realizará cada uno.

8.1. Principales plataformas

Existen muchas plataformas para el desarrollo colaborativo de *software*, vamos a ver una descripción de las más destacadas.



GitHub



Fuente: <https://www.flickr.com/photos/appleboy/13158675193>

GitHub es un gran repositorio (almacén) de código usado para desarrollar programas, *apps*, páginas web, servicios de Internet, etcétera, que trabaja sobre Git. Tiene más de 28 millones de usuarios registrados y almacena más de 75 millones de proyectos.

Usar GitHub es gratis, pero también tiene un servicio de suscripción con funciones adicionales, como obtener tu propio repositorio privado.

GitHub es también una plataforma de desarrollo colaborativo (forja), que sirve para crear proyectos en grupo. De este modo, alguien propone una duda, un problema o un proyecto, y otros desarrolladores se unen para intentar resolverlo.

(Una forja es una plataforma de desarrollo colaborativo de software.)

Miles de proyectos y aplicaciones que actualmente utilizan millones de personas, se ha creado gracias a la colaboración desinteresada en GitHub.

GitHub se **"había"** convertido en la plataforma de desarrolladores más grande del mundo y en uno de los principales bastiones del *software* libre.



El experto opina

Hemos puesto **"había"** en lugar de "ha" porque el 4 de junio de 2018 fue comprada por Microsoft, por 7.500 millones de dólares.

En el inicio de 2020 continúa siendo software libre.

Puedes ver más información en su web. (Usa el traductor para verla en español).

<https://github.com/>



GitLab



Fuente: https://es.m.wikipedia.org/wiki/Archivo:GitLab_logo.png

GitLab era la mejor alternativa a GitHub, por lo que la mayoría de los desarrolladores que se están marchando de GitHub han emigrado a GitLab.

Hay dos razones para ello:

- GitLab dispone de una herramienta de importación que permite traer tu código y tus relaciones desde GitHub de forma sencilla.
- Su estructura y funcionamiento es similar al de GitHub.

Características:

- Es algo más complejo de manejar, pero a cambio es más seguro y privado.
- Permite configurar con total libertad:
 - Grupos de usuarios.
 - Consultas.
 - Problemas.
 - Tareas.
- Permite mover tareas entre diferentes proyectos.
- Posee:
 - Un completo sistema de gestión del *branching*.
 - Notificaciones personalizadas.
 - Seguimiento de las diferentes versiones de un código.
 - Etcétera.



Repositorios: estructura y actualización. Generación de código y documentación. Metodologías de desarrollo. Pruebas. Programas para control de versiones. Plataformas de desarrollo colaborativo de software

- Puedes instalarlo en tu propio servidor, para usarlo con tus propios dominios.
- Tanto los repositorios públicos como los privados son gratuitos (aunque hay versiones de pago para empresas).

Compañías como IBM o Microsoft usan o han usado GitLab.

SourceForge



Fuente: <https://sourceforge.net/>

SourceForge es una web de proyectos colaborativos y desarrollo de *software*.



+ Info

La mayoría de los usuarios la conocen como una web para descargar aplicaciones gratuitas.

Características:

- Posee una herramienta para migrar a GitHub.
- Está enfocado a la colaboración a través de:
 - Foros.
 - Blogs.
 - Wikis.
 - Listas de correo.
 - Repositorio.



- Dispone además de:
 - Herramientas de rastreo de fallos.
 - Soporte técnico de usuarios.
 - Test de velocidad de Internet.
 - Almacenamiento en la nube.
- Trabaja con:
 - Git.
 - Mercurial.
 - Subversion.

Atlassian Bitbucket



Fuente: https://es.wikipedia.org/wiki/Soubor:Atlassian_Bitbucket_Logo.png



El experto opina

Esta es, posiblemente, la opción que más nos gusta.

Atlassian es una gran compañía famosa por desarrollar muy buen *software*.

Bitbucket es un repositorio de código. Tiene las siguientes características:

- Ofrece un importador de GitHub rápido y eficaz.
- Es gratuito tanto en repositorios públicos como privados:
 - En los privados pueden trabajar un máximo de cinco personas solamente.
 - Tiene planes de pago para más usuarios.



- Funciona desde el navegador.
- Trabaja con:
 - Git.
 - Mercurial.
- Los usuarios puedan dejar análisis del código.
- Seguimiento de tareas y problemas.
- Permiso de acceso a las diferentes ramas de modificaciones.
- Búsqueda y comparación de código.
- Disponible en español.

Kubernetes



kubernetes

Fuente: https://es.wikipedia.org/wiki/Archivo:Kubernetes_logo.svg

Kubernetes es una plataforma portable y extensible que facilita la automatización y la configuración declarativa para administrar cargas de trabajo y servicios, destacando su gestión de contenedores.

Es un sistema de código libre para la automatización del despliegue, ajuste de escala y manejo de aplicaciones en contenedores.

Kubernetes, tiene un ecosistema grande y en rápido crecimiento. El soporte, las herramientas y los servicios para Kubernetes están ampliamente disponibles.

Es de código abierto, con un rápido crecimiento, ya que se basa en la experiencia de Google, y a las mejores ideas y prácticas de la comunidad.

Kubernetes fue liberado por Google en el año 2014.



Anécdota

El nombre en clave original para Kubernetes dentro de Google, era "Project Seven" (en español "Proyecto Siete"), una referencia a un personaje de Star Trek. Los siete radios en la rueda del logo de Kubernetes es una referencia al nombre en clave.

8.2. Archivos de documentación comunes en las plataformas

A continuación mostramos la lista de archivos de documentación más comunes utilizados en proyectos de software, ordenados por frecuencia y relevancia. Se usan en la mayoría de los repositorios de software en plataformas de desarrollo colaborativo como GitHub, GitLab, Bitbucket, y otras plataformas similares. Sin embargo, no todos estos archivos son obligatorios en todos los proyectos, y su presencia depende en gran medida de las necesidades del proyecto, del equipo de desarrollo y de la plataforma utilizada.

- **README.md:** Archivo fundamental para cualquier proyecto. Contiene información crucial como el propósito del proyecto, cómo instalarlo, cómo usarlo, cómo contribuir y detalles generales sobre el funcionamiento.
- **LICENSE:** Especifica los términos de la licencia bajo la cual se distribuye el proyecto. Es importante para determinar cómo otros pueden usar, modificar y distribuir el código.
- **CITATION:** Archivo que generalmente se utiliza en proyectos académicos o de investigación, proporcionando detalles sobre cómo citar el proyecto correctamente.
- **CONTRIBUTING.md:** Establece las pautas para que los contribuyentes sepan cómo pueden involucrarse en el proyecto, cómo hacer pull requests, las reglas de estilo y el código de conducta.
- **CODEOWNERS:** Especifica qué partes del código son responsabilidad de qué personas o equipos. Facilita la revisión del código y ayuda a gestionar el flujo de contribuciones.
- **CHANGELOG.md:** Archivo que contiene un registro de los cambios y mejoras realizadas en el proyecto a lo largo del tiempo. Es útil para mantener un historial detallado de las versiones del proyecto.
- **ARCHITECTURE.md:** Proporciona información sobre la arquitectura del proyecto, cómo están estructurados los componentes y cómo interactúan entre sí. Este archivo es importante para desarrolladores que desean comprender la estructura interna del sistema.
- **SECURITY.md:** Ofrece pautas sobre cómo manejar vulnerabilidades de seguridad en el proyecto, cómo reportar problemas de seguridad y prácticas recomendadas.



- **FAQ.md:** Un archivo con preguntas frecuentes y sus respuestas. Ayuda a resolver dudas comunes que los usuarios o desarrolladores puedan tener sobre el proyecto.
- **TROUBLESHOOTING.md:** Similar al archivo FAQ, pero enfocado específicamente en problemas técnicos que los desarrolladores o usuarios pueden encontrar al trabajar con el proyecto.

9. Bibliografía

- PRESMAN, R. S. *Ingeniería del software. Un enfoque práctico*. Editorial McGraw Hill, 2010.
- http://users.dsic.upv.es/asignaturas/eui/mtp/doc-practicas/intro_case_SA.pdf.
- <http://picazomarianaha.blogspot.com/2013/11/i-case.html>.
- https://www.ecured.cu/Herramienta_CASE.
- https://www.tutorialspoint.com/es/software_engineering/case_tools_overview.htm.
- https://es.wikipedia.org/wiki/Magnolia_CMS.
- https://www.um.es/docencia/barzana/IAGP/Enlaces/CASE_principales.html.
- https://es.wikiversity.org/wiki/Herramientas_CASE.
- <https://es.wikipedia.org>.
- https://es.wikipedia.org/wiki/Integración_continua.
- [www.atlassian.com › software › bamboo](http://www.atlassian.com/software/bamboo).
- <https://es.wikipedia.org/wiki/SonarQube>.
- <https://www.ecured.cu/EasyCASE>.
- [https://es.wikipedia.org/wiki/Artefacto_\(diseño_de_software\)](https://es.wikipedia.org/wiki/Artefacto_(diseño_de_software)).
- [https://es.wikipedia.org/wiki/Pruebas_de_aceptación_\(informática\)](https://es.wikipedia.org/wiki/Pruebas_de_aceptación_(informática)).
- https://es.wikipedia.org/wiki/Pruebas_de_regresión.
- https://es.wikipedia.org/wiki/Gestión_de_configuración_de_software.
- <http://www.itcelaya.edu.mx/ojs/index.php/pistas/article/view/1304>.
- <http://certified-es.blogspot.com/2009/11/eleva-el-nivel-de-abstraccion.html>.



- <https://www.obs-edu.com/es/blog-project-management/diagramas-de-gantt/microsoft-visio-analisis-del-software>.
- <http://www.gobiernodigital.gob.pe/publica/metodologias/Lib5083/cap2015.HTM>.
- <http://mami.uclm.es/rhervas/images/downloads/HyEP-Tema2-TecCASE-Part1Low.pdf>.
- <http://www.profesordeinformatica.com/descargas/controldeversiones.pdf>.
- <http://nicolassoarez.blogspot.es/1431379803/plataforma-de-desarrollo-colaborativo-de-software/>.
- <https://computerhoy.com/reportajes/tecnologia/5-mejores-alternativas-github-2018-275973>.
- https://es.wikipedia.org/wiki/Acuerdo_de_nivel_de_servicio.
- https://es.wikipedia.org/wiki/Sistema_de_monitorización.
- <https://www.acronis.com/es-es/articles/monitoring-tools/>.
- https://www.123emprende.com/www/book/adjuntos/829/Acta_de_reunion.pdf.
- http://es.wikipedia.org/wiki/Diagrama_de_Gantt.
- https://www.123emprende.com/www/book/adjuntos/829/Acta_de_reunion.pdf.
- https://administracionelectronica.gob.es/pae_Home/pae_Documentacion/pae_Metodolog/pae_Metrica_v3.html.
- http://es.wikipedia.org/wiki/Ingenier%C3%ADa_del_software.
- https://es.wikipedia.org/wiki/Ciclo_de_vida_del_lanzamiento_de_software.
- https://es.wikipedia.org/wiki/Desarrollo_%C3%A1gil_de_software.
- <https://uv-mdap.com/programa-desarrollado/bloque-i-el-ciclo-de-vida-del-proyecto/presentacion-del-ciclo-de-vida-del-proyecto/>.
- https://es.wikipedia.org/wiki/Testing_aleatorio.
- https://es.wikipedia.org/wiki/Prueba_unitaria.
- <https://www.redhat.com/es/topics/devops/what-is-application-lifecycle-management-alm>.
- <https://universidadalnus.com/diferencia-entre-agile-y-scrum/>.
- https://es.wikipedia.org/wiki/Systems_Development_Life_Cycle.



- [https://www.redhat.com/es/topics/devops/what-is-application-lifecycle-management-alm.](https://www.redhat.com/es/topics/devops/what-is-application-lifecycle-management-alm)
- [https://es.wikipedia.org/wiki/Desarrollo_en_cascada.](https://es.wikipedia.org/wiki/Desarrollo_en_cascada)
- [https://es.wikipedia.org/wiki/Desarrollo_en_espiral#cite_note-4.](https://es.wikipedia.org/wiki/Desarrollo_en_espiral#cite_note-4)
- [http://www.hanantek.com/win-win.](http://www.hanantek.com/win-win)
- [https://www.bbva.com/es/metodologia-agile-la-revolucion-las-formas-trabajo/.](https://www.bbva.com/es/metodologia-agile-la-revolucion-las-formas-trabajo/)
- [https://azure.microsoft.com/es-es/services/devops/server/.](https://azure.microsoft.com/es-es/services/devops/server/)
- [https://es.wikipedia.org/wiki/Capability_Maturity_Model_Integration.](https://es.wikipedia.org/wiki/Capability_Maturity_Model_Integration)
- [https://es.wikipedia.org/wiki/Scrum_\(desarrollo_de_software\).](https://es.wikipedia.org/wiki/Scrum_(desarrollo_de_software))